

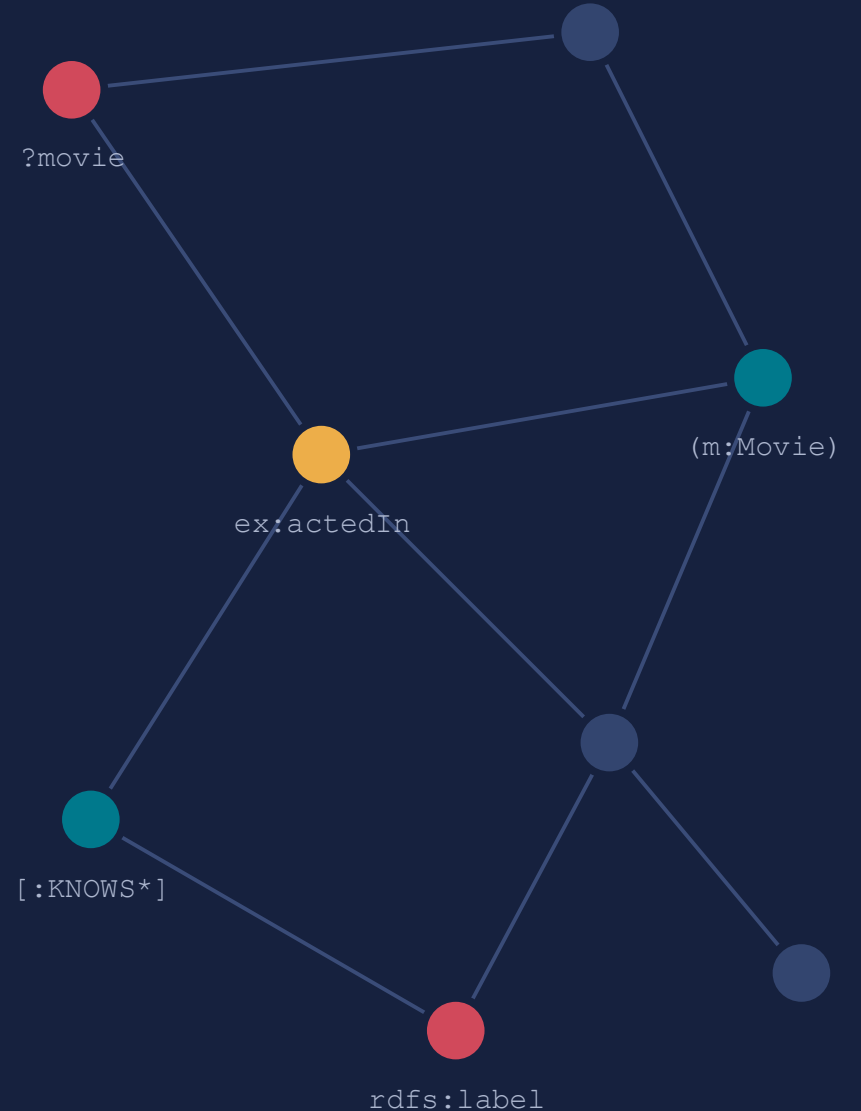
SPARQL & Cypher

Querying knowledge graphs and property graphs: Production-grade patterns, performance and graph ML.

SPARQL 1.1 · RDF · SHACL

Cypher · openCypher · GQL

Side-by-side mastery





SPARQL / RDF

Raspberry marks everything from the W3C semantic-web stack: RDF, Turtle, SPARQL, SHACL.



Cypher / Property graphs

Teal marks the labeled-property-graph world: Cypher, openCypher, GQL, Neo4j.



Both / comparison

Purple marks side-by-side material that maps one language onto the other.

```
SELECT ?title WHERE {
1  ex:keanu ex:actedIn ?movie .
2  ?movie  ex:title   ?title .
}
```

SPARQL

1

Numbered **badges** in the code gutter point to the matching numbered callout.

2

Highlighted lines are the ones the callout explains. Inline code in prose looks like `?movie.`

CALLOUT BOX KINDS



TIP

Practical advice you can apply immediately.



PITFALL

A mistake that silently gives wrong results or slow queries.



INSIGHT

The underlying semantics — why it behaves this way.



PRO MOVE

Production-grade technique used by experienced engineers.



RULE OF THUMB

A decision heuristic.

Roadmap: seven parts, one continuous learning path

Each part builds on the previous one. Checkpoint quizzes close every part.



RDF & Semantic-Web Foundations

Triples, IRIs, literals, Turtle, named graphs, RDFS/OWL, vocabularies



SPARQL Fundamentals

Graph patterns, FILTER, OPTIONAL, UNION, negation, aggregation, subqueries



Advanced SPARQL

Property paths, CONSTRUCT, federation, Wikidata, Update, RDF-star, SHACL, optimisation



Property Graphs & Cypher Fundamentals

Patterns, MATCH, WHERE, WITH, lists & maps, MERGE, writes



Advanced Cypher

Paths, QPP, subqueries, indexes, plans, batching, GDS, vectors & GraphRAG



Side by Side

Concept mapping, eight head-to-head translations, semantics, interop, choosing



Mastery

Capstone projects with full solutions, cheat sheets, glossary, resources



RULE OF THUMB

Beginners: go linearly. Practitioners: skim I and IV, then dive into III, V and VI. Everyone: do the checkpoints before moving on.

Why dedicated graph query languages?

The same question "who do Keanu's friends know?" in three languages. Relationships are first-class in graph languages.

SQL (relational)

```
-- two self-joins per hop
SELECT DISTINCT p3.name
FROM person p1
JOIN knows k1 ON k1.src = p1.id
JOIN knows k2 ON k2.src = k1.dst
JOIN person p3 ON p3.id = k2.dst
WHERE p1.name = 'Keanu Reeves';
```

SQL

Each extra hop adds joins; "any number of hops" needs a recursive CTE.

SPARQL (RDF)

```
SELECT DISTINCT ?name WHERE {
  ?p ex:name "Keanu Reeves" ;
  ex:knows/ex:knows ?fof .
  ?fof ex:name ?name .
}
```

SPARQL

A property path `ex:knows/ex:knows` expresses two hops;
`ex:knows+` means any number.

Cypher (property graph)

```
MATCH (:Person {name:'Keanu Reeves'})
-[:KNOWS*2]->(fof)
RETURN DISTINCT fof.name
```

CYPHER

The pattern is drawn as ASCII art; `*2` fixes the hop count, `*` leaves it open.

INSIGHT

Graph languages make **traversal** declarative: you describe the *shape* of the sub-graph you want and the engine finds all matches. Joins become implicit through shared variables or connected patterns.

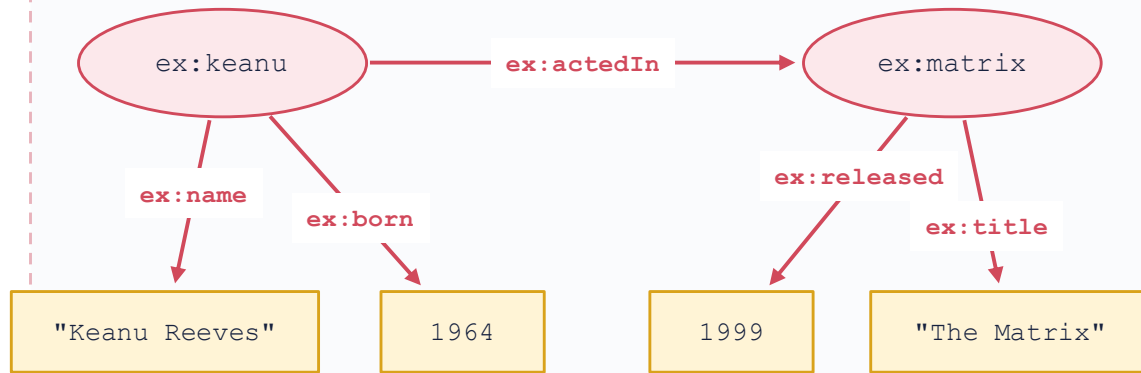
NOTE

On our running dataset all three return **Laurence Fishburne** (Keanu → Carrie-Anne → Laurence). Chad Stahelski, Keanu's other contact, knows nobody.

Two graph data models: RDF vs. labeled property graph

Same facts, two philosophies. RDF decomposes everything into triples; LPG groups properties inside nodes and edges.

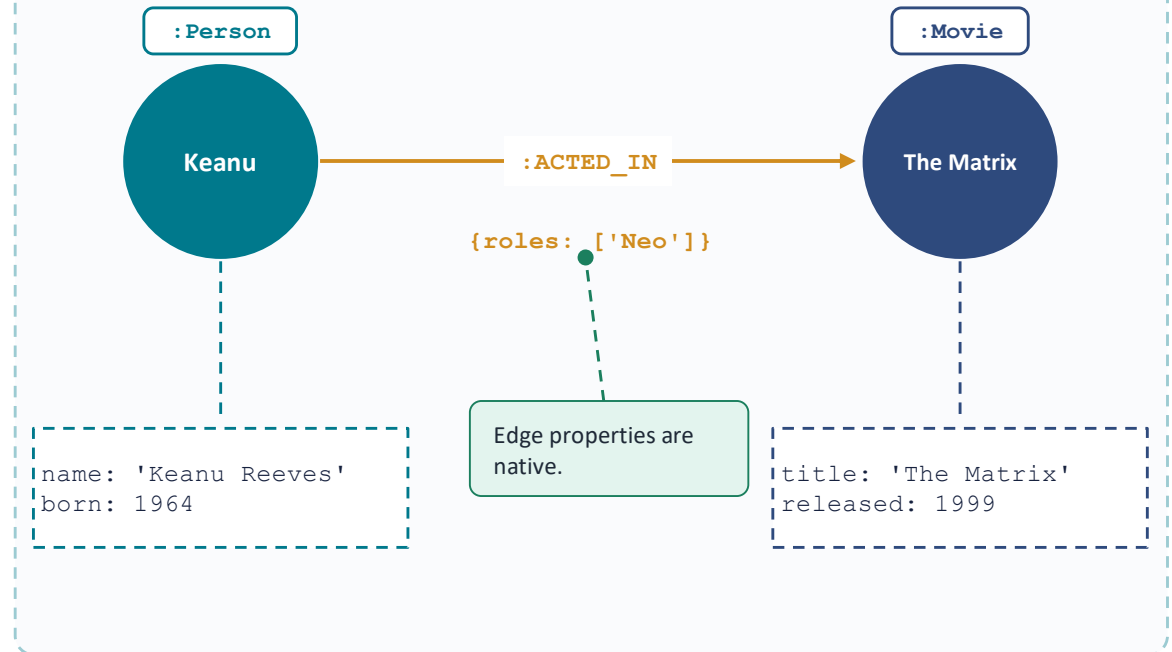
RDF — EVERYTHING IS A TRIPLE



Where does the role "Neo" go? It describes the *edge* keanu→matrix, and a plain RDF edge is not a node. Solutions (n-ary nodes, RDF-star) come in Part III.

RDF: nodes are IRIs, literals or blank nodes; edges are IRIs too. Globally unique identifiers make **merging datasets from different sources trivial**. Schema lives in ontologies (RDFS/OWL) and validation in SHACL.

LABELLED PROPERTY GRAPH — RICH NODES & EDGES



LPG: nodes carry labels + key/value properties; relationships have one type, a direction and their own properties. Optimised for **deep traversal and application development**; identifiers are local to the database.

The two languages at a glance

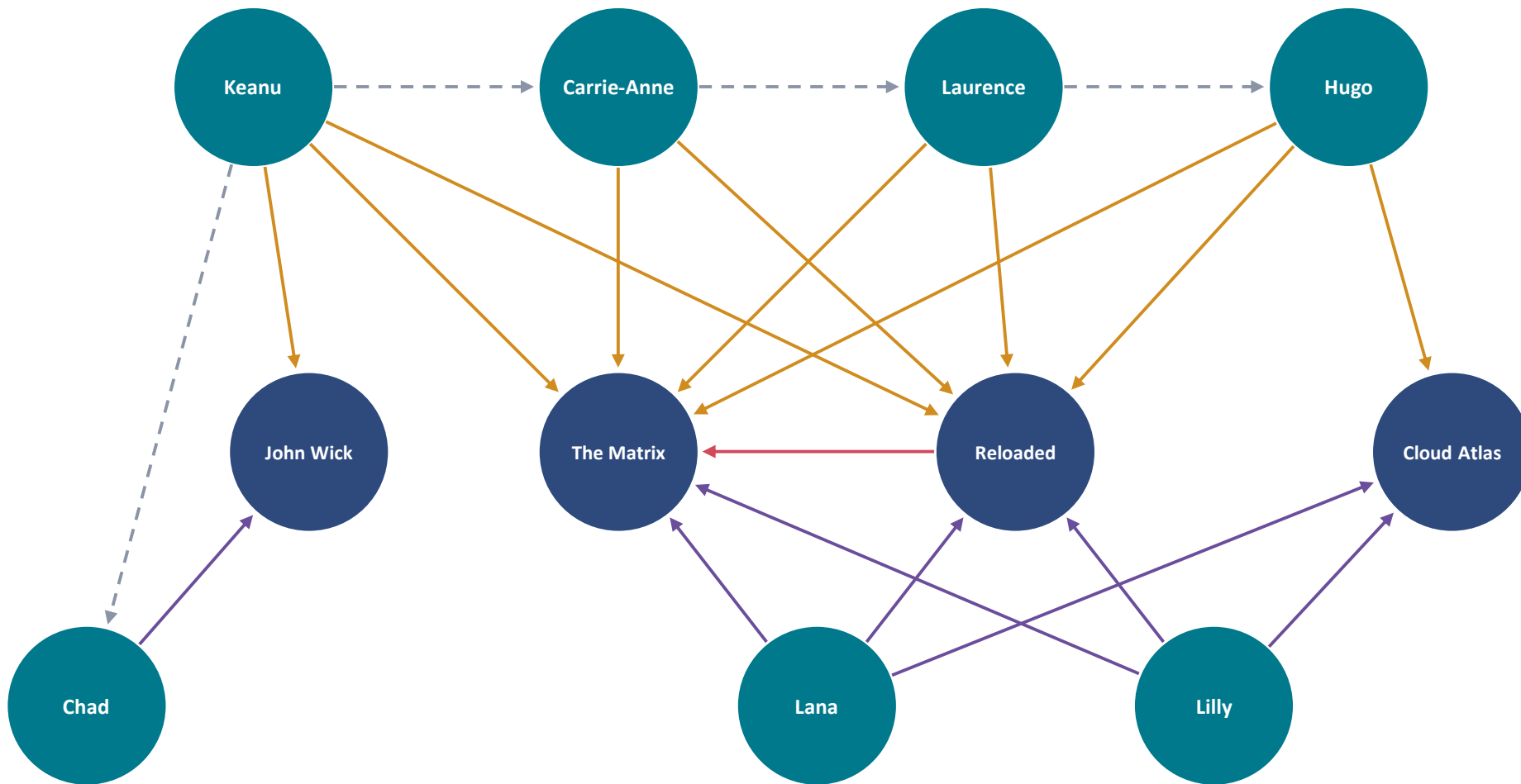
Dimension	SPARQL	Cypher
Data model	RDF: triples (subject, predicate, object) grouped into graphs; quads add a graph name	Labeled property graph: nodes with labels & properties, typed directed relationships with properties
Standard	W3C Recommendation — SPARQL 1.0 (2008), SPARQL 1.1 (2013); SPARQL 1.2 in progress alongside RDF 1.2	Created at Neo4j (2011); openCypher (2015); pattern core adopted by ISO GQL (ISO/IEC 39075:2024)
Query style	Declarative triple patterns joined through shared <code>?variables</code>	Declarative ASCII-art patterns <code>(a) -[:R]-> (b)</code> composed in a clause pipeline
Identity	Global IRIs — the same IRI means the same thing everywhere on the Web	Internal element ids + your own keys enforced by constraints
Schema	Optional; ontologies (RDFS/OWL) add inference, SHACL adds validation	Optional; indexes and constraints (uniqueness, existence, type, keys)
Typical engines	Apache Jena/Fuseki, GraphDB, Stardog, Virtuoso, QLever, Oxigraph, Amazon Neptune	Neo4j, Memgraph, Amazon Neptune (openCypher), Apache AGE, FalkorDB
Sweet spot	Data integration, linked open data, ontologies, publishing, reasoning	Operational graph apps, path analytics, recommendations, fraud, graph data science



These are **complementary**, not rival, technologies. Many platforms (e.g. Amazon Neptune) expose both, and bridges such as Neo4j's neosemantics import and export RDF. Part VI maps one onto the other concept by concept.

The running example: a small film knowledge graph

7 people, 4 films, 22 relationships — small enough to verify every result by hand, rich enough for joins, paths and aggregation.



LEGEND

- ACTED_IN**
- DIRECTED**
- KNOWS**
- SEQUEL_OF**

People: Keanu Reeves (1964), Carrie-Anne Moss (1967), Laurence Fishburne (1961), Hugo Weaving (1960), Lana (1965) & Lilly Wachowski (1967), Chad Stahelski (1968).

Films: The Matrix (1999, 8.7), Reloaded (2003, 7.2), Cloud Atlas (2012, 7.4), John Wick (2014, 7.4).

Your lab: engines and tools to follow along

Everything in this deck runs on free tooling. Pick one SPARQL engine and one Cypher engine.



Apache Jena Fuseki

Open-source SPARQL 1.1 server with a web UI and TDB2 storage.

`docker run -p 3030:3030 stain/jena-fuseki` (community image) or run the standalone server JAR.



GraphDB · Oxigraph · QLever

GraphDB Free adds RDFS/OWL reasoning; Oxigraph is a tiny Rust store; QLever handles billions of triples on one machine.



Wikidata Query Service

Zero-install practice on >10 billion triples at query.wikidata.org. Built-in examples, map and chart views.



Neo4j (Desktop / Aura / Docker)

`docker run -p 7474:7474 -p 7687:7687 -e NEO4J_AUTH=neo4j/password neo4j:5` then open `localhost:7474`. AuraDB Free runs in the cloud.



Memgraph & others

Memgraph is an in-memory openCypher engine; Apache AGE adds Cypher to PostgreSQL; Amazon Neptune supports openCypher.



Python clients

`pip install rdflib SPARQLWrapper neo4j` — used in Parts III and V to run both languages from notebooks and pipelines.



TIP

Load the running dataset twice — the Turtle version (slide 17) into your SPARQL store and the Cypher CREATE script (Part IV) into Neo4j — then run every example as you read.

01

PART I

RDF & Semantic-Web Foundations

Before you can query RDF you must think in triples. This part builds the data model SPARQL operates on.

Triples

IRIs & prefixes

Literals & datatypes

Blank nodes

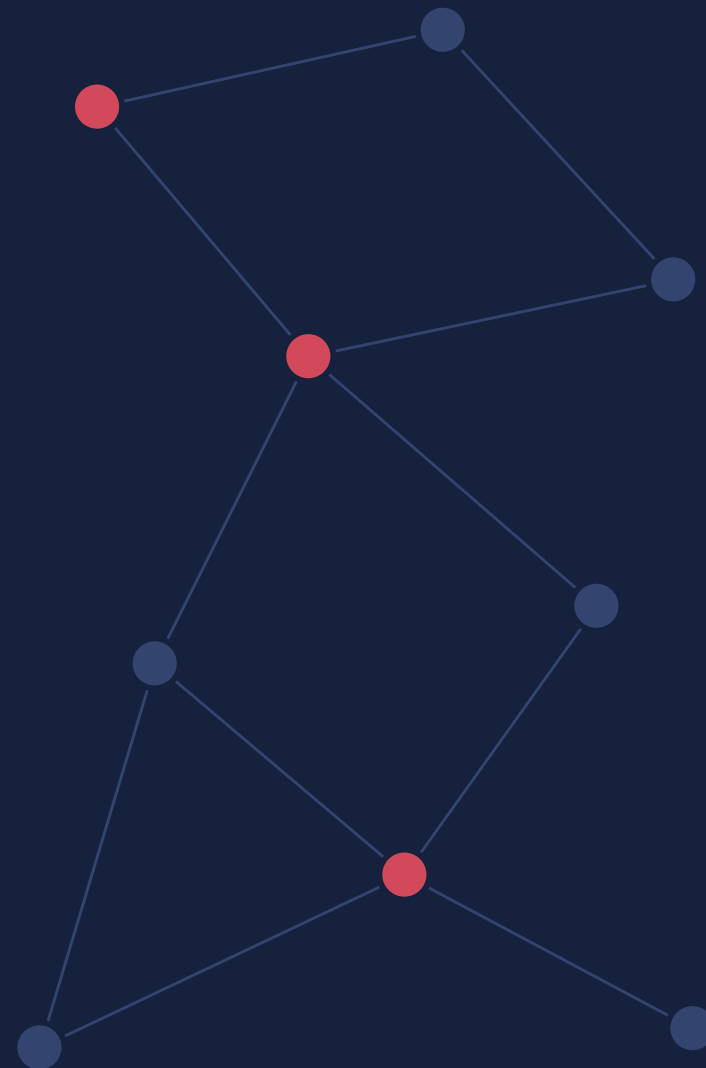
Turtle

Named graphs

RDFS

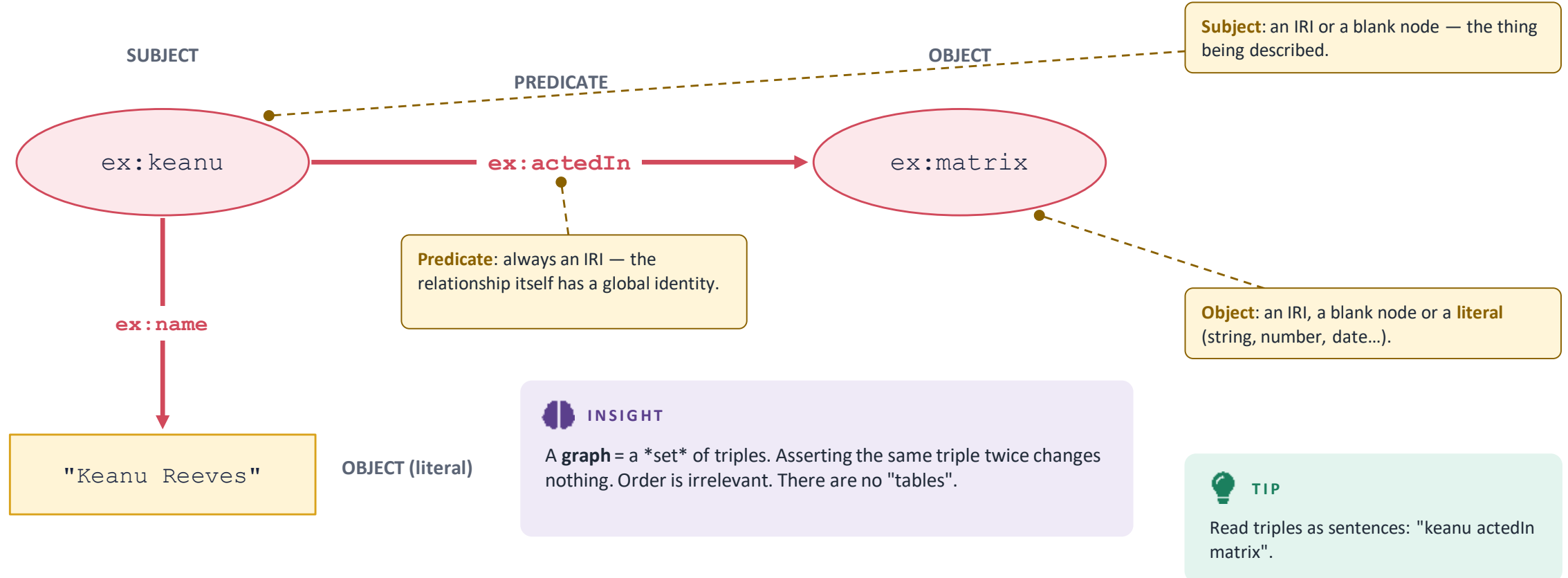
OWL

Vocabularies



The atom of RDF: the triple

Every statement is *subject* → *predicate* → *object*. A graph is simply a set of triples.



The three kinds of RDF terms

Every node in an RDF graph is exactly one of these. Knowing which one you are matching is half of writing correct SPARQL.



IRIs

Internationalised Resource Identifiers name things globally: `<http://example.org/keanu>`.

- Two datasets using the same IRI talk about the same thing
- May be dereferenceable (Linked Data), but need not be
- Written in full `<...>` or abbreviated `ex:keanu`



Literals

Values: a lexical form + a datatype IRI (+ language tag for text).

- `"Keanu Reeves" → xsd:string`
- `1964 → xsd:integer`
- `"Matrix"@de → rdf:langString`
- Literals can only appear in the **object** position



Blank nodes

Anonymous nodes: `_:b1` or `[ ... ]` in Turtle.

- Existential: "there exists something that..."
- Labels are local to one document/store
- Useful for structured values (addresses, measurements), but hard to reference later



PITFALL

Blank nodes cannot be addressed from outside the document: a SPARQL query cannot say "give me `_:b1`". If you will ever need to link to a thing, give it an IRI (skolemise).

Namespaces and prefixes

Prefixes are pure textual abbreviations — they expand by string concatenation before anything else happens.

```
1 @prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
  @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
2 @prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
3 @prefix ex: <http://example.org/> .
4 @base <http://example.org/> .

# Three spellings of the SAME subject IRI
5 <http://example.org/keanu> ex:name "Keanu Reeves" .
6 ex:keanu ex:born 1964 .
7 <keanu> a ex:Person .
```

TURTLE



PITFALL

A prefix without a trailing / or # glues words together: `ex: <http://example.org>` makes `ex:keanu` = `http://example.orgkeanu`.

- 1 `rdf:` is the core vocabulary: `rdf:type`, `rdf:Property`, `rdf:langString`...
- 2 `xsd:` provides datatypes: `xsd:integer`, `xsd:date`, `xsd:decimal`...
- 3 `ex:` is *our* namespace. The trailing / matters: `ex:keanu` → `http://example.org/keanu`.
- 4 `@base` resolves relative IRIs such as `<keanu>`.
- 5 Full IRI in angle brackets.
- 6 Prefixed name — identical after expansion.
- 7 `a` is Turtle/SPARQL shorthand for `rdf:type`.

Turtle, step 1: one triple per line

Turtle (Terse RDF Triple Language) is the human-friendly serialisation used throughout this deck.

```
@prefix ex: <http://example.org/> .
```

```
1 ex:keanu      a          ex:Person .
2 ex:keanu      ex:name     "Keanu Reeves" .
3 ex:keanu      ex:born     1964 .
4 ex:keanu      ex:actedIn  ex:matrix .
  ex:matrix     a          ex:Movie .
  ex:matrix     ex:title    "The Matrix" .
  ex:matrix     ex:released 1999 .
5 ex:matrix     ex:rating   8.7 .
```

TURTLE

- 1 Every statement ends with a **full stop**. Whitespace and alignment are cosmetic.
- 2 Quoted text is an `xsd:string` literal.
- 3 A bare integer is shorthand for `"1964"^^xsd:integer`.
- 4 An IRI object creates an **edge** to another node.
- 5 `8.7` is an `xsd:decimal`; `8.7e0` would be an `xsd:double`.

THE SAME DATA AS A TRIPLE TABLE (FIRST 4 ROWS)

subject	predicate	object
ex:keanu	rdf:type	ex:Person
ex:keanu	ex:name	"Keanu Reeves"
ex:keanu	ex:born	1964
ex:keanu	ex:actedIn	ex:matrix

Turtle, step 2: predicate lists and object lists

Two punctuation marks remove almost all repetition: ";" repeats the subject, "," repeats subject + predicate.

```
@prefix ex: <http://example.org/> .
```

```
1 ex:keanu a ex:Person ;  
  ex:name   "Keanu Reeves" ;  
  ex:born   1964 ;  
2   ex:actedIn ex:matrix ,  
      ex:reloaded ,  
3   ex:johnwick .
```

TURTLE

1 ; ends a predicate–object pair and **keeps the subject** for the next one.

2 , keeps **subject and predicate**, adding another object.

3 A single . closes the whole group.



TIP

SPARQL uses the **same** ; and , syntax inside `WHERE { }`, so fluency in Turtle transfers directly to queries.

EXPANDS TO EXACTLY 6 TRIPLES

subject	predicate	object
ex:keanu	rdf:type	ex:Person
ex:keanu	ex:name	"Keanu Reeves"
ex:keanu	ex:born	1964
ex:keanu	ex:actedIn	ex:matrix
ex:keanu	ex:actedIn	ex:reloaded
ex:keanu	ex:actedIn	ex:johnwick

Literals in depth: datatypes and language tags

A literal is a lexical form plus a datatype IRI. Language-tagged strings carry a BCP 47 tag instead.

```
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
```

```
1 ex:matrix ex:title      "The Matrix"@en ,
                                "Matrix"@de ;
2 ex:releaseDate "1999-03-31"^^xsd:date ;
3 ex:rating      "8.7"^^xsd:decimal ;
4 ex:runtime     "PT2H16M"^^xsd:duration ;
5 ex:isCult      true .
```

TURTLE



PITFALL

"1999" (a string) and 1999 (an integer) are **different terms**. `FILTER(?y > 1998)` fails on the string version — datatypes must be consistent across your data.



INSIGHT

Value vs. term equality: 8.7 and 8.70 are different terms but equal **values**, so `FILTER(?r = 8.70)` matches while the triple pattern `?m ex:rating 8.70` may not (store-dependent canonicalisation).

- 1 `@en` makes an `rdf:langString`. Same text, different tag = **different literal**.
- 2 `^^` attaches an explicit datatype. Dates enable range filters and date functions.
- 3 `"8.7"^^xsd:decimal` is exactly what the shorthand `8.7` produces.
- 4 ISO-8601 duration: 2 h 16 min (136 minutes).
- 5 `true` / `false` are `xsd:boolean` shorthands.

Blank nodes and RDF collections

Anonymous structure: square brackets create a fresh blank node; parentheses create an ordered list.

```
@prefix ex: <http://example.org/> .
```

- 1

```
ex:matrix ex:budget [  
    ex:amount    63000000 ;  
    ex:currency  "USD"  
] .
```
- 2

```
_:loc ex:city "Sydney" .  
ex:matrix ex:filmedIn _:loc .
```
- 3

```
ex:matrix ex:genres ( "Sci-Fi" "Action" ) .
```

TURTLE

1 [...] = an unnamed node; the triples inside describe it. Great for compound values such as money.

2 Labelled blank node `_:loc` — the label lets you refer to it twice **within this document only**.

3 (...) builds an `rdf:first` / `rdf:rest` chain ending in `rdf:nil` — the only **ordered** structure in core RDF.

INSIGHT

The list on line 11 expands to **5 triples** and 2 blank nodes. Querying lists requires the path `rdf:rest*/rdf:first` (Part III).

PITFALL

`_:loc` in two different files are two different nodes. Merging files renames blank nodes apart.

The running dataset in Turtle (movies.ttl)

Load this file into your SPARQL store — every SPARQL result in Parts II and III is computed from it.

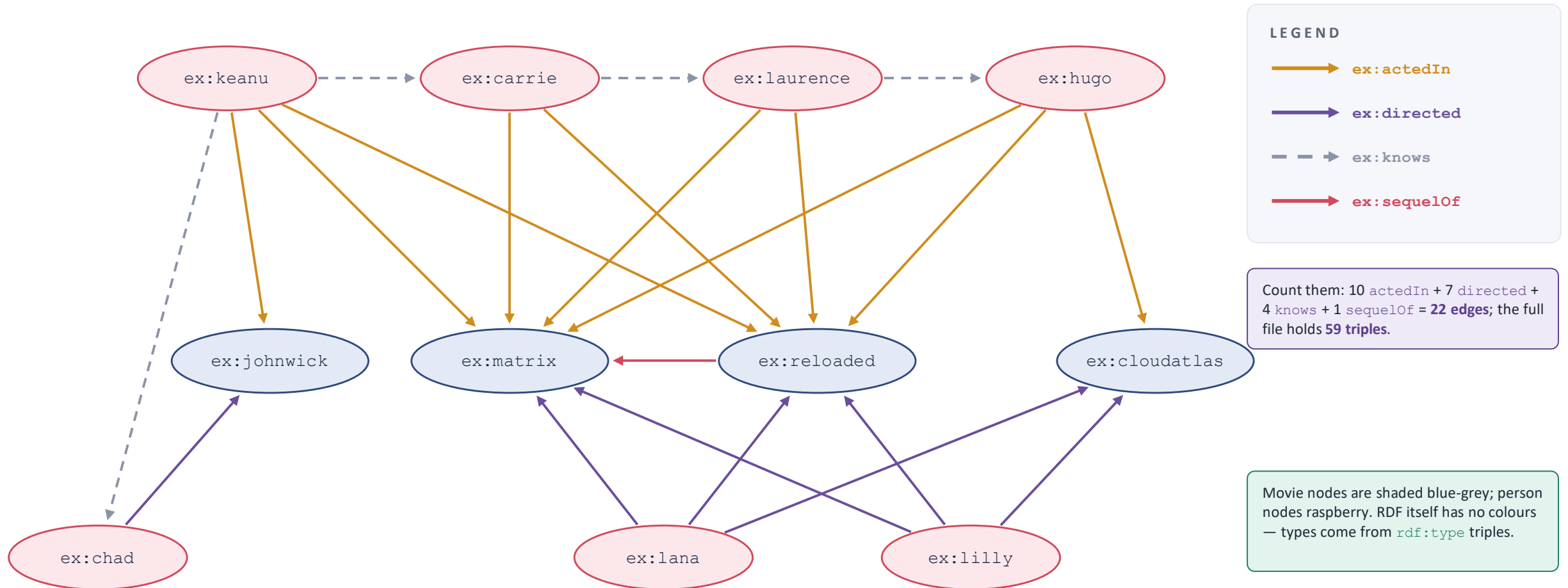
```
@prefix ex: <http://example.org/> .

ex:keanu      a ex:Person ; ex:name "Keanu Reeves"          ; ex:born 1964 ;
              ex:actedIn ex:matrix, ex:reloaded, ex:johnwick ; ex:knows ex:carrie, ex:chad .
ex:carrie     a ex:Person ; ex:name "Carrie-Anne Moss"       ; ex:born 1967 ;
              ex:actedIn ex:matrix, ex:reloaded ; ex:knows ex:laurence .
ex:laurence   a ex:Person ; ex:name "Laurence Fishburne"     ; ex:born 1961 ;
              ex:actedIn ex:matrix, ex:reloaded ; ex:knows ex:hugo .
ex:hugo       a ex:Person ; ex:name "Hugo Weaving"           ; ex:born 1960 ;
              ex:actedIn ex:matrix, ex:reloaded, ex:cloudatlas .
ex:lane       a ex:Person ; ex:name "Lana Wachowski"         ; ex:born 1965 ;
              ex:directed ex:matrix, ex:reloaded, ex:cloudatlas .
ex:lilly      a ex:Person ; ex:name "Lilly Wachowski"        ; ex:born 1967 ;
              ex:directed ex:matrix, ex:reloaded, ex:cloudatlas .
ex:chad       a ex:Person ; ex:name "Chad Stahelski"          ; ex:born 1968 ;
              ex:directed ex:johnwick .

ex:matrix     a ex:Movie ; ex:title "The Matrix"             ; ex:released 1999 ; ex:rating 8.7 .
ex:reloaded   a ex:Movie ; ex:title "The Matrix Reloaded"    ; ex:released 2003 ; ex:rating 7.2 ;
              ex:sequelOf ex:matrix .
ex:johnwick   a ex:Movie ; ex:title "John Wick"              ; ex:released 2014 ; ex:rating 7.4 .
ex:cloudatlas a ex:Movie ; ex:title "Cloud Atlas"            ; ex:released 2012 ; ex:rating 7.4 .
```

The same dataset drawn as an RDF graph

IRIs are nodes, predicates are edges. Literal-valued triples (names, years, ratings) are omitted here for clarity.



One model, many serialisations

Syntax is interchangeable; the graph is the same. Choose by tooling and use case.

Format	Media type	Ext.	Best for
Turtle	text/turtle	.ttl	Humans, hand-written data, docs
N-Triples	application/n-triples	.nt	Streaming, line-based bulk loads, diffing
N-Quads	application/n-quads	.nq	Bulk loads with named graphs
TriG	application/trig	.trig	Turtle + named graphs
JSON-LD	application/ld+json	.jsonld	Web APIs, schema.org markup, JSON developers
RDF/XML	application/rdf+xml	.rdf	Legacy tooling, OWL files
RDF 1.2 variants	Turtle/N-Triples 1.2	—	Triple terms / annotations (RDF-star)

N-TRIPLES EXAMPLE

One absolute triple per line, no prefixes:
`<http://example.org/keanu>
<http://example.org/born>
"1964"^^<http://www.w3.org/2001/XMLSchema#integer> .`

PRO MOVE

For multi-GB loads use N-Triples/N-Quads: they split trivially across workers and any bad line fails alone. Most stores ship a bulk loader faster than SPARQL [LOAD](#).

Named graphs: from triples to quads

A dataset = one default graph + any number of named graphs. The graph name is itself an IRI you can describe.

```
@prefix ex: <http://example.org/> .  
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
```

1 `ex:keanu ex:name "Keanu Reeves" .`

2 `ex:sourceA {
 ex:matrix ex:rating 8.7 .
}`

3 `ex:sourceB {
 ex:matrix ex:rating 8.1 .
}`

4 `ex:sourceA ex:retrieved "2026-01-15"^^xsd:date .`

TRIG

1 Outside any braces → the **default graph**.

2 Inside `ex:sourceA { }` → the quad `(ex:matrix, ex:rating, 8.7, ex:sourceA)`.

3 Conflicting claims coexist safely in separate graphs.

4 Provenance: statements *about the graph* (source, date, licence, version).

NOTE

The values 8.7 / 8.1 are illustrative source ratings used later in the named-graph queries.

INSIGHT

Named graphs are the standard way to track **provenance, versions, access control and trust** in RDF. SPARQL addresses them with `FROM`, `FROM NAMED` and `GRAPH` (Part III).

RDFS: a lightweight schema that drives inference

RDF Schema describes classes and properties. Its semantics is inference — not validation.

```
@prefix ex:    <http://example.org/> .
@prefix rdfs:  <http://www.w3.org/2000/01/rdf-schema#> .
```

```
1 ex:Actor      rdfs:subClassOf    ex:Person .
2 ex:Director   rdfs:subClassOf    ex:Person .
3 ex:actedIn    rdfs:domain        ex:Actor ;
4              rdfs:range         ex:Movie ;
5              rdfs:label         "acted in"@en ;
              rdfs:comment        "Links a performer to a film."@en .
ex:directed     rdfs:subPropertyOf ex:contributedTo .
```

TURTLE



PITFALL

With `rdfs:range ex:Movie`, the triple `ex:keanu ex:actedIn ex:chad` does **not** raise an error — a reasoner concludes Chad is a Movie! Use SHACL (Part III) when you want constraints.

- 1 Every `ex:Actor` is inferred to be an `ex:Person`.
- 2 domain: whoever has `ex:actedIn` is inferred to be an `ex:Actor`.
- 3 range: whatever is the object is inferred to be an `ex:Movie`.
- 4 Human-readable labels & comments — use them for UIs and documentation.
- 5 Each `ex:directed` triple entails an `ex:contributedTo` triple.

OWL in one slide: richer semantics when you need them

The Web Ontology Language adds logic on top of RDFS. Use the smallest profile that meets your needs.



Identity

`owl:sameAs` declares two IRIs denote the same individual — the glue of Linked Data (e.g. your `ex:keanu` = a Wikidata item).



Property characteristics

`owl:inverseOf` (`actedIn` $\leftrightarrow$ `hasCastMember`),
`owl:TransitiveProperty`,
`owl:SymmetricProperty`,
`owl:FunctionalProperty`.



Class axioms

`owl:equivalentClass`, `owl:disjointWith`,
unions and intersections of classes.



Restrictions

`owl:someValuesFrom`, `owl:allValuesFrom`,
cardinalities — e.g. "a Director directed at least one Movie".



Profiles

OWL 2 EL (large ontologies, e.g. SNOMED CT), **QL**
(query rewriting over databases), **RL** (rule-based,
what most triple stores implement).



Open-world assumption

Missing information is **unknown**, not false. OWL
never concludes "Keanu has no email" from an
absent triple — a key difference from databases.

Vocabularies you will meet in the wild

Reuse before you invent: shared vocabularies are what make RDF data interoperable.

Prefix	Namespace IRI	Used for
rdf	http://www.w3.org/1999/02/22-rdf-syntax-ns#	Core: <code>rdf:type</code> , <code>lists</code> , <code>rdf:langString</code>
rdfs	http://www.w3.org/2000/01/rdf-schema#	Schema, <code>rdfs:label</code> , <code>rdfs:subClassOf</code>
owl	http://www.w3.org/2002/07/owl#	Ontologies, <code>owl:sameAs</code>
xsd	http://www.w3.org/2001/XMLSchema#	Literal datatypes
schema	https://schema.org/	General-purpose web vocabulary (Movie, Person, actor...)
foaf	http://xmlns.com/foaf/0.1/	People and social links (<code>foaf:knows</code> , <code>foaf:name</code>)
dcterms	http://purl.org/dc/terms/	Metadata: title, creator, date, licence
skos	http://www.w3.org/2004/02/skos/core#	Taxonomies & thesauri (<code>skos:broader</code> , <code>skos:prefLabel</code>)
prov	http://www.w3.org/ns/prov#	Provenance: <code>prov:wasDerivedFrom</code>
sh	http://www.w3.org/ns/shacl#	SHACL shapes for validation
wd / wdt	http://www.wikidata.org/entity/ · .../prop/direct/	Wikidata items and "truthy" properties

 **TIP**
schema.org is published under both <http://> and <https://>. Pick one and be consistent — the two are different IRIs to a SPARQL engine.

Checkpoint I — RDF foundations

Q1

How many triples does this Turtle produce?

```
ex:a ex:p ex:b, ex:c ; ex:q "x" .
```

✓ ANSWER

Three: (a p b), (a p c) and (a q "x"). The comma repeats subject + predicate; the semicolon repeats only the subject.

Q2

Can a literal ever be the **subject** of a triple?

✓ ANSWER

No. Subjects are IRIs or blank nodes (RDF 1.2 adds triple terms, but only in the object position). To describe a value, model it as a node.

Q3

Are "1999" and 1999 the same RDF term?

✓ ANSWER

No. The first is `xsd:string`, the second `xsd:integer`. They are different terms and compare differently in FILTERs.

Q4

`ex:title rdfs:domain ex:Movie` is declared and the data contains `ex:keanu ex:title "x"`. What happens?

✓ ANSWER

No error. An RDFS reasoner **infers** `ex:keanu a ex:Movie`. Domain/range are inference rules, not constraints — use SHACL to validate.

02

PART II

SPARQL Fundamentals

Graph pattern matching, filtering, optional data, alternatives, negation, sorting and aggregation — the 80% of SPARQL you will use every day.

Query anatomy

Basic graph patterns

FILTER

OPTIONAL

UNION

MINUS / NOT EXISTS

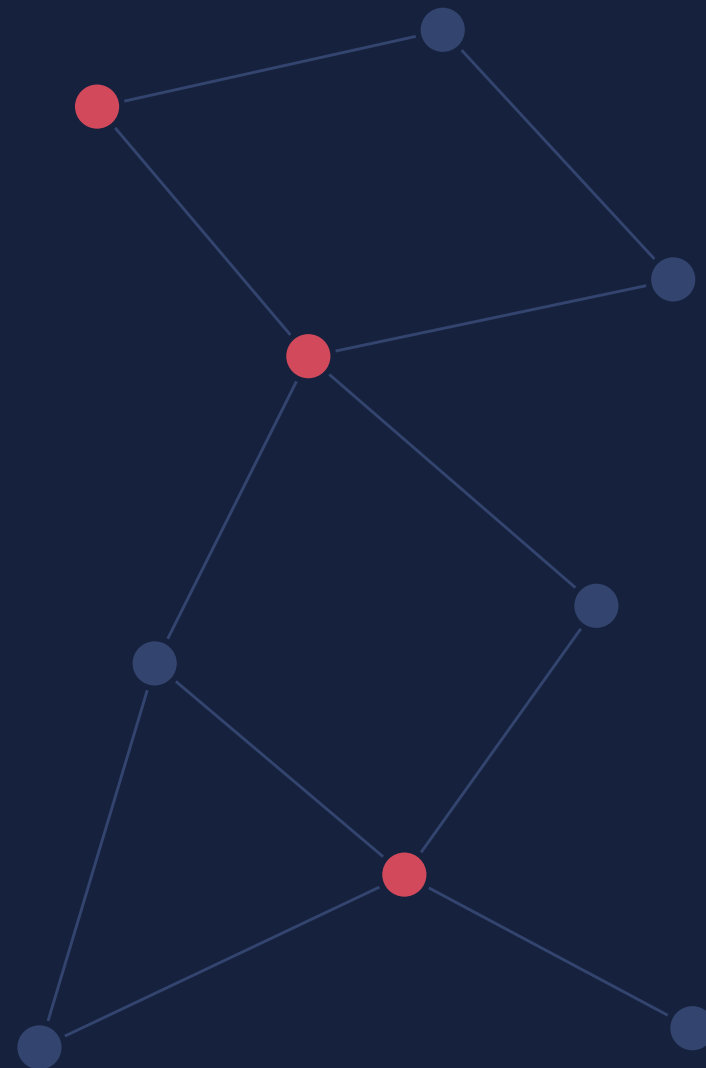
BIND

VALUES

GROUP BY

Subqueries

Datatypes



Anatomy of a SPARQL query

Five parts, always in this order: prologue, query form, dataset, pattern, solution modifiers.

```
1 PREFIX ex: <http://example.org/>
  PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

2 SELECT ?title ?year
3 FROM <http://example.org/movies>
  WHERE {
4   ?movie a ex:Movie ;
          ex:title    ?title ;
          ex:released ?year .
5  FILTER (?year >= 2000)
6  }
  ORDER BY DESC (?year)
  LIMIT 10
```

SPARQL

RESULT

?title	?year
John Wick	2014
Cloud Atlas	2012
The Matrix Reloaded	2003

- 1 Prologue:** `PREFIX` (and optional `BASE`) declarations — no trailing dot, unlike Turtle.
- 2 Query form + projection:** which variables appear in the result.
- 3 Dataset clause** (optional): which graph(s) to query. Omit it to use the store default.
- 4 WHERE:** the graph pattern — Turtle-like triple patterns with `?variables`.
- 5 FILTER** removes solutions whose expression is false or errors.
- 6 Solution modifiers:** `ORDER BY`, `LIMIT`, `OFFSET` (plus `GROUP BY` / `HAVING`).

The four query forms

Same WHERE clause, four different outputs. Choose the form by what your application consumes.



SELECT

Returns a **table** of variable bindings.

```
SELECT ?t WHERE { ?m  
  ex:title ?t }
```

Use for: apps, analytics, dataframes.



CONSTRUCT

Returns a new **RDF graph** built from a triple template.

```
CONSTRUCT { ?m a  
  schema:Movie } WHERE { ?m  
  a ex:Movie }
```

Use for: ETL, vocabulary mapping, derived edges.



ASK

Returns **true / false** — does any match exist?

```
ASK { ex:keanu ex:actedIn  
  ex:matrix }
```

Use for: checks, guards, tests.



DESCRIBE

Returns an **implementation-defined** graph about resources.

```
DESCRIBE ex:keanu
```

Use for: exploration; never rely on its exact shape.



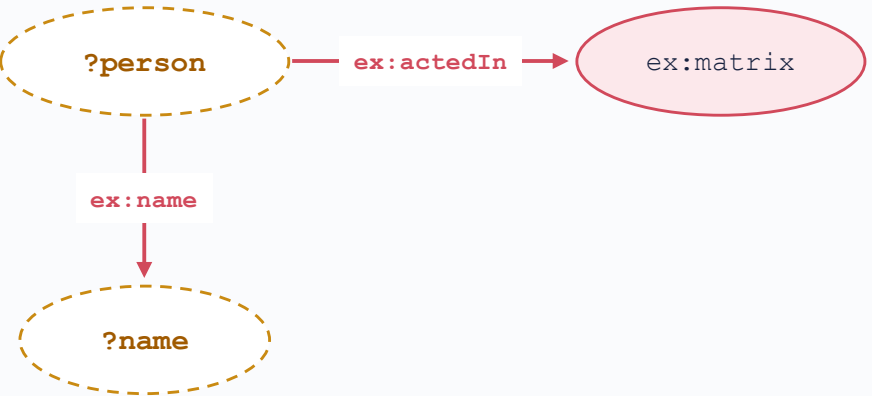
INSIGHT

Only SELECT returns tabular results. CONSTRUCT and DESCRIBE return RDF (Turtle, JSON-LD, N-Triples... via content negotiation); ASK returns a boolean in the SPARQL results format.

Basic graph patterns: matching a template against the graph

A BGP is a set of triple patterns. A solution is any assignment of variables that turns every pattern into a triple of the data.

THE PATTERN (WHERE CLAUSE)



```
SELECT ?person ?name WHERE {
  ?person ex:actedIn ex:matrix .
  ?person ex:name ?name .
}
```

SPARQL

SOLUTIONS (ONE ROW PER MATCH)

?person	?name
ex:keanu	Keanu Reeves
ex:carrie	Carrie-Anne Moss
ex:laurence	Laurence Fishburne
ex:hugo	Hugo Weaving

Variables (dashed) are **holes**; IRIs and literals are **fixed** and must match exactly.

The shared `?person` forces both patterns to bind the **same node** — that is a join.

The engine finds every place in the data graph where your template fits. Formally SPARQL uses **homomorphism**: two different variables may bind the same node.

Your first query: which films did Keanu act in?

A constant subject, a variable object — then a second pattern to fetch the title.

```
PREFIX ex: <http://example.org/>
```

```
1 SELECT ?title
   WHERE {
2   ex:keanu ex:actedIn ?movie .
3   ?movie ex:title ?title .
   }
```

SPARQL

1 Only `?title` is projected; `?movie` is used internally and then dropped.

2 Fixed subject + predicate → `?movie` binds to each matching object (3 solutions).

3 Same `?movie` → implicit **join**: each movie is extended with its title.

RESULT

?title
The Matrix
The Matrix Reloaded
John Wick

PITFALL

Result order is **undefined** without `ORDER BY`. Two runs on two stores may list the rows differently — never depend on it.

Joins through shared variables: Keanu's co-stars

Four patterns, three shared variables, one FILTER — the SPARQL equivalent of a 4-way SQL join.

```
1 SELECT DISTINCT ?coName
   WHERE {
2   ?keanu ex:name      "Keanu Reeves" .
3   ?keanu ex:actedIn ?movie .
4   ?co    ex:actedIn ?movie .
         ?co    ex:name   ?coName .
5   FILTER (?co != ?keanu)
   }
   ORDER BY ?coName
```

SPARQL

RESULT

?coName
Carrie-Anne Moss
Hugo Weaving
Laurence Fishburne

1 **DISTINCT**: Carrie-Anne, Laurence and Hugo co-starred in **two** films, so each would appear twice.

2 Anchor by **literal value** — no need to know the IRI.

3 **?movie** links Keanu to his films...

4 ...and back out to everyone else in the same films.

5 Homomorphism: without this FILTER, **?co** could equal Keanu himself.

Mental model: patterns produce tables, tables are joined

Each triple pattern yields a table of bindings; the BGP is their natural join on shared variables.

ex:keanu ex:actedIn ?movie

?movie
ex:matrix
ex:reloaded
ex:johnwick



?movie ex:title ?title

?movie	?title
ex:matrix	The Matrix
ex:reloaded	The Matrix Reloaded
ex:johnwick	John Wick
ex:cloudatlas	Cloud Atlas



JOIN ON ?movie

?movie	?title
ex:matrix	The Matrix
ex:reloaded	The Matrix Reloaded
ex:johnwick	John Wick



ex:cloudatlas has no partner on the left, so the inner join **drops** it.



INSIGHT

SPARQL algebra works on **multisets (bags)** of solution mappings — partial functions from variables to RDF terms. Duplicates survive joins; **DISTINCT** turns the bag into a set.



PRO MOVE

Engines reorder patterns freely — they are not executed top-to-bottom. Most start from the **most selective** pattern (here 3 rows) using triple indexes such as SPO, POS and OSP.

FILTER: keep only solutions that satisfy a condition

Filters test bound values with boolean expressions. They never create bindings; they only remove rows.

```
SELECT ?name ?born
WHERE {
  ?p a ex:Person ;
    ex:name ?name ;
    ex:born ?born .
  1  FILTER (?born >= 1962 && ?born < 1968)
  3  ORDER BY ?born ?name
}
```

SPARQL

RESULT

?name	?born
Keanu Reeves	1964
Lana Wachowski	1965
Carrie-Anne Moss	1967
Lilly Wachowski	1967

- 1 &&, || and ! combine conditions. Comparison works on numbers, strings, dates and booleans.
- 2 Scope: a FILTER applies to the **whole group** { ... } it appears in, wherever it is written.
- 3 Two sort keys: Carrie-Anne and Lilly were both born in 1967, so ?name breaks the tie.

INSIGHT

Three-valued logic: a type error (comparing a string with a number, an unbound variable...) makes the expression an **error**, and the row is dropped — just as if it were false.

Operators and built-in functions — the reference

Category	Operators/ functions	Example
Comparison	<code>=</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code> , <code>IN (...)</code> , <code>NOT IN (...)</code>	<code>FILTER(?y IN (1999, 2003))</code>
Logical	<code>&&</code> <code> </code> <code>!</code>	<code>FILTER(!(?r < 7))</code>
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> , unary minus	<code>BIND(?y - ?born AS ?age)</code>
Term tests	<code>isIRI</code> , <code>isBlank</code> , <code>isLiteral</code> , <code>isNumeric</code> , <code>bound</code> , <code>sameTerm</code>	<code>FILTER(!bound(?email))</code>
Accessors	<code>STR</code> , <code>LANG</code> , <code>DATATYPE</code> , <code>IRI</code> , <code>BNODE</code> , <code>STRDT</code> , <code>STRLANG</code>	<code>STR(?movie)</code>
Strings	<code>STRLEN</code> , <code>SUBSTR</code> , <code>UCASE</code> , <code>LCASE</code> , <code>STRSTARTS</code> , <code>STREND</code> , <code>CONTAINS</code> , <code>STRBEFORE</code> , <code>STRAFTER</code> , <code>CONCAT</code> , <code>REPLACE</code> , <code>REGEX</code> , <code>ENCODE_FOR_URI</code>	<code>CONCAT(?first, " ", ?last)</code>
Numeric	<code>ABS</code> , <code>ROUND</code> , <code>CEIL</code> , <code>FLOOR</code> , <code>RAND</code>	<code>ROUND(?rating)</code>
Date / time	<code>NOW</code> , <code>YEAR</code> , <code>MONTH</code> , <code>DAY</code> , <code>HOURS</code> , <code>MINUTES</code> , <code>SECONDS</code> , <code>TIMEZONE</code> , <code>TZ</code>	<code>YEAR(?date)</code>
Hash / ids	<code>MD5</code> , <code>SHA1</code> , <code>SHA256</code> , <code>SHA384</code> , <code>SHA512</code> , <code>UUID</code> , <code>STRUUID</code>	<code>IRI(CONCAT("urn:x:", STRUUID()))</code>
Conditional	<code>IF(cond, a, b)</code> , <code>COALESCE(a, b, ...)</code>	<code>COALESCE(?nick, ?name)</code>

 **TIP**
`COALESCE` returns the first expression that evaluates without error — the idiomatic way to provide defaults for OPTIONAL values.

String matching: REGEX and its cheaper cousins

Regular expressions follow XPath/XQuery syntax. Prefer simple string functions whenever they suffice.

```
SELECT ?title
WHERE {
  ?m a ex:Movie ; ex:title ?title .
  1 FILTER (REGEX(?title, "^the matrix", "i"))
}
```

```
# Equivalent, usually cheaper:
2 # FILTER (STRSTARTS(LCASE(?title), "the matrix"))
# Substring anywhere:
3 # FILTER (CONTAINS(?title, "Matrix"))
```

SPARQL

1 `REGEX(text, pattern, flags)`. `^` anchors the start; `"i"` = case-insensitive.

2 `STRSTARTS` / `CONTAINS` avoid compiling a regex and are easier to optimise.

3 `CONTAINS` is case-sensitive: `"matrix"` would match nothing here.



PRO MOVE

Any string `FILTER` over millions of literals is a full scan. For search workloads use a **full-text index** (Jena text, GraphDB connectors, QLever `ql:contains-word`) — see Part III.

RESULT

?title

The Matrix

The Matrix Reloaded

OPTIONAL: keep a row even when part of the pattern is missing

A left outer join. Variables inside *OPTIONAL* stay unbound when the optional part does not match.

```
SELECT ?title ?prequelTitle
WHERE {
  1 ?m a ex:Movie ; ex:title ?title .
  2 OPTIONAL {
    ?m ex:sequelOf ?pre .
    3 ?pre ex:title ?prequelTitle .
  }
}
ORDER BY ?title
```

SPARQL

RESULT

?title	?prequelTitle
Cloud Atlas	—
John Wick	—
The Matrix	—
The Matrix Reloaded	The Matrix

1 Mandatory part: every movie produces a row.

2 The **whole block** must match together, or nothing from it is added.

3 Only Reloaded has a prequel; the other rows keep **?prequelTitle unbound**.

INSIGHT

Unbound is **not** a value (not NULL, not empty string, not 0). Test it with `bound(?x)`; supply defaults with `COALESCE(?x, "n/a")`.

OPTIONAL pitfall: where you put the FILTER changes the answer

Inside OPTIONAL a FILTER restricts what is attached; outside it restricts which rows survive.

A · FILTER inside OPTIONAL

```
SELECT ?title ?rating WHERE {  
  ?m ex:title ?title .  
  OPTIONAL { ?m ex:rating ?rating .  
             FILTER (?rating > 8) }  
}
```

SPARQL

RESULT

?title	?rating
The Matrix	8.7
The Matrix Reloaded	—
John Wick	—
Cloud Atlas	—

INSIGHT

A: all 4 movies survive; the rating is attached only where it exceeds 8. **B:** the filter runs on the joined rows; 7.2 and 7.4 fail, so only 1 row remains.

B · FILTER after OPTIONAL

```
SELECT ?title ?rating WHERE {  
  ?m ex:title ?title .  
  OPTIONAL { ?m ex:rating ?rating . }  
  FILTER (?rating > 8)  
}
```

SPARQL

RESULT

?title	?rating
The Matrix	8.7

PITFALL

Several OPTIONALs are **left-associative**: $P \text{ OPT } A \text{ OPT } B = (P \text{ OPT } A) \text{ OPT } B$. Reordering them can change results when A and B share variables.

UNION: alternatives merged into one result

Each branch is evaluated independently; the solutions are concatenated (bag union).

```
SELECT ?name ?role
WHERE {
  {
    1 ?p ex:actedIn ex:cloudatlas .
    2 BIND ("actor" AS ?role)
  }
  UNION
  {
    3 ?p ex:directed ex:cloudatlas .
    BIND ("director" AS ?role)
  }
  4 ?p ex:name ?name .
}
ORDER BY ?role ?name
```

SPARQL

1 Branch 1: actors of Cloud Atlas.

2 BIND tags each row with the branch that produced it.

3 Branch 2: directors of Cloud Atlas.

4 Joined **after** the union — fetch names once instead of in each branch.



TIP

For alternatives on a single hop, the property path `ex:actedIn|ex:directed` (Part III) is shorter than a UNION.

RESULT

?name	?role
Hugo Weaving	actor
Lana Wachowski	director
Lilly Wachowski	director

Negation: FILTER NOT EXISTS vs. MINUS

Both answer "people who never directed a film" — but they are not the same operator.

FILTER NOT EXISTS (a test)

```
SELECT ?name WHERE {  
  ?p a ex:Person ; ex:name ?name .  
  FILTER NOT EXISTS { ?p ex:directed ?m }  
}
```

SPARQL

RESULT

?name
Keanu Reeves
Carrie-Anne Moss
Laurence Fishburne
Hugo Weaving

PITFALL

With **no shared variable** they diverge: `MINUS { ?x ?y ?z }` removes nothing, while `FILTER NOT EXISTS { ?x ?y ?z }` removes every row whenever the data is non-empty.

MINUS (set difference)

```
SELECT ?name WHERE {  
  ?p a ex:Person ; ex:name ?name .  
  MINUS { ?p ex:directed ?m }  
}
```

SPARQL

Same 4 rows here. `MINUS` removes left rows that are **compatible** with some right row *and share at least one variable* with it.

RULE OF THUMB

Default to `FILTER NOT EXISTS` — its meaning ("no match for this pattern given the current bindings") is what you almost always intend. `FILTER EXISTS` is the positive test.

Duplicates: DISTINCT and REDUCED

Joins multiply rows. Remove duplicates deliberately — and understand where they came from.

Without DISTINCT

```
SELECT ?name WHERE {  
  ?m ex:title ?t .  
  FILTER STRSTARTS(?t, "The Matrix")  
  ?p ex:actedIn ?m ; ex:name ?name .  
}
```

SPARQL

8 rows: 4 actors × 2 Matrix films. Each name appears once per film that matched.



INSIGHT
`DISTINCT` compares whole projected rows. `REDUCED` *permits* (but does not require) the engine to drop duplicates — a hint for cheap partial de-duplication.

With DISTINCT

```
SELECT DISTINCT ?name WHERE {  
  ?m ex:title ?t .  
  FILTER STRSTARTS(?t, "The Matrix")  
  ?p ex:actedIn ?m ; ex:name ?name .  
}
```

SPARQL

RESULT

?name

Keanu Reeves

Carrie-Anne Moss

Laurence Fishburne

Hugo Weaving



PITFALL
`SELECT DISTINCT` on millions of rows forces a sort or hash of the full result. If duplicates come from an unnecessary join, remove the join instead.

ORDER BY, LIMIT and OFFSET: sorting and paging

Evaluation order: WHERE → GROUP → HAVING → SELECT → ORDER BY → DISTINCT → OFFSET → LIMIT.

```
SELECT ?title ?rating
WHERE {
  ?m ex:title ?title ; ex:rating ?rating .
}
1 ORDER BY DESC(?rating) ?title
3 LIMIT 2
  OFFSET 1
```

SPARQL

1 `DESC()` / `ASC()` ; keys apply left to right. Full order: 8.7, 7.4 (Cloud Atlas), 7.4 (John Wick), 7.2.

2 The `?title` tie-breaker makes paging **deterministic** — essential when ratings tie.

3 `OFFSET 1` skips The Matrix; `LIMIT 2` keeps the next two. `LIMIT/OFFSET` may be written in either order.

RESULT

?title	?rating
Cloud Atlas	7.4
John Wick	7.4



Deep `OFFSET` paging is $O(\text{offset})$. For large exports use **keyset paging**: remember the last `(rating, title)` and add `FILTER(?rating < 7.4 || (?rating = 7.4 && ?title > "John Wick"))`.

BIND: computing new variables

BIND assigns an expression to a fresh variable, which later patterns and filters can use.

```
SELECT ?name ?ageAtMatrix
WHERE {
  ?p ex:actedIn ex:matrix ;
      ex:name ?name ;
      ex:born ?born .
  1 ex:matrix ex:released ?year .
  2 BIND (?year - ?born AS ?ageAtMatrix)
  3 FILTER (?ageAtMatrix < 36)
}
ORDER BY ?ageAtMatrix
```

SPARQL

- 1 A pattern sharing no variable with the rest simply adds `?year = 1999` to every row.
- 2 `BIND (expr AS ?var)` — the variable must **not** already be bound earlier in the group.
- 3 Ages in 1999: Carrie-Anne 32, Keanu 35, Laurence 38, Hugo 39.

RESULT

?name	?ageAtMatrix
Carrie-Anne Moss	32
Keanu Reeves	35



TIP

Alternative: compute in the projection — `SELECT ?name ((?year - ?born) AS ?age)`. `BIND` is needed when later patterns, FILTERs or joins must use the value.

VALUES: inline data tables

Inject a fixed set of bindings — ideal for parameterised queries, lookups and batch processing.

```
SELECT ?name ?title
WHERE {
  1 VALUES ?name { "Keanu Reeves" "Hugo Weaving" }
  2 ?p ex:name ?name ;
    ex:actedIn ?m .
    ?m ex:title ?title .
}
ORDER BY ?name ?title

# Multi-column form, with a missing value:
# VALUES (?name ?minRating) {
  3 # ("Keanu Reeves" 7.3) ("Hugo Weaving" UNDEF)
  # }
```

SPARQL

RESULT

?name	?title
Hugo Weaving	Cloud Atlas
Hugo Weaving	The Matrix
Hugo Weaving	The Matrix Reloaded
Keanu Reeves	John Wick
Keanu Reeves	The Matrix
Keanu Reeves	The Matrix Reloaded

- 1 One-column form: `?name` takes each listed value in turn, joined with the rest of the pattern.
- 2 The engine can start from these 2 known names — a highly selective anchor.
- 3 `UNDEF` leaves a variable unbound in that row, so it joins with anything.

Aggregate functions

Aggregates collapse groups of solutions into one row. Without GROUP BY, the whole result is one group.

Aggregate	Returns	Notes
COUNT (*)	Number of rows	Counts solutions
COUNT (?x)	Rows where ?x is bound	Skips unbound
COUNT (DISTINCT ?x)	Distinct values	The usual fix for join duplication
SUM (?x) / AVG (?x)	Numeric	A type error makes the aggregate an error
MIN (?x) / MAX (?x)	Smallest / largest	Uses SPARQL ordering across types
SAMPLE (?x)	Any one value	Non-deterministic "pick one"
GROUP_CONCAT (?x; SEPARATOR=" , ")	One string	Order of items is unspecified

INSIGHT

Every aggregate accepts `DISTINCT`:
`SUM (DISTINCT ?x)`,
`GROUP_CONCAT (DISTINCT ?n)`. On an empty group, `COUNT` and `SUM` give 0 while `AVG`, `MIN` and `MAX` are unbound.

TIP

Aggregates must be aliased: `(COUNT (?m) AS ?films)`. The same expression can be reused in `HAVING` and `ORDER BY`.

GROUP BY and HAVING: films per actor

GROUP BY partitions solutions; HAVING filters groups (a FILTER that runs after aggregation).

```
1 SELECT ?name (COUNT(?m) AS ?films)
   WHERE {
     ?p ex:actedIn ?m ;
        ex:name ?name .
   }
2 GROUP BY ?name
3 HAVING (COUNT(?m) >= 3)
4 ORDER BY DESC(?films) ?name
```

SPARQL

RESULT

?name	?films
Hugo Weaving	3
Keanu Reeves	3

 PRO MOVE

Group by the IRI (?p) rather than the name when names might collide, and fetch the label with (SAMPLE(?name) AS ?label).

- 1 The projection may contain only group keys and aggregates.
- 2 One group per distinct ?name: Keanu 3, Hugo 3, Carrie-Anne 2, Laurence 2.
- 3 HAVING sees aggregates — a FILTER could not reference COUNT.
- 4 Sort by the alias, then by name for a stable order.

Several aggregates at once — and the classic error

Cast size, cast list and average birth year for every film in a single pass.

```
1 SELECT ?title
2   (COUNT(DISTINCT ?actor) AS ?cast)
3   (GROUP_CONCAT(DISTINCT ?actorName; SEPARATOR=", ") AS ?names)
4   (AVG(?born) AS ?avgBorn)
5 WHERE {
6   ?m ex:title ?title .
7   ?actor ex:actedIn ?m ;
8         ex:name ?actorName ;
9         ex:born ?born .
10 }
11 GROUP BY ?title
12 ORDER BY DESC(?cast) ?title
```

SPARQL

RESULT

?title	?cast	?names	?avgBorn
The Matrix	4	Keanu Reeves, Carrie-Anne Moss, Laurence Fishburne, Hugo Weaving	1963
The Matrix Reloaded	4	(the same four)	1963
Cloud Atlas	1	Hugo Weaving	1960
John Wick	1	Keanu Reeves	1964

- 1 **DISTINCT** inside the aggregate protects against duplicated rows.
- 2 **GROUP_CONCAT** builds a list string; the order of names is **not** guaranteed.
- 3 Matrix cast: $(1964 + 1967 + 1961 + 1960) / 4 = 1963$.

PITFALL

Projecting **?m** here is an **error**: it is neither grouped nor aggregated (some engines silently return an arbitrary value). Group by it, or wrap it: `(SAMPLE(?m) AS ?anyMovie)`.

Subqueries: the best-rated film of each director

An inner *SELECT* computes an aggregate; the outer query joins back to recover the row that produced it.

```
SELECT ?director ?title ?rating
WHERE {
  {
    1 SELECT ?d (MAX(?r) AS ?rating)
    2 WHERE { ?d ex:directed ?m . ?m ex:rating ?r . }
    GROUP BY ?d
  }
  3 ?d ex:directed ?m ;
    ex:name ?director .
    ?m ex:rating ?rating ;
    ex:title ?title .
}
ORDER BY ?director
```

1 Inner query: one row per director with their maximum rating.

2 This `?m` is **private** to the subquery — only projected variables (`?d`, `?rating`) are visible outside.

3 The outer `?m` is a new variable: find the film whose rating equals the maximum ("argmax").

INSIGHT

Subqueries are (logically) evaluated first and joined like any other pattern. Ties at the maximum return **several** rows per director — decide whether that is what you want.

RESULT

?director	?title	?rating
Chad Stahelski	John Wick	7.4
Lana Wachowski	The Matrix	8.7
Lilly Wachowski	The Matrix	8.7

Multilingual data: language tags in queries

Uses the extended data from slide 15: "The Matrix"@en and "Matrix"@de.

```
1 SELECT ?t (LANG(?t) AS ?lang)
   WHERE {
2     FILTER (LANGMATCHES (LANG(?t), "en"))
3     # These do NOT match a tagged literal:
4     #   ?m ex:title "The Matrix" .
5     # These do:
6     #   ?m ex:title "The Matrix"@en .
7     #   FILTER (STR(?t) = "The Matrix")
```

SPARQL

1 `LANG()` returns the tag ("en") or "" for untagged literals.

2 `LANGMATCHES` understands ranges: "en" also matches en-GB and en-US; "*" matches any tag.

3 A tagged and an untagged literal are **different terms** — the #1 cause of "my query returns nothing".

4 `STR()` strips the tag and datatype, leaving the lexical form.

RESULT

?t	?lang
"The Matrix"@en	en



PRO MOVE

Label fallback: `OPTIONAL { ?x rdfs:label ?en FILTER(LANG(?en)="en") } OPTIONAL { ?x rdfs:label ?any } then BIND(COALESCE(?en, ?any) AS ?label).`

Datatypes and casting

Inspect types with `DATATYPE()`; convert with `XSD` constructor functions; build typed literals with `STRDT()`.

```
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
```

```
SELECT ?title ?rating
```

```
1 (DATATYPE(?rating) AS ?dt)
```

```
2 (xsd:integer(ROUND(?rating)) AS ?rounded)
```

```
3 (STR(?rating) AS ?asString)
```

```
WHERE { ?m ex:title ?title ; ex:rating ?rating . }
```

```
ORDER BY DESC(?rating) ?title
```

SPARQL

1 Returns the datatype **IRI** — handy for data-quality checks.

2 Constructors: `xsd:integer()`, `xsd:decimal()`, `xsd:double()`, `xsd:dateTime()`, `xsd:boolean()`, `xsd:string()`.

3 `STR()` yields a plain string — e.g. for `CONCAT`.

RESULT

?title	?rating	?dt	?rounded	?asString
The Matrix	8.7	xsd:decimal	9	"8.7"
Cloud Atlas	7.4	xsd:decimal	7	"7.4"
John Wick	7.4	xsd:decimal	7	"7.4"
The Matrix Reloaded	7.2	xsd:decimal	7	"7.2"



PITFALL

A failed cast such as `xsd:integer("abc")` is an **error**: in a projection the variable stays unbound; in a `FILTER` the row disappears.

Dates and times

Typed dates compare chronologically and expose their components through accessor functions.

```
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

1 SELECT ?title ?date (YEAR(?date) AS ?y) (MONTH(?date) AS ?mo)
WHERE {
  ?m ex:title ?title ;
2   ex:releaseDate ?date .
3   FILTER (?date >= "1999-01-01"^^xsd:date
    && ?date < "2000-01-01"^^xsd:date)
}
```

RESULT

?title	?date	?y	?mo
The Matrix	1999-03-31	1999	3

 **NOTE**

SPARQL 1.1 formally defines the accessors on `xsd:dateTime`; most engines also accept `xsd:date`.
`NOW()` returns the query time and is constant within one query.

- 1 YEAR, MONTH, DAY, HOURS, MINUTES, SECONDS, TIMEZONE, TZ extract parts.
- 2 Needs the `ex:releaseDate` triple from slide 15.
- 3 Compare typed literal with typed literal. A half-open range `[start, end)` avoids boundary bugs.

 **PITFALL**

Date and duration arithmetic (`?d2 - ?d1`) is **not** in SPARQL 1.1 core. Many stores offer it as an extension — check your engine.

Checkpoint II — SPARQL fundamentals

Q1

Titles of films directed by **Lana Wachowski** released after 2000.

```
SELECT ?t WHERE {  
  ?d ex:name "Lana Wachowski" ;  
    ex:directed ?m .  
  ?m ex:title ?t ; ex:released ?y .  
  FILTER (?y > 2000) }  
# → Reloaded, Cloud Atlas
```

Q2

Everyone who has **no outgoing** `ex:knows` edge.

```
SELECT ?name WHERE {  
  ?p a ex:Person ; ex:name ?name .  
  FILTER NOT EXISTS { ?p ex:knows ?x }  
}  
# → Hugo, Lana, Lilly, Chad
```

Q3

The **average rating** of all films.

```
SELECT (AVG(?r) AS ?avg)  
WHERE { ?m ex:rating ?r }  
# → 7.675
```

Q4

Why can `COUNT(?m)` exceed `COUNT(DISTINCT ?m)` in a grouped query?



ANSWER

Joins **multiply rows**: if a film matches through several paths (e.g. several actors), it appears once per path. `COUNT` counts rows; `COUNT(DISTINCT)` counts values.

03

PART III

Advanced SPARQL

Paths of arbitrary length, graph construction, federation across endpoints, updates, statements about statements, validation and performance.

Property paths

CONSTRUCT

ASK / DESCRIBE

Named graphs

SERVICE

Wikidata

SPARQL Update

Reasoning

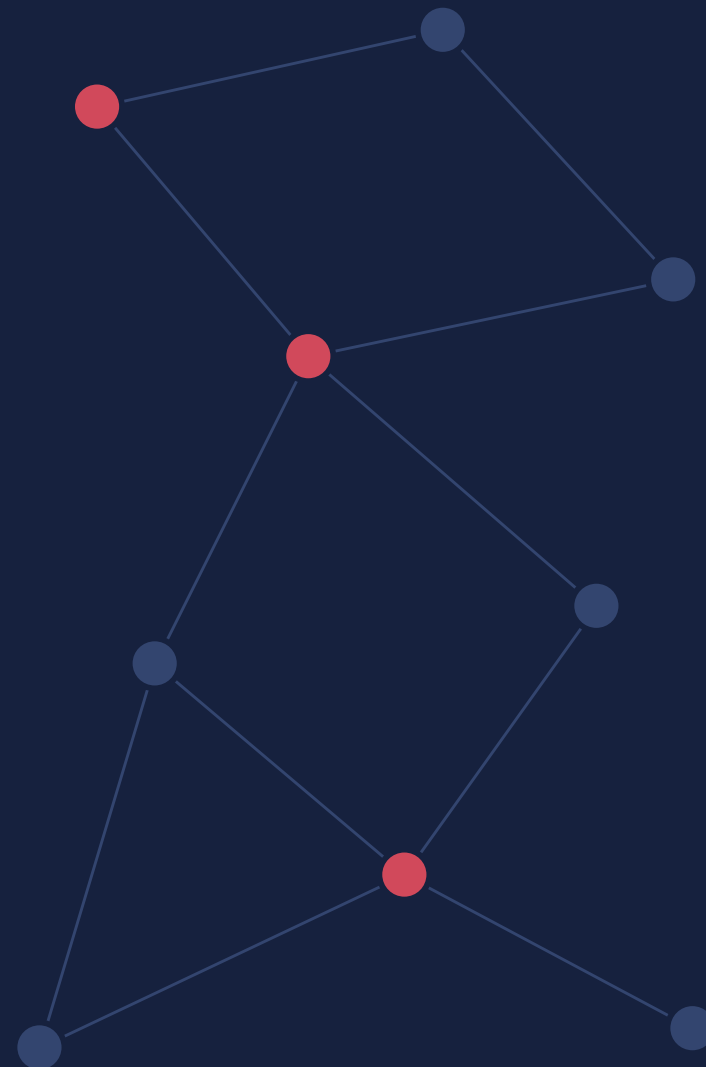
RDF-star

Protocol

Python

Algebra

SHACL



Property paths: regular expressions over predicates

Paths let one triple pattern traverse many edges. They are written in the predicate position.

Syntax	Meaning	Example	Reads as
<code>iri</code>	One edge	<code>?p ex:knows ?f</code>	p knows f
<code>^elt</code>	Inverse direction	<code>?m ^ex:actedIn ?p</code>	p actedIn m
<code>e1 / e2</code>	Sequence	<code>?p ex:actedIn/ex:title ?t</code>	title of a film p acted in
<code>e1 e2</code>	Alternative	<code>?p ex:actedIn ex:directed ?m</code>	acted in or directed
<code>elt*</code>	Zero or more	<code>?c rdfs:subClassOf* ?super</code>	c itself or any ancestor
<code>elt+</code>	One or more	<code>ex:keanu ex:knows+ ?x</code>	transitively reachable
<code>elt?</code>	Zero or one	<code>?p ex:knows? ?x</code>	p itself or a direct contact
<code>!(p1 p2)</code>	Negated property set	<code>?s !(rdf:type ex:name) ?o</code>	any edge except these
<code>(...)</code>	Grouping	<code>?p (ex:knows/ex:knows)+ ?x</code>	even number of hops

NOTE

Bounded repetition such as `elt{2,4}` appeared in drafts but is **not** in SPARQL 1.1. Some engines support it as an extension; portable queries spell hops out with `/` or `?`.

Property paths in action

Transitive reachability and the most useful idiom in RDF querying: instances of a class and its subclasses.

```
# Everyone reachable from Keanu through ex:knows (1+ hops)
```

```
SELECT ?name WHERE {
```

```
1  ex:keanu ex:knows+ ?p .
```

```
2  ?p ex:name ?name .
```

```
}
```

```
# Instances of ex:Person, including subclass instances
```

```
SELECT ?x WHERE {
```

```
3  ?x a/rdfs:subClassOf* ex:Person .
```

```
}
```

SPARQL

1 **+** = transitive closure. Carrie-Anne and Chad (1 hop), Laurence (2), Hugo (3).

2 An ordinary pattern continues from each reached node.

3 **a/rdfs:subClassOf***: take the direct type, then walk up the hierarchy 0+ steps — "poor man's RDFS reasoning".

RESULT

?name (first query)

Carrie-Anne Moss

Chad Stahelski

Laurence Fishburne

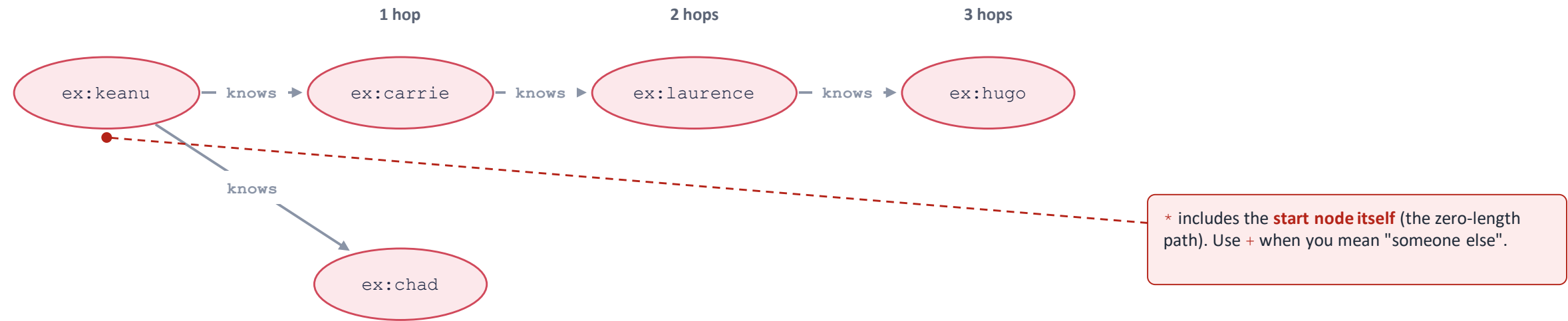
Hugo Weaving

INSIGHT

Arbitrary-length paths (*****, **+**) have **set semantics**: each reachable node is returned once, even if there are many routes. Cycles are handled safely.

What each path expression reaches from ex:keanu

The ex:knows sub-graph of our dataset, and the answer sets of five path expressions.



ANSWERS FOR ?x IN ex:keanu <path> ?x

Path	Answer set for ?x
ex:knows	carrie, chad
ex:knows/ex:knows	laurence
ex:knows+	carrie, chad, laurence, hugo
ex:knows*	keanu (zero hops!), carrie, chad, laurence, hugo
^ex:knows	(empty — nobody knows Keanu)

Direction matters: `ex:knows` is directed. For "connected either way" use `(ex:knows | ^ex:knows) +`.

Property paths: semantics you must know



No path objects

A path pattern returns only its **endpoints**. You cannot get the intermediate nodes or the route itself — model explicitly or use an engine extension.



No length

There is no built-in hop count. A known trick counts intermediate nodes, but it is only correct when paths are **unique** (trees).



Cycles are safe

***** and **+** terminate on cyclic data because they compute a set of reachable nodes, not a list of walks.



Bag vs. set

Fixed-length paths (`a/b`, `a|b`) are rewritten into joins/unions and **keep duplicates**; arbitrary-length ones (*****, **+**) return distinct nodes.



Cost explodes on dense predicates

`?x ex:knows* ?y` with **both ends unbound** computes the full transitive closure. Always anchor at least one end with a constant or a selective pattern.



Lists

Walk an RDF collection with `ex:matrix`
`ex:genres/rdf:rest*/rdf:first ?g` —
returns every list member (order not guaranteed).

Inverse paths and negated property sets

Navigate edges backwards without rewriting the pattern; match "any edge except".

```
# Who directed The Matrix? (walk directed backwards, then name)
SELECT ?name WHERE {
1  ex:matrix ^ex:directed/ex:name ?name .
}

# Everything Keanu links to, except via these predicates
SELECT ?o WHERE {
2  ex:keanu !(ex:actedIn|ex:name|ex:born|rdf:type) ?o .
}
```

SPARQL

RESULT

?name (first query)

Lana Wachowski

Lilly Wachowski

INSIGHT

A negated property set matches exactly **one** edge and does not bind the predicate. To see which predicate matched, use a variable: `ex:keanu ?pred ?o FILTER(?pred NOT IN (...))`.

1 `^ex:directed` moves from film to director; `/ex:name` continues to the label. Answer: Lana, Lilly.

2 Negated property set: any single predicate **not** in the list. Answer: `ex:carrie`, `ex:chad` (via `ex:knows`).

TIP

`! (^ex:knows)` matches inverse edges other than `knows` — inverse elements can appear inside a negated set too.

CONSTRUCT: building a new graph (vocabulary mapping)

Translate our ex: vocabulary into schema.org — the bread and butter of RDF data integration.

```
PREFIX ex:      <http://example.org/>
PREFIX schema: <https://schema.org/>

1 CONSTRUCT {
    ?m a schema:Movie ;
      schema:name ?title ;
2    schema:actor ?p .
    ?p a schema:Person ;
      schema:name ?name .
}
3 WHERE {
    ?m a ex:Movie ; ex:title ?title .
    OPTIONAL { ?p ex:actedIn ?m ; ex:name ?name . }
}
```

SPARQL

OUTPUT GRAPH (EXCERPT)

subject	predicate	object
ex:matrix	rdf:type	schema:Movie
ex:matrix	schema:name	"The Matrix"
ex:matrix	schema:actor	ex:keanu
ex:keanu	schema:name	"Keanu Reeves"

1 The **template**: a Turtle-like graph with variables, instantiated once per solution.

2 For films with no actor, **?p** is unbound and every triple mentioning it is **skipped**, not an error.

3 The WHERE clause is an ordinary pattern — everything from Part II applies.

 INSIGHT

The result is an RDF **graph**, so it is a set: identical triples produced by several solutions collapse into one.

CONSTRUCT for derived edges: a co-star network

Materialise relationships that are only implicit in the data — perfect input for graph algorithms and ML.

```
1 CONSTRUCT { ?a ex:coStarredWith ?b . }
  WHERE {
    ?a ex:actedIn ?m .
    ?b ex:actedIn ?m .
2 FILTER (?a != ?b)
  }

# Short form: template = pattern (plain BGP only)
3 CONSTRUCT WHERE { ?s ex:knows ?o }
```

SPARQL

1 One derived triple per pair of actors sharing a film.

2 Excludes self-pairs. Both directions ($a \rightarrow b$ and $b \rightarrow a$) are produced — the relation is symmetric.

3 **CONSTRUCT WHERE** copies matching triples verbatim; no FILTERs or OPTIONALs allowed.



PRO MOVE

Load the constructed graph into a named graph (**INSERT ... WHERE** with the same pattern) and version it — a reproducible **feature table** for downstream graph ML.

SIZE OF THE RESULT

Source of pairs	Ordered pairs
Matrix cast (4 actors)	$4 \times 3 = 12$
Reloaded cast (same 4)	same 12 pairs — merged
John Wick / Cloud Atlas (1 actor each)	0
Total triples	12

ASK and DESCRIBE

ASK answers yes/no questions cheaply; DESCRIBE asks the store to summarise a resource.

1 **ASK** { `ex:keanu ex:actedIn ex:cloudatlas` }

2 **ASK** {
 `?p ex:directed ?m ;`
 `ex:actedIn ?m .`
}

3 **DESCRIBE** `ex:keanu`

4 **DESCRIBE** `?m WHERE` {
 `?m ex:rating ?r .`
 FILTER (`?r > 8`)
}

SPARQL



PITFALL

DESCRIBE output is **implementation-defined**. Some stores include incoming triples, some do not. Use CONSTRUCT when you need a precise, portable shape.

1 → **false**. Keanu is not in Cloud Atlas.

2 → **false**: nobody both directed and acted in the same film in our data. ASK may stop at the first match.

3 Returns a graph about `ex:keanu`. Many stores return its **Concise Bounded Description** (outgoing triples, recursing into blank nodes).

4 DESCRIBE can take a WHERE clause: here, describe The Matrix.



TIP

ASK is ideal for data-quality tests in CI: "ASK { ?m a ex:Movie FILTER NOT EXISTS { ?m ex:title ?t } }" should be false.

Querying named graphs: FROM, FROM NAMED and GRAPH

Uses the TriG data from slide 20: the same rating claim stored in two source graphs.

```
SELECT ?g ?rating ?when
1 FROM NAMED ex:sourceA
  FROM NAMED ex:sourceB
  WHERE {
2   GRAPH ?g {
     ex:matrix ex:rating ?rating .
   }
3   OPTIONAL { ?g ex:retrieved ?when }
  }
ORDER BY ?g
```

RESULT

?g	?rating	?when
ex:sourceA	8.7	2026-01-15
ex:sourceB	8.1	—

1 FROM NAMED makes these graphs available to GRAPH blocks.

2 GRAPH ?g { ... } matches inside each named graph and binds ?g to its name.

3 Outside GRAPH, patterns hit the default graph — where the provenance triple lives in the file, if the store includes it.

 PITFALL

When a query has FROM/FROM NAMED, the default graph is **only** what FROM lists — here it is empty, so ?when is unbound unless you add FROM <default-graph>. Stores differ; many offer a "union default graph" setting.

The RDF dataset seen by a query

A query runs against exactly one default graph plus a set of named graphs — assembled by the dataset clause or the store.

DEFAULT GRAPH

```
ex:keanu ex:name "Keanu Reeves"  
ex:sourceA ex:retrieved 2026-01-15
```

Patterns written *outside* GRAPH see only this graph.

NAMED GRAPH ex:sourceA

```
ex:matrix ex:rating 8.7
```

NAMED GRAPH ex:sourceB

```
ex:matrix ex:rating 8.1
```

HOW EACH CLAUSE SHAPES THE DATASET

Clause	Effect
FROM <g>	Merges g into the default graph
FROM NAMED <g>	Adds g as a named graph
GRAPH <g> { }	Matches inside graph g only
GRAPH ?g { }	Iterates over all named graphs
(none)	Store-defined dataset

INSIGHT

Graph names are IRIs, so a graph can itself be the subject of triples (source, licence, timestamp, trust score). This is RDF's primary provenance mechanism.

PRO MOVE

Pattern for versioned ingestion: one named graph per source *per load*, plus a catalogue graph describing them. Rollback = `DROP GRAPH`.

Federated queries with SERVICE

Join local data with a remote SPARQL endpoint inside one query.

```
PREFIX wdt: <http://www.wikidata.org/prop/direct/>
PREFIX owl: <http://www.w3.org/2002/07/owl#>

SELECT ?name ?birthPlaceLabel
WHERE {
  ?p ex:name ?name ;
  1 owl:sameAs ?wd .
  2 SERVICE <https://query.wikidata.org/sparql> {
  3 ?wd wdt:P19 ?birthPlace .
    ?birthPlace rdfs:label ?birthPlaceLabel .
    FILTER (LANG(?birthPlaceLabel) = "en")
  }
}
```

SPARQL

1 Assumes we linked our people to Wikidata items with `owl:sameAs` (entity linking).

2 `SERVICE <endpoint> { ... }` evaluates this block remotely. `SERVICE SILENT` ignores failures.

3 `wdt:P19` = place of birth (e.g. Keanu Reeves → Beirut).

NOTE

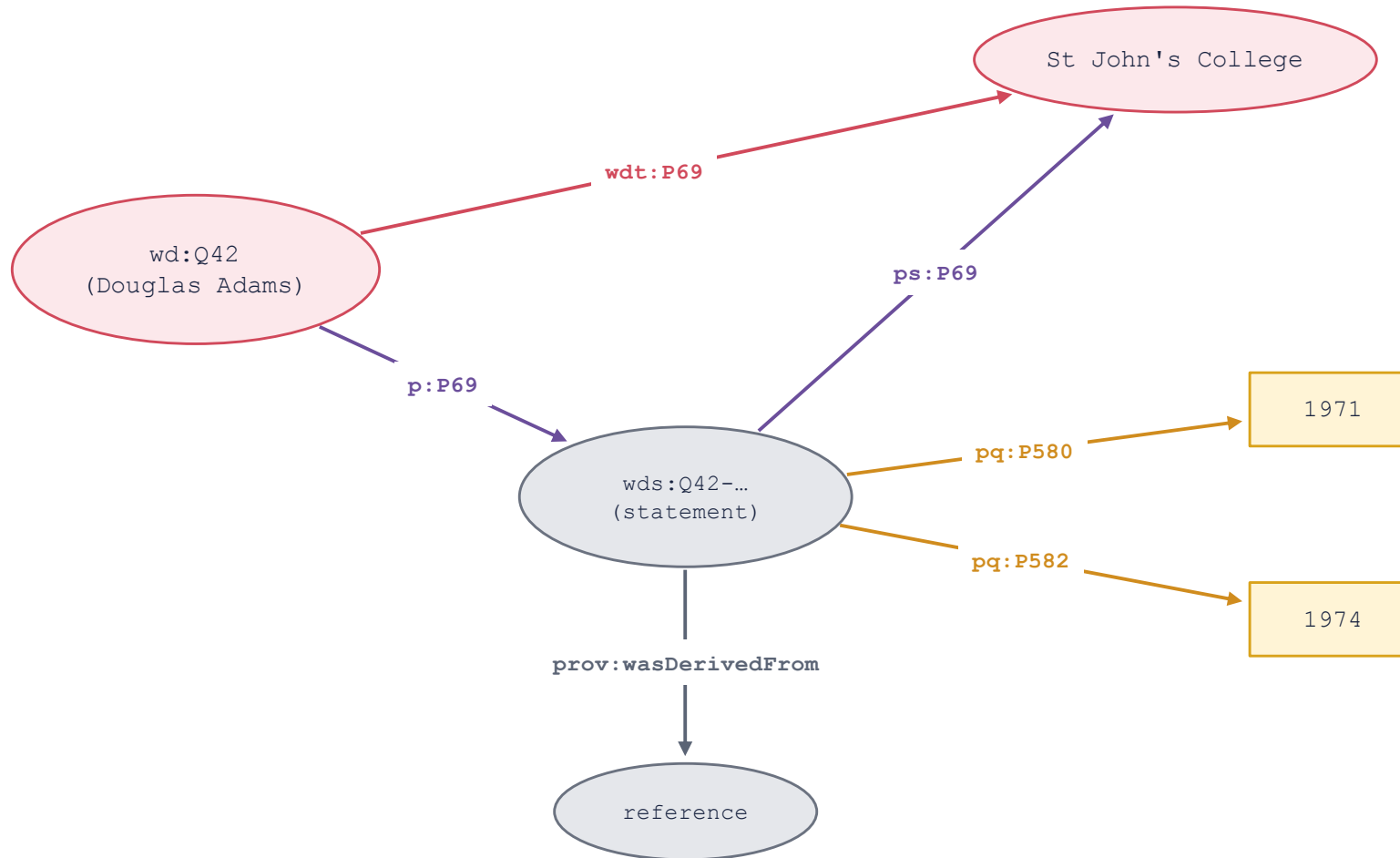
Public endpoints enforce timeouts, rate limits and user-agent policies; some disable federation entirely. For production joins, cache the remote data locally.

PITFALL

Engines differ in whether they send your local bindings to the remote side. A naïve engine may ask Wikidata for **every** birthplace on Earth. Keep the remote block selective, or push constants in with `VALUES`.

The Wikidata statement model: truthy triples vs. full statements

Wikidata stores each claim twice: a simple "truthy" edge, and a statement node carrying qualifiers and references.



wdt: "truthy" direct edge — best-rank value only, no qualifiers. Fast and simple.

p: → **ps:** reaches the same value via a **statement node**, which can carry **pq:** qualifiers (start/end time, role...) and references.

P69 = educated at · P580 = start time · P582 = end time.

Use **wdt:** for quick lookups; use **p:** / **ps:** / **pq:** when time, rank or provenance matters.

A real Wikidata query

Run it at query.wikidata.org — the common prefixes are predefined there.

```
SELECT ?film ?filmLabel ?date WHERE {  
1  ?film wdt:P31 wd:Q11424 ;  
2      wdt:P57 ?director ;  
3      wdt:P577 ?date .  
4  ?director rdfs:label "Lana Wachowski"@en .  
5  SERVICE wikibase:label {  
    bd:serviceParam wikibase:language "en" .  
  }  
}  
ORDER BY DESC (?date)  
LIMIT 20
```

SPARQL

PITFALL

Multiple publication dates produce multiple rows per film. Aggregate with `(MIN(?date) AS ?firstRelease)` and `GROUP BY ?film ?filmLabel`.

TIP

Wikidata's query UI can render results as a table, map, timeline, graph or chart via `#defaultView: comments`.

1 P31 instance of → Q11424 film.

2 P57 director.

3 P577 publication date (a film may have several, one per country).

4 Matching on a label is convenient but slow; in production use the item id (QID).

5 The **label service** fills `?xLabel` for every projected `?x` automatically.

SPARQL 1.1 Update: INSERT DATA and DELETE DATA

Ground triples in, ground triples out. The simplest and fastest update operations.

```
PREFIX ex: <http://example.org/>
```

1

```
INSERT DATA {  
  ex:johnwick2 a ex:Movie ;  
  ex:title "John Wick: Chapter 2" ;  
  ex:released 2017 ;  
  ex:sequelOf ex:johnwick .  
  ex:keanu ex:actedIn ex:johnwick2 .  
  ex:laurence ex:actedIn ex:johnwick2 .  
}
```

2

```
;
```

```
DELETE DATA {
```

3

```
GRAPH ex:sourceB { ex:matrix ex:rating 8.1 . }  
}
```

SPARQL

1

No variables allowed — only concrete triples (blank nodes create fresh nodes).

2

;
separates several operations in one request. Stores usually apply the request atomically.

3

Wrap in **GRAPH** <g> to target a named graph. Deleting a triple that is absent is a no-op.



INSIGHT

Because a graph is a set, inserting an existing triple changes nothing. Updates are therefore **idempotent** — safe to retry.



NOTE

Later examples do **not** assume this update was applied — the canonical dataset remains the 59 triples of slide 17.

Pattern-based updates: DELETE / INSERT ... WHERE

The *WHERE* clause finds bindings once; then all deletions, then all insertions, are applied.

```
# 1. Rename a predicate across the whole graph
1 DELETE { ?s ex:released ?y }
  INSERT { ?s ex:releaseYear ?y }
  WHERE { ?s ex:released ?y } ;

# 2. Repair ratings stored as strings
DELETE { ?m ex:rating ?old }
INSERT { ?m ex:rating ?new }
WHERE {
2   ?m ex:rating ?old .
  FILTER (DATATYPE(?old) = xsd:string)
3   BIND (xsd:decimal(?old) AS ?new)
} ;

# 3. Wipe everything about a film in one graph
4 WITH ex:sourceA
5 DELETE WHERE { ex:matrix ?p ?o }
```

SPARQL

- 1 Templates may use any variable bound by *WHERE*.
- 2 Only the broken rows are touched; valid decimals are left alone.
- 3 *BIND* computes the replacement value inside *WHERE*.
- 4 *WITH* <g> sets the graph for the whole operation.
- 5 *DELETE WHERE* shorthand: the pattern is also the delete template.

PITFALL

Rename queries touch every matching triple in one transaction. On large stores, batch by subject ranges or use the bulk tools, and always test with a *SELECT* first.

Graph management operations

Whole-graph operations from SPARQL 1.1 Update. Add *SILENT* to suppress errors (e.g. graph does not exist).

Operation	Effect	Example
LOAD	Fetch an RDF document and insert its triples	LOAD <https://example.org/movies.ttl> INTO GRAPH ex:movies
CLEAR	Remove all triples, keep the (empty) graph	CLEAR GRAPH ex:sourceB · CLEAR DEFAULT · CLEAR ALL
CREATE	Create an empty named graph (stores that track empty graphs)	CREATE GRAPH ex:staging
DROP	Remove the graph entirely	DROP SILENT GRAPH ex:staging
COPY	Overwrite target with source (source kept)	COPY ex:staging TO ex:movies
MOVE	Overwrite target with source, then drop source	MOVE ex:staging TO ex:movies
ADD	Append source triples into target	ADD ex:sourceA TO DEFAULT

 PRO MOVE

Blue/green loading: **LOAD** into `ex:staging`, validate with SHACL or ASK tests, then **MOVE** `ex:staging TO ex:live` — an atomic swap in most stores.

 NOTE

The **Graph Store HTTP Protocol** offers the same operations RESTfully: GET/PUT/POST/DELETE on `.../data?graph=<iri>`.

Entailment regimes and reasoning

The same query can return more answers when the store reasons over RDFS/OWL. Know which mode you are in.



What entailment adds

With RDFS, `?x a ex:Person` also returns every `ex:Actor` (subclassOf) and everyone with `ex:actedIn`(domain) — even if no such triple was asserted.



Materialisation

Forward chaining: inferred triples are computed at load time and stored. Fast queries, more storage, updates must retract inferences (GraphDB, RDFox).



Query rewriting

Backward chaining: the query is expanded at run time (e.g. into UNIONs or paths). No extra storage, slower queries (Stardog, OWL 2 QL systems).



Rule profiles

Most stores support RDFS and **OWL 2 RL** rule sets; some add custom rules (SWRL, SHACL rules, Datalog in RDFox).



No reasoner? Use paths

`?x a/rdfs:subClassOf* ex:Person` recovers subclass reasoning in plain SPARQL, independent of store configuration.



Surprises

Counts change, `owl:sameAs` merges nodes, and a single wrong domain axiom can type half the graph. Always test queries with reasoning on **and** off.

Statements about statements: RDF-star / RDF 1.2

Edge properties for RDF. Standardised in RDF 1.2 / SPARQL 1.2 and already supported by many stores as "RDF-star".

```
# Turtle 1.2: assert the triple AND annotate it
1 ex:keanu ex:actedIn ex:matrix { | ex:role "Neo" | } .

# SPARQL: match annotations on a triple
SELECT ?name ?role WHERE {
2   << ?p ex:actedIn ex:matrix >> ex:role ?role .
   ?p ex:name ?name .
}

# Classic alternative that works everywhere: an n-ary node
3 #   ex:perfl a ex:Performance ; ex:actor ex:keanu ;
#   ex:film ex:matrix ; ex:role "Neo" .
```

SPARQL

RESULT

?name	?role
Keanu Reeves	Neo

1 `{ | ... | }` annotation syntax: the base triple is asserted, and a reifier node carries `ex:role`.

2 `<< s p o >>` in a query matches the reifier of that triple, so it can be the subject of further patterns.

3 The **n-ary relation** pattern turns the edge into a node — portable, but queries need one extra hop.

NOTE

Syntax details changed between the early RDF-star community report and RDF 1.2 (triple terms `<< ( ... ) >>` vs. reifiers). Check which version your store implements before relying on exact semantics.

SPARQL result formats: what your code receives

application/sparql-results+json is the default choice for applications.

```
{
  1 "head": { "vars": [ "title", "year" ] },
    "results": {
      "bindings": [
        2 {
          "title": { "type": "literal", "value": "John Wick" },
          "year": { "type": "literal", "value": "2014",
        3                  "datatype": "http://www.w3.org/2001/XMLSchema#integer" }
        }
      ]
    }
}
```

JSON

1 `head.vars` lists the projected variables in order.

2 Each value is a typed RDF term: `"uri"`, `"literal"` or `"bnode"` (plus `xml:lang` for tagged strings).

3 Numbers arrive as **strings** plus a datatype — convert explicitly in your client.

INSIGHT

An unbound variable is simply **absent** from that binding object — check with `"x" in row`, not `row["x"] is None`.

NOTE

Other formats: XML (`...results+xml`), CSV (lossy: loses types), TSV (keeps RDF term syntax), and for CONSTRUCT any RDF syntax.

The SPARQL protocol over HTTP

Every SPARQL store speaks the same HTTP protocol, so `curl` is a universal client. (Fuseki endpoint paths shown.)

```
# GET with a URL-encoded query
1 curl -G 'http://localhost:3030/movies/sparql' \
  --data-urlencode 'query=SELECT * WHERE { ?s ?p ?o } LIMIT 5' \
2  -H 'Accept: application/sparql-results+json'

# POST a query file directly
3 curl -X POST 'http://localhost:3030/movies/sparql' \
  -H 'Content-Type: application/sparql-query' \
  --data-binary @top-rated.rq

# POST an update (separate endpoint)
4 curl -X POST 'http://localhost:3030/movies/update' \
  -H 'Content-Type: application/sparql-update' \
  --data-binary 'INSERT DATA { <urn:a> <urn:b> <urn:c> }'
```

SHELL

- 1 GET works for short read queries and is cacheable by HTTP proxies.
- 2 The `Accept` header picks the result format (content negotiation).
- 3 POST with `application/sparql-query` avoids URL length limits.
- 4 Updates always use POST, usually on a separately secured endpoint.



PRO MOVE

Expose only the query endpoint publicly, behind a timeout and result limit. Keep `/update` on an internal network with authentication.

SPARQL from Python: rdflib and SPARQLWrapper

rdflib runs SPARQL over in-memory graphs; SPARQLWrapper talks to any remote endpoint.

```
from rdflib import Graph, Literal
from SPARQLWrapper import SPARQLWrapper, JSON

1 g = Graph()
  g.parse("movies.ttl", format="turtle")

q = """
PREFIX ex: <http://example.org/>
SELECT ?title ?rating WHERE {
  ?m ex:title ?title ; ex:rating ?rating .
  FILTER (?rating > ?min)
} ORDER BY DESC(?rating) ?title
"""
2
3 for row in g.query(q, initBindings={"min": Literal(7.3)}):
  print(row.title, row.rating)

4 sw = SPARQLWrapper("https://query.wikidata.org/sparql",
                    agent="movie-tutorial/1.0 (you@example.org)")
  sw.setQuery("SELECT ?c WHERE { wd:Q42 wdt:P31 ?c }")
  sw.setReturnFormat(JSON)
  print(sw.query().convert()["results"]["bindings"])
```

PYTHON

1 Parse local Turtle into an in-memory graph.

2 `initBindings` injects values safely — never build queries by string concatenation (SPARQL injection).

3 Rows allow attribute access: The Matrix 8.7, Cloud Atlas 7.4, John Wick 7.4.

4 Wikidata requires a descriptive **User-Agent**; anonymous clients get blocked.



TIP

For analytics, convert results to pandas: build a DataFrame from `[row.asdict() for row in result]` or use the JSON bindings.

Under the hood: the SPARQL algebra

Every query is translated into an operator tree. Understanding it explains OPTIONAL, FILTER scope and optimisation.

```
SELECT ?title ?year WHERE {  
  ?m ex:title ?title ;  
    ex:released ?year .  
  OPTIONAL { ?m ex:rating ?r }  
  FILTER (?year >= 2000)  
}  
ORDER BY DESC(?year)  
LIMIT 10
```

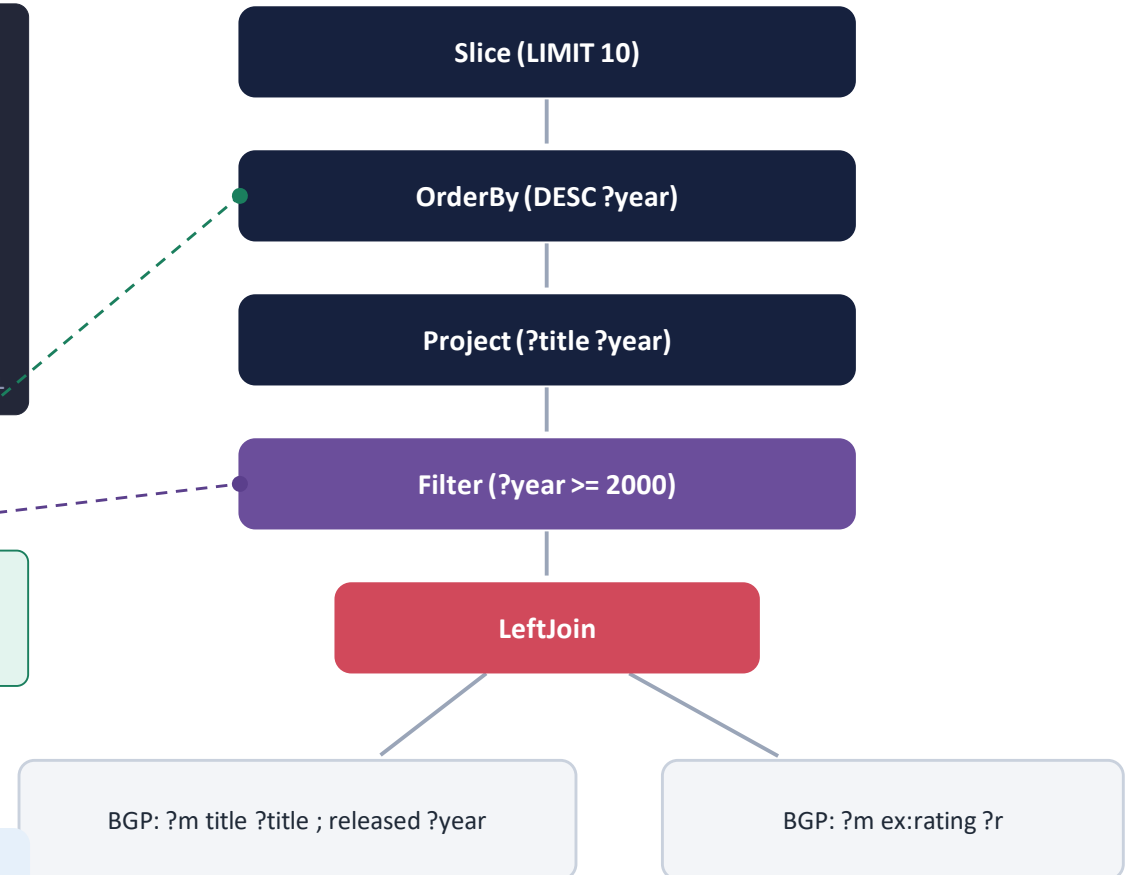
SPARQL

The FILTER sits **above** the LeftJoin: it applies to the whole group, even though it is written after OPTIONAL.

Solution modifiers wrap the pattern from the inside out: Project → OrderBy → Slice.



Jena: `qparse --print=op query.rq` prints this algebra (SSE syntax).



How SPARQL engines optimise your query



Triple indexes

Stores keep several sorted permutations (SPO, POS, OSP, often with G for graphs). Any pattern with bound terms becomes a **range scan**.



Selectivity estimation

Statistics (predicate counts, distinct subjects/objects) estimate how many rows each pattern yields; the rarest pattern goes first.



Join ordering

BGPs are reordered freely. Hash, merge and index-nested-loop joins are chosen per step.



Filter placement

FILTERs are pushed as close to the data as possible; equality filters (`?x = ex:a`) may become constant substitution.



Bind joins for SERVICE

Federating engines batch local bindings into **VALUES** blocks sent to the remote endpoint.



Explain plans

Most stores have one: GraphDB `onto:explain`, Stardog `query explain`, Jena `--explain`, Virtuoso `explain()`. Read them.

Performance playbook for SPARQL

Anchor early. Start from constants or **VALUES**; never leave both ends of a path open.

Avoid cartesian products. Every pattern should share a variable with the rest of its group.

Type your data. Numeric and date filters on string literals disable optimisations and silently fail.

Prefer FILTER NOT EXISTS over OPTIONAL + `!bound()` — clearer, and usually better optimised.

Project only what you need. `SELECT *` with OPTIONALS drags wide rows through the plan.

Aggregate in subqueries before joining to labels and other metadata.

Use full-text / geo indexes rather than REGEX or distance maths in FILTERs.

Set timeouts and LIMITs on exploratory queries against large stores.



RULE OF THUMB

If a query is slow, first ask: *which pattern is the engine starting from, and how many rows does it produce?*



PITFALL

`FILTER(?x = ?y)` between two variables is a join condition in disguise — reuse the same variable instead so the engine can use indexes.



PRO MOVE

Measure with realistic data volumes; a query that is instant on 10k triples may time out on 1B.

Anti-patterns and their fixes

✗ Scan everything, then filter

```
SELECT ?s ?p WHERE {  
  ?s ?p ?o .  
  FILTER (?o = "Keanu Reeves")  
}  
  
SELECT ?m WHERE {  
  ?m ex:title ?t .  
  ?p ex:name ?n .  
}
```

SPARQL

The first scans all triples; the second has no shared variable — a **cartesian product** ($4 \times 7 = 28$ rows).

✓ Let the index do the work

```
SELECT ?s ?p WHERE {  
  ?s ?p "Keanu Reeves" .  
}  
  
SELECT ?m ?n WHERE {  
  ?p ex:actedIn ?m ;  
  ex:name ?n .  
}
```

SPARQL

The literal is placed **in the pattern** (an OSP/POS lookup). Patterns are connected through **?p**.



Two more classics: `FILTER (REGEX (STR (?s), "keanu"))` over IRIs (use the IRI directly), and `OPTIONAL { ?x ?p ?o }` "just in case" — every unneeded `OPTIONAL` is an extra left join.

SHACL: validating RDF data with shapes

Where RDFS infers, SHACL checks. Shapes are RDF too, and validation produces an RDF report.

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix ex: <http://example.org/> .
@prefix xsd: <http://www.w3.org/2001/XMLSchema#> .
```

```
ex:MovieShape a sh:NodeShape ;
```

1 `sh:targetClass ex:Movie ;`

```
  sh:property [
    sh:path ex:title ;
```

2 `sh:minCount 1 ; sh:maxCount 1 ;`

3 `sh:datatype xsd:string ;`

```
  ] ;
```

```
  sh:property [
    sh:path ex:rating ;
    sh:datatype xsd:decimal ;
```

4 `sh:minInclusive 0 ; sh:maxInclusive 10 ;`

```
  ] .
```

SHACL

1 **Target:** every node of type `ex:Movie` is checked.

2 Cardinality: exactly one title.

3 Datatype constraint on the value.

4 Value range: a rating of 12 or "8.7" (string) would be reported.



TIP

Validators: pySHACL, Jena SHACL, TopBraid, RDF4J, GraphDB.
`sh:sparql` constraints let you write arbitrary rules as SPARQL.



PITFALL

Our multilingual titles ("The Matrix"@en, "Matrix"@de) would **violate** this shape twice: two values, and `rdf:langString` is not `xsd:string`. Use `sh:datatype rdf:langString ; sh:uniqueLang true` instead.

Beyond the core: geospatial, full-text and vectors



GeoSPARQL (OGC)

Geometries as WKT literals (`geo:asWKT`) plus functions such as `geof:sfWithin`, `geof:sfIntersects`, `geof:distance`.

```
FILTER (geof:sfWithin (?g,  
?cityPolygon))
```

Supported by GraphDB, Jena, Stardog, Virtuoso, QLever and others.



Full-text search

Not standardised; each store offers a magic predicate or connector:

- Jena: `?s text:query "matrix"`
- GraphDB: Lucene / Elastic / OpenSearch connectors
- Virtuoso: `bif:contains`
- QLever: `ql:contains-word`



Vector similarity

Several stores now index embeddings next to triples (e.g. GraphDB similarity plugins, Stardog, Neptune Analytics), enabling **semantic search + graph expansion** — the basis of GraphRAG.



RULE OF THUMB

Extensions lock you to a vendor. Isolate them in a thin query layer so the rest of your SPARQL stays portable.

The triple-store landscape

Engine	Licence / model	Notable strengths
Apache Jena (TDB2 + Fuseki)	Open source, Java	Reference-quality SPARQL 1.1, SHACL, text & spatial indexes, great for learning and mid-size data
GraphDB (Ontotext/Graphwise)	Free & commercial	Rule-based reasoning (RDFS, OWL RL), connectors to Elastic/Lucene, GraphRAG features
Stardog	Commercial	Query-time reasoning, virtual graphs over SQL sources, knowledge-graph platform
Virtuoso (OpenLink)	Open source & commercial	Hybrid relational/RDF engine; powers DBpedia and many LOD endpoints
QLever	Open source, C++	Extremely fast on very large graphs (full Wikidata on one server), text search
Amazon Neptune	Managed cloud	SPARQL, openCypher and Gremlin on one service
Oxigraph · RDFox	Rust OSS · commercial	Oxigraph: tiny embeddable store; RDFox: in-memory Datalog reasoning
Blazegraph	Open source, unmaintained	Historically behind the Wikidata Query Service, which is migrating to a new backend

 **TIP**
Benchmark with **your** queries and data shape. Published benchmarks (BSBM, WatDiv, LDBC SPB) rarely match production workloads.

Checkpoint III — advanced SPARQL

Q1

All superclasses of `ex:Actor`, excluding `ex:Actor` itself.

```
SELECT ?c WHERE {  
  ex:Actor rdfs:subClassOf+ ?c .  
}  
# + (not *) excludes the zero-length path
```

Q2

Convert every `ex:rating` stored as a string into an `xsd:decimal`.

```
DELETE { ?m ex:rating ?o }  
INSERT { ?m ex:rating ?n }  
WHERE { ?m ex:rating ?o .  
  FILTER (DATATYPE(?o) = xsd:string)  
  BIND (xsd:decimal(?o) AS ?n) }
```

Q3

What does `GRAPH ?g { ?s ?p ?o }` **not** match?

✓ ANSWER

Triples in the **default graph** — `GRAPH ?g` ranges over named graphs only (unless the store is configured with a union default graph).

Q4

Why is `?a ex:knows* ?b` (both ends unbound) dangerous on a large social graph?

✓ ANSWER

It computes the **full transitive closure** — potentially every pair of connected people, plus every node paired with itself (zero-length paths).

04

PART IV

Property Graphs & Cypher Fundamentals

Patterns drawn in ASCII art, a pipeline of clauses, and first-class relationships with their own properties.

LPG model

Pattern syntax

CREATE

MATCH / RETURN

WHERE

OPTIONAL MATCH

Aggregation

WITH

Lists & maps

UNWIND

CASE

NULL

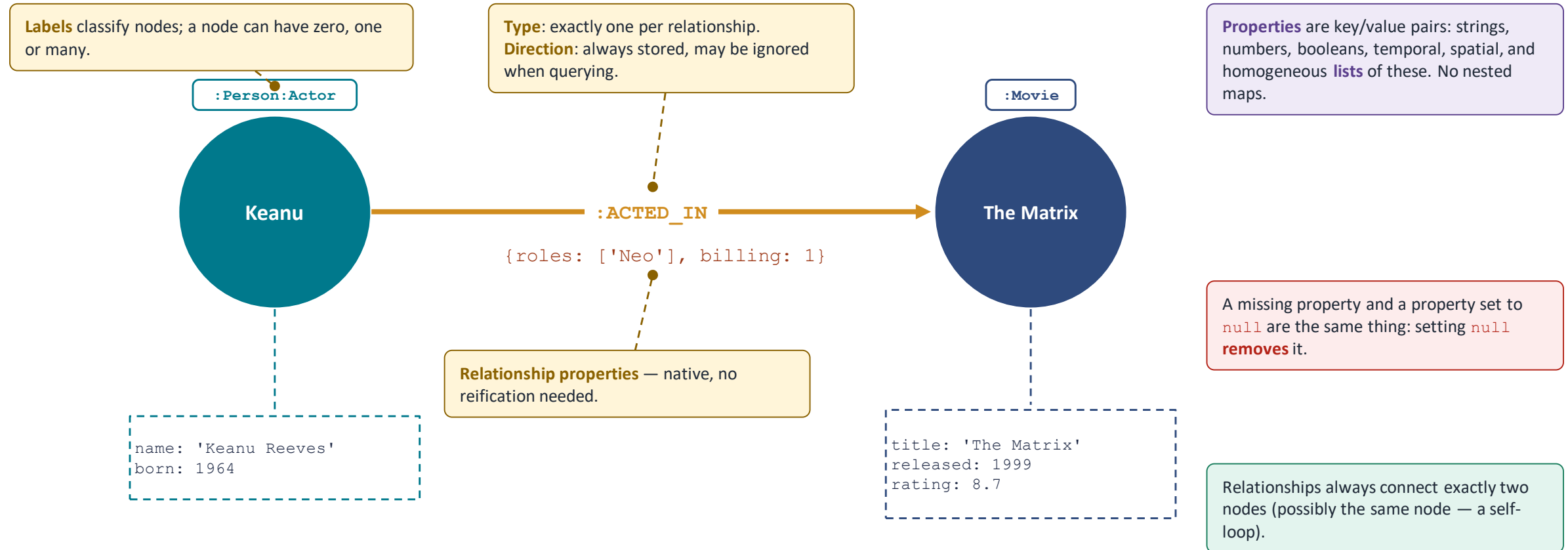
MERGE

SET / DELETE



The labeled property graph model

Four ingredients: nodes, labels, relationships (with exactly one type and a direction) and properties on both.



Building blocks and naming conventions



Node (n)

Written in parentheses. `()` anonymous, `(p)` bound to a variable, `(p:Person)` with a label.



Label :Person

UpperCamelCase by convention. Label expressions:
`:Person|Movie` (either), `:Person&Actor` (both),
`:!Movie` (not).



Relationship -[r:TYPE]->

Square brackets on an arrow. Types in UPPER_SNAKE_CASE: `ACTED_IN`, `DIRECTED`. `-->` any type, `--` any direction.



Properties {k: v}

lowerCamelCase keys. Inline maps in patterns are equality tests: `(p:Person {name: 'Keanu Reeves'})`.



Path p = (...)-[...]-(...)

An alternating sequence of nodes and relationships.
Functions: `length(p)`, `nodes(p)`,
`relationships(p)`.



Identity

`elementId(n)` is the internal id — **not** stable across deletes or copies. Use your own business keys with uniqueness constraints.



NOTE

Keywords are case-insensitive (`match = MATCH`); labels, types, property keys and variables are **case-sensitive**.

From Cypher to GQL: a short history

Cypher's pattern syntax became the basis of the first new ISO database language since SQL.

2011



Cypher is born

Introduced in Neo4j 1.4 as a declarative alternative to imperative traversal APIs.

2015



openCypher

Neo4j opens the language; adopted by Memgraph, SAP HANA Graph, Amazon Neptune, AgensGraph and others.

2019



GQL project

ISO/IEC JTC 1 approves work on GQL, a standard graph query language, alongside SQL/PGQ.

2023



SQL/PGQ + QPP

SQL:2023 adds property-graph queries (Part 16). Neo4j 5 ships GQL-style quantified path patterns.

2024



ISO GQL published

ISO/IEC 39075:2024. Cypher and GQL share patterns, label expressions and path modes.

2025+



Cypher 25

Versioned Cypher in Neo4j with continued GQL alignment and new clauses.



INSIGHT

Learning Cypher today is learning GQL tomorrow: the MATCH/pattern core is shared. Differences are mostly in DDL, graph types and some clause names (e.g. GQL `INSERT` vs. Cypher `CREATE`).

Patterns are ASCII art

If you can draw it on a whiteboard, you can write it in Cypher.

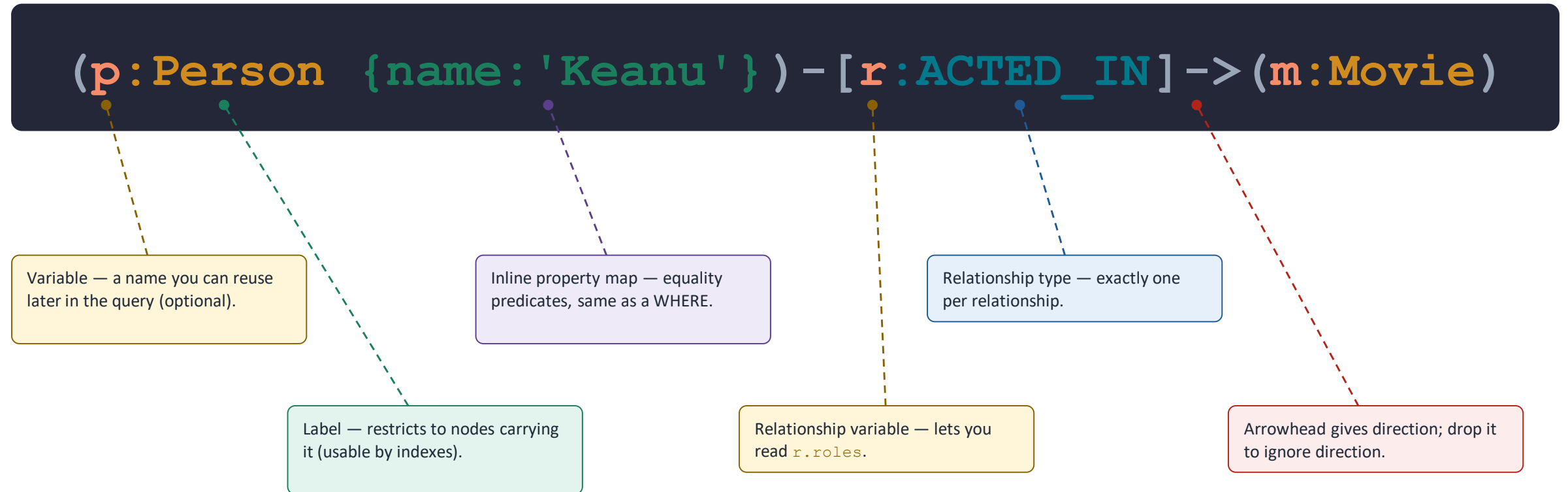
```
1 (p:Person {name: 'Keanu Reeves'})
2 (p) -[:ACTED_IN]->(m:Movie)
3 (m) <-[:DIRECTED]-(d)
4 (a) -[:KNOWS]-(b)
5 (a) -[r:ACTED_IN {roles: ['Neo']}]->(m)
6 (a) -[:KNOWS*1..3]->(b)
7 p = (a) -[:KNOWS]->()-[:KNOWS]->(c)
```

CYPHER

- 1 A node with label and property constraint — the anchor of most queries.
- 2 Outgoing relationship of type `ACTED_IN`.
- 3 Arrows can point either way; the pattern reads naturally from the film.
- 4 No arrowhead = match **either direction**.
- 5 Bind the relationship to `r` and constrain its properties.
- 6 Variable length: 1 to 3 hops.
- 7 Name the whole path; `()` is an anonymous intermediate node.

Anatomy of a pattern, piece by piece

Every character has a job. Colours below match the annotations.



Writing data: CREATE

CREATE adds exactly the pattern you describe — nodes, relationships and properties.

```
1 CREATE (k:Person {name: 'Keanu Reeves', born: 1964})
2 CREATE (m:Movie {title: 'The Matrix', released: 1999, rating: 8.7})
3 CREATE (k)-[:ACTED_IN {roles: ['Neo']}]->(m)
4 RETURN k, m
```

CYPHER



PITFALL

CREATE **always** creates. Running this statement twice gives two Keanus, two Matrices and two relationships. Use **MERGE** (slide 106) for idempotent loads.



TIP

Relationship properties such as `roles` are the LPG answer to the "where does Neo go?" question from Part I.

1 New node with a label and two properties; bound to `k` for the rest of the statement.

2 Reusing `k` and `m` connects the **existing** variables — no new nodes. Relationships must have a type and a direction.

3 **RETURN** is optional for writes; Neo4j Browser draws returned nodes.

The running dataset in Cypher (one statement)

Paste both columns as ONE statement into Neo4j Browser — variables must stay in scope from nodes to relationships.

① nodes...

```
CREATE
(keanu:Person {name:'Keanu Reeves', born:1964}),
(carrie:Person {name:'Carrie-Anne Moss', born:1967}),
(laurence:Person {name:'Laurence Fishburne', born:1961}),
(hugo:Person {name:'Hugo Weaving', born:1960}),
(lana:Person {name:'Lana Wachowski', born:1965}),
(lilly:Person {name:'Lilly Wachowski', born:1967}),
(chad:Person {name:'Chad Stahelski', born:1968}),
(matrix:Movie {title:'The Matrix', released:1999, rating:8.7}),
(reloaded:Movie {title:'The Matrix Reloaded',
                  released:2003, rating:7.2}),
(wick:Movie {title:'John Wick', released:2014, rating:7.4}),
(atlas:Movie {title:'Cloud Atlas', released:2012, rating:7.4}),
```

CYPHER

11 nodes: 7 :Person, 4 :Movie.

② ... continued: relationships

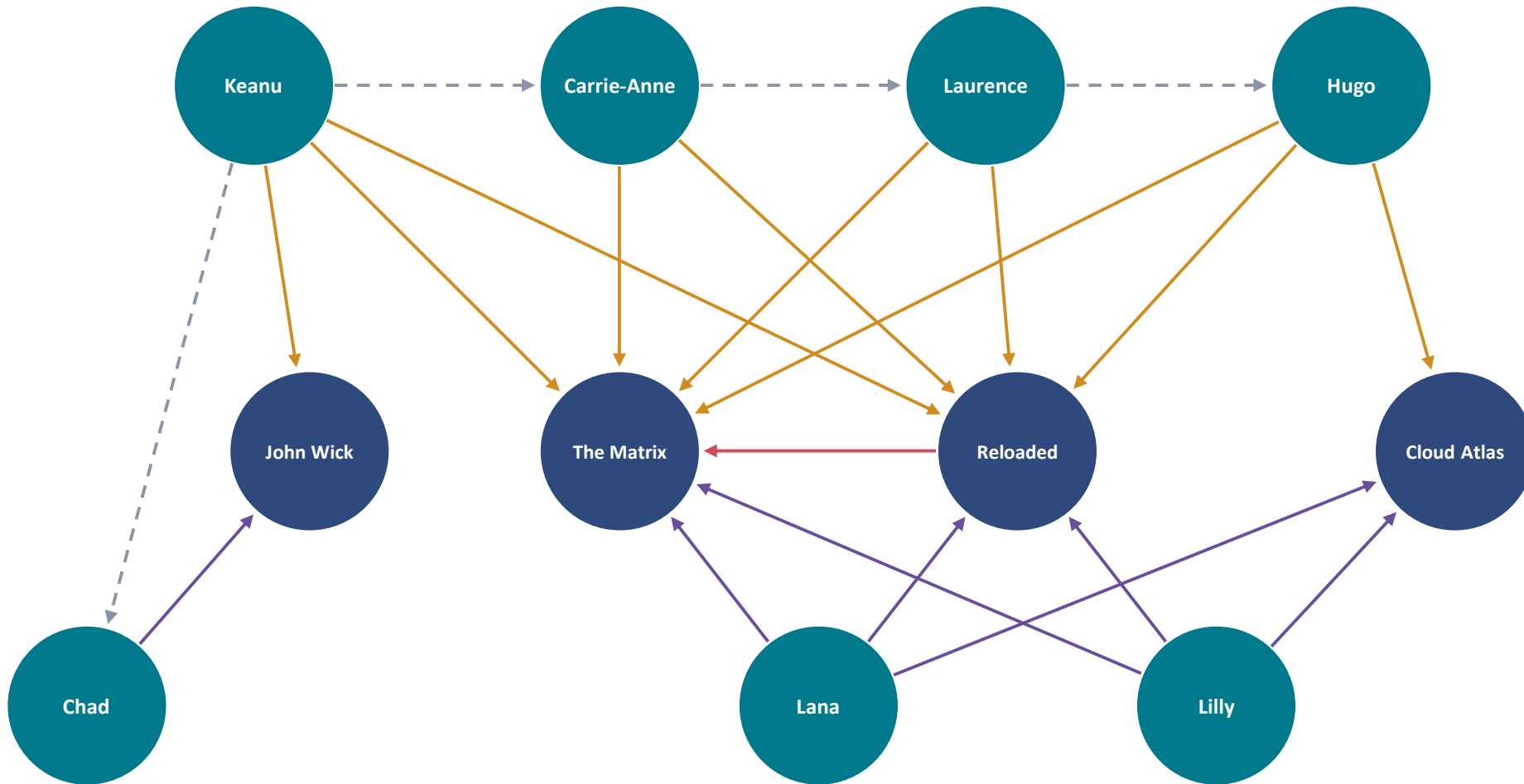
```
(keanu)-[:ACTED_IN {roles:['Neo']}]>(matrix),
(keanu)-[:ACTED_IN {roles:['Neo']}]>(reloaded),
(keanu)-[:ACTED_IN {roles:['John Wick']}]>(wick),
(carrie)-[:ACTED_IN {roles:['Trinity']}]>(matrix),
(carrie)-[:ACTED_IN {roles:['Trinity']}]>(reloaded),
(laurence)-[:ACTED_IN {roles:['Morpheus']}]>(matrix),
(laurence)-[:ACTED_IN {roles:['Morpheus']}]>(reloaded),
(hugo)-[:ACTED_IN {roles:['Agent Smith']}]>(matrix),
(hugo)-[:ACTED_IN {roles:['Agent Smith']}]>(reloaded),
(hugo)-[:ACTED_IN {roles:['Bill Smoke','Nurse Noakes']}]>(atlas),
(lana)-[:DIRECTED]>(matrix), (lana)-[:DIRECTED]>(reloaded),
(lana)-[:DIRECTED]>(atlas), (lilly)-[:DIRECTED]>(matrix),
(lilly)-[:DIRECTED]>(reloaded), (lilly)-[:DIRECTED]>(atlas),
(chad)-[:DIRECTED]>(wick),
(keanu)-[:KNOWS]>(carrie), (carrie)-[:KNOWS]>(laurence),
(laurence)-[:KNOWS]>(hugo), (keanu)-[:KNOWS]>(chad),
(reloaded)-[:SEQUEL_OF]>(matrix)
```

CYPHER

22 relationships: 10 ACTED_IN, 7 DIRECTED, 4 KNOWS, 1 SEQUEL_OF.

The dataset as Neo4j Browser would draw it

Circles are nodes (teal :Person, navy :Movie); arrows are typed relationships. Compare with the RDF drawing on slide 18.



LEGEND

- ACTED_IN
- DIRECTED
- - → KNOWS
- SEQUEL_OF

Explore it yourself: `MATCH (n) RETURN n` shows everything. Only do that on small graphs!

Same 22 edges as the RDF version, but names, years and ratings live **inside** the nodes — 11 nodes instead of 59 triples.

MATCH and RETURN: your first read query

MATCH finds every occurrence of the pattern; RETURN projects the columns you want.

```
1 MATCH (k:Person {name: 'Keanu Reeves'}) -[:ACTED_IN]->(m:Movie)
2 RETURN m.title AS title,
        m.released AS year
3 ORDER BY year
```

CYPHER

RESULT

title	year
The Matrix	1999
The Matrix Reloaded	2003
John Wick	2014



TIP

Compare with slide 29: the SPARQL version needed two triple patterns joined on `?movie`. Here the connection is expressed by drawing the relationship.

- 1 Anchor node + outgoing `ACTED_IN` + target labelled `:Movie`. One row per match.
- 2 `m.title` reads a property; `AS` names the output column.
- 3 `ORDER BY` may use the aliases defined in RETURN.

WHERE: filtering on nodes, relationships and lists

WHERE attaches predicates to the preceding MATCH. Relationship properties are filterable like node properties.

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
1 WHERE m.released >= 2000
2 AND p.born < 1965
3 AND 'Agent Smith' IN r.roles
RETURN p.name, m.title, r.roles
```

CYPHER

1 Comparison on a node property.

2 Keanu (1964), Laurence (1961) and Hugo (1960) qualify.

3 **IN** tests list membership — here on a **relationship** property.

RESULT

p.name	m.title	r.roles
Hugo Weaving	The Matrix Reloaded	['Agent Smith']

INSIGHT

Inline maps `{name: ...}` only express equality. Anything else — ranges, OR, NOT, regex, list checks — goes in WHERE.

Cypher operators — reference

Category	Operators	Example
Comparison	<code>= <> < > <= >=</code>	<code>m.rating >= 7.4</code>
Boolean	<code>AND OR XOR NOT</code>	<code>NOT p.born > 1965</code>
String	<code>STARTS WITH, ENDS WITH, CONTAINS, =~ (regex)</code>	<code>m.title STARTS WITH 'The'</code>
Null	<code>IS NULL, IS NOT NULL</code>	<code>p.email IS NULL</code>
List	<code>IN, [i], [a..b], + (concatenation)</code>	<code>'Neo' IN r.roles</code>
Math	<code>+ - * / % ^</code>	<code>m.rating * 10</code>
Property access	<code>n.key, n['key'], n[\$key] (dynamic)</code>	<code>p['name']</code>
Label / type tests	<code>n:Label, n:A B, n:!A, type(r) = 'X'</code>	<code>WHERE n:Person Movie</code>
Type predicate	<code>IS :: TYPE</code>	<code>m.rating IS :: FLOAT</code>
Pattern predicate	<code>EXISTS { ... }, COUNT { ... }</code>	<code>EXISTS { (p)-[:DIRECTED]->() }</code>



PITFALL

String concatenation uses `+` and Cypher does **not** coerce types: `'Year ' + 1999` works in Neo4j, but comparisons such as `'5' = 5` are simply false. Convert with `toString()`, `toInteger()`, `toFloat()`.

String matching in Cypher

Three separate queries showing the three styles of text predicate.

```
1 MATCH (m:Movie)
   WHERE m.title STARTS WITH 'The Matrix'
   RETURN m.title;

MATCH (p:Person)
2 WHERE p.name =~ '(?i)lana.*'
   RETURN p.name;

MATCH (p:Person)
3 WHERE toLower(p.name) CONTAINS 'wachowski'
   RETURN p.name;
```

CYPHER

1 → The Matrix, The Matrix Reloaded. **STARTS WITH** can use a range or text index.

2 Java regex; **(?i)** = case-insensitive. **=~** must match the **whole** string → Lana Wachowski.

3 Normalising case with **toLower()** → Lana and Lilly Wachowski. Functions on the property prevent index use.



PRO MOVE

For real search (stemming, fuzzy match, scoring) create a **full-text index** and call `db.index.fulltext.queryNodes()` — see Part V.



INSIGHT

All three predicates are case-sensitive by default and return `null` (row dropped) when the property is missing.

Direction and variable-length relationships

Arrowheads constrain direction; an asterisk turns a single hop into a range of hops.

```
// Directed: who does Keanu know?  
1 MATCH (:Person {name:'Keanu Reeves'}) -[:KNOWS]->(f)  
  RETURN f.name;  
  
// Undirected: KNOWS neighbours in either direction  
2 MATCH (:Person {name:'Carrie-Anne Moss'}) -[:KNOWS]-(f)  
  RETURN f.name;  
  
// Variable length: 1 to 3 hops  
3 MATCH (:Person {name:'Keanu Reeves'}) -[:KNOWS*1..3]->(f)  
  RETURN DISTINCT f.name;
```

CYPHER

1 → Carrie-Anne Moss, Chad Stahelski.

2 → Keanu Reeves (incoming) and Laurence Fishburne (outgoing).

3 → Carrie-Anne, Chad (1 hop), Laurence (2), Hugo (3). * alone means 1..∞; *0.. includes the start node.



PITFALL

An unbounded `-[*]-` on a large, well-connected graph enumerates **paths**, not nodes — the number of paths can grow exponentially. Always set an upper bound or use shortest-path / QPP syntax (Part V).

OPTIONAL MATCH: outer joins with nulls

Like SPARQL OPTIONAL: missing parts produce null instead of eliminating the row.

```
1 MATCH (m:Movie)
2 OPTIONAL MATCH (m)-[:SEQUEL_OF]->(prev:Movie)
RETURN m.title AS title,
       prev.title AS sequelOf
ORDER BY title
```

CYPHER

RESULT

title	sequelOf
Cloud Atlas	null
John Wick	null
The Matrix	null
The Matrix Reloaded	The Matrix

 **PITFALL**

A **WHERE** directly after **OPTIONAL MATCH** belongs **to** the optional part (it only nulls out the match). To drop rows, put the filter after a **WITH**.

1

Mandatory part: all 4 movies.

2

If no **SEQUEL_OF** exists, **prev** is **null** — and **prev.title** is null too.

 **INSIGHT**

This is exactly the "FILTER inside vs. outside OPTIONAL" distinction from slide 36, expressed with clause position.

Shaping output: DISTINCT, ORDER BY, SKIP, LIMIT

Same ideas as SPARQL, different keywords. SKIP is Cypher's OFFSET.

```
MATCH (m:Movie)
1 RETURN DISTINCT m.rating AS rating
2 ORDER BY rating DESC
3 SKIP 1
4 LIMIT 2
```

CYPHER

RESULT

rating
7.4
7.2



TIP

SKIP and LIMIT accept parameters and expressions (LIMIT \$pageSize), which keeps paging queries cacheable.

- 1 Distinct ratings: 8.7, 7.4 (twice → once), 7.2.
- 2 DESC/DESCENDING, ASC/ASCENDING; nulls sort last ascending, first descending.
- 3 Skip 8.7...
- 4 ...keep the next two.

Aggregation with implicit grouping

There is no `GROUP BY`: every non-aggregated expression in `RETURN` or `WITH` becomes a grouping key.

```

MATCH (p:Person) -[:ACTED_IN]->(m:Movie)
1 RETURN p.name AS actor,
2 count(m) AS films,
3 collect(m.title) AS titles,
4 round(avg(m.rating), 2) AS avgRating
ORDER BY films DESC, actor
  
```

RESULT

actor	films	titles	avgRating
Hugo Weaving	3	['The Matrix', 'The Matrix Reloaded', 'Cloud Atlas']	7.77
Keanu Reeves	3	['The Matrix', 'The Matrix Reloaded', 'John Wick']	7.77
Carrie-Anne Moss	2	['The Matrix', 'The Matrix Reloaded']	7.95
Laurence Fishburne	2	['The Matrix', 'The Matrix Reloaded']	7.95



PITFALL

Adding `m.title` as a plain column would change the grouping to `(actor, title)` — every group of size 1. Implicit grouping is convenient but easy to break.

- 1 `p.name` is not aggregated → it is the **grouping key**.
- 2 Rows per group.
- 3 `collect()` builds a list — Cypher's `GROUP_CONCAT`, but typed.
- 4 Keanu: $(8.7 + 7.2 + 7.4) / 3 = 7.77$.

Aggregating functions

Function	Returns	Null handling
<code>count (*)</code>	Number of rows	Counts every row
<code>count (expr)</code>	Rows with non-null expr	Ignores nulls
<code>count (DISTINCT expr)</code>	Distinct non-null values	Ignores nulls
<code>sum / avg</code>	Numeric total / mean	Ignores nulls; avg of nothing = null
<code>min / max</code>	Extremes	Work on strings, numbers, temporals
<code>collect (expr)</code>	List of values	Drops nulls
<code>percentileCont / Disc</code>	Interpolated / actual percentile	e.g. median = 0.5
<code>stDev / stDevP</code>	Sample / population std. dev.	Ignores nulls

 INSIGHT

`count (*)` after an `OPTIONAL MATCH` counts the null rows too; `count (m)` does not. That is how you count "zero films" correctly.

 TIP

Aggregate inside a list comprehension for per-row maths: `reduce (t = 0, x IN xs | t + x)`.

WITH: chaining query parts into a pipeline

WITH is a mid-query RETURN: it projects, aggregates and filters, then hands the rows to the next clause.

```

MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
1 WITH p, count(m) AS films
2 WHERE films >= 3
3 MATCH (p)-[:ACTED_IN]->(m2:Movie)
4 WITH p, films, max(m2.rating) AS best
RETURN p.name AS actor, films, best
ORDER BY actor
  
```

- 1 Aggregate per person. Only **p** and **films** survive — **m** is **out of scope** afterwards.
- 2 WHERE after WITH filters aggregated rows — Cypher's HAVING.
- 3 Continue matching from the surviving people.
- 4 A second aggregation stage.

RESULT

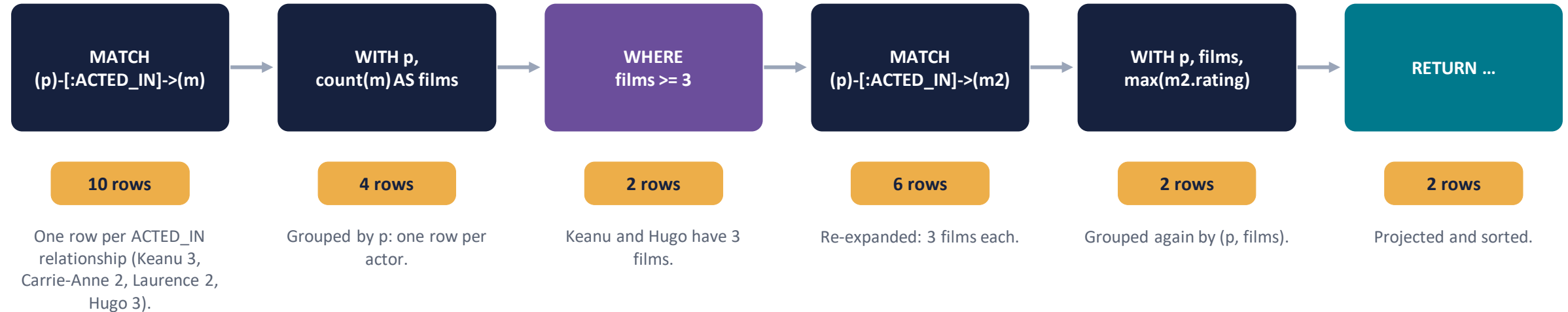
actor	films	best
Hugo Weaving	3	8.7
Keanu Reeves	3	8.7

INSIGHT

Where SPARQL nests subqueries, Cypher **chains** stages. Each **WITH** is a checkpoint: you decide exactly which variables flow on.

Visualising the WITH pipeline: rows flowing through stages

The query from the previous slide as a data flow. Numbers show how many rows leave each stage.



PRO MOVE

Keep row counts small early: aggregate or `LIMIT` in an early `WITH` before expanding again. Every row entering an expansion multiplies the work.



TIP

`WITH DISTINCT p` de-duplicates between stages; `WITH *` passes everything along; `WITH p ORDER BY ... LIMIT 10` enables top-k per stage.

Lists: indexing, slicing, comprehensions and reduce

Lists are first-class values — the key to expressive single-query transformations.

```
1 WITH [1999, 2003, 2012, 2014] AS years
2 RETURN years[0] AS first,
3        years[-1] AS last,
4        years[1..3] AS middle,
5        size(years) AS n,
6        [y IN years WHERE y > 2000 | y - 2000] AS since2000,
7        reduce(s = 0, y IN years | s + y) AS total
```

CYPHER

RESULT

first	last	middle	n	since2000	total
1999	2014	[2003, 2012]	4	[3, 12, 14]	8028



TIP

Other list functions: `head`, `last`, `tail`, `range(0, 10, 2)`, `reverse`, `any/all/none/single(x IN l WHERE ...)`.

- 1 A literal list bound with `WITH` — no `MATCH` needed.
- 2 Negative indexes count from the end.
- 3 Slices are half-open: indexes 1 and 2.
- 4 Comprehension: `[x IN list WHERE filter | map]`.
- 5 `reduce(acc = init, x IN list | expr)` — a fold.

Maps and map projections: returning JSON-shaped documents

Map projection builds nested structures straight from the graph — ideal for APIs.

```
1 MATCH (m:Movie {title: 'The Matrix'})<-[r:ACTED_IN]-(p:Person)
2 RETURN m {
3   .title,
   .released,
   cast: collect(p { .name, role: r.roles[0] })
} AS movie
```

CYPHER

RESULT

```
{
  title: 'The Matrix',
  released: 1999,
  cast: [
    {name: 'Keanu Reeves',    role: 'Neo'},
    {name: 'Carrie-Anne Moss', role: 'Trinity'},
    {name: 'Laurence Fishburne', role: 'Morpheus'},
    {name: 'Hugo Weaving',    role: 'Agent Smith'}
  ]
}
```

JSON

1 `m { ... }` starts a projection of node `m`.

2 `.title` copies a property with the same key.

3 Nested projection per actor, aggregated with `collect()` — implicit grouping by `m`.



TIP

`m { .* }` copies all properties; `m { .*, score: 1 }` adds computed keys. Order of the `cast` list is not guaranteed.

UNWIND: turning a list into rows

The standard way to write many records in one statement — and the basis of batched imports.

```
1 UNWIND [
  {title: 'John Wick: Chapter 2', released: 2017},
  {title: 'John Wick: Chapter 3', released: 2019}
] AS row
2 MERGE (m:Movie {title: row.title})
  SET m.released = row.released
  WITH m
3 MATCH (k:Person {name: 'Keanu Reeves'})
  MERGE (k) -[:ACTED_IN]->(m)
```

CYPHER

1 One row per list element; `row` is a map.

2 Idempotent upsert per row.

3 The `MATCH` runs once per surviving row.



PRO MOVE

In applications pass the list as a parameter: `UNWIND $rows AS row ...` with 1k–10k rows per call. This is far faster than one query per record.



PITFALL

`UNWIND []` produces **zero rows**, silently ending the pipeline. Guard with `UNWIND CASE WHEN size(l) = 0 THEN [null] ELSE l END AS x`.

CASE expressions: conditional logic

Two forms: generic (WHEN condition) and simple (compare one value against cases).

```
MATCH (m:Movie)
RETURN m.title,
1 CASE
    WHEN m.rating >= 8.5 THEN 'classic'
    WHEN m.rating >= 7.3 THEN 'solid'
2 ELSE 'mixed'
END AS verdict,
3 CASE m.released WHEN 1999 THEN 'origin'
    ELSE 'later' END AS era
ORDER BY m.released
```

CYPHER

1 Generic form: the first true **WHEN** wins.

2 Without **ELSE**, the result is **null**.

3 Simple form: equality against a single expression.

RESULT

m.title	verdict	era
The Matrix	classic	origin
The Matrix Reloaded	mixed	later
Cloud Atlas	solid	later
John Wick	solid	later

NULL semantics: three-valued logic

null means "unknown". Most operations involving null return null — and WHERE drops null rows.

Expression	Result
<code>null = null</code>	<code>null</code>
<code>null <> 1</code>	<code>null</code>
<code>null IS NULL</code>	<code>true</code>
<code>1 + null</code>	<code>null</code>
<code>null AND false</code>	<code>false</code>
<code>null AND true</code>	<code>null</code>
<code>null OR true</code>	<code>true</code>
<code>null IN [1, 2]</code>	<code>null</code>
<code>2 IN [1, null, 2]</code>	<code>true</code>
<code>3 IN [1, null, 2]</code>	<code>null</code>
<code>m.missingKey</code>	<code>null</code>
<code>count(null)</code>	<code>0</code>

PITFALL

`WHERE p.email <> 'x'` silently excludes everyone **without** an email — the comparison is null, not true. Write `p.email IS NULL OR p.email <> 'x'`.

INSIGHT

Compare SPARQL: an unbound variable in a FILTER raises an **error** that eliminates the row. Different mechanism, same practical effect.

TIP

`coalesce(p.nick, p.name, 'unknown')` returns the first non-null value.

MERGE: match-or-create (the upsert)

MERGE looks for the whole pattern; if it is not found, it creates the whole pattern.

```
1 MERGE (p:Person {name: 'Keanu Reeves'})
2   ON CREATE SET p.born = 1964, p.createdAt = datetime()
3   ON MATCH SET p.lastSeen = datetime()
4 MERGE (m:Movie {title: 'John Wick'})
5 MERGE (p) -[r:ACTED_IN]->(m)
6   ON CREATE SET r.roles = ['John Wick']
7 RETURN p.name, m.title, r.roles
```

CYPHER



RULE OF THUMB

MERGE nodes on their key, then MERGE the relationship between bound nodes. Never put non-key properties in the MERGE pattern.



PRO MOVE

Back every merged key with a **uniqueness constraint**: it makes MERGE use an index and prevents duplicates under concurrent writes.

- 1 Merge on the **key** only — the property that identifies a Person.
- 2 Runs only if a new node was created: set the remaining properties here.
- 3 Runs only if an existing node was found.
- 4 With both ends bound, MERGE only decides whether the relationship exists.

The classic MERGE trap

MERGE is all-or-nothing for the entire pattern it is given.

✗ One big MERGE

```
// Keanu exists, Speed does not.  
MERGE (p:Person {name:'Keanu Reeves'})  
  -[:ACTED_IN]->  
    (m:Movie {title:'Speed'})
```

CYPHER

The full path does not exist, so MERGE **creates the full path** — including a *second* Keanu Reeves node.

⚠️ PITFALL

Also avoid `MERGE (m:Movie {title:'Speed', rating: 7.3})`: if a Speed node exists with a different or missing rating, a duplicate is created.

✓ Merge piece by piece

```
MERGE (p:Person {name:'Keanu Reeves'})  
MERGE (m:Movie {title:'Speed'})  
MERGE (p) -[:ACTED_IN]->(m)
```

CYPHER

Each node is matched or created independently; the relationship is merged between the two bound nodes.

💡 INSIGHT

With a uniqueness constraint on `:Person(name)`, the bad version fails with a constraint error instead of silently duplicating — another reason to always declare constraints.

Updating and deleting: SET, REMOVE, DELETE

Three independent statements. Deletes are permanent — preview with the MATCH alone first.

```
1 MATCH (m:Movie {title: 'The Matrix'})
2 SET m.rating = 8.7, m:Classic
3 SET m += {genres: ['Sci-Fi', 'Action']}
4 REMOVE m.tmpFlag, m:Draft;

MATCH (p:Person {name: 'Chad Stahelski'})
5 DETACH DELETE p;

MATCH () -[r:KNOWS]->()
DELETE r;
```

CYPHER

PITFALL

`SET m = {title: 'The Matrix'}` wipes every other property (released, rating...). Use `+=` unless you really mean "replace everything".

- 1 Set a property and **add a label** in one clause.
- 2 `+=` merges a map into existing properties; `=` would **replace** all properties.
- 3 Remove a property and a label.
- 4 `DETACH DELETE` removes the node and all its relationships. Plain `DELETE` fails if relationships remain.
- 5 Delete relationships only — nodes stay.

Checkpoint IV — Cypher fundamentals

Q1

Titles of movies with **two or more** directors.

```
MATCH (m:Movie) <-[:DIRECTED]-(d:Person)
WITH m, count(d) AS n
WHERE n >= 2
RETURN m.title
// → The Matrix, The Matrix Reloaded, Cloud Atlas
```

Q2

Actors who **never** worked on a film directed by Chad Stahelski.

```
MATCH (p:Person) -[:ACTED_IN]->(:Movie)
WHERE NOT EXISTS {
  (p) -[:ACTED_IN]->(:Movie) <-[:DIRECTED]-
  (:Person {name:'Chad Stahelski'}) }
RETURN DISTINCT p.name
// → Carrie-Anne, Laurence, Hugo
```

Q3

What is the difference between `count(*)` and `count(p.email)`?

✓ ANSWER

`count(*)` counts **rows**; `count(p.email)` counts rows where `p.email` is **not null**.

Q4

Why can `MERGE (a)-[:R]->(b:B {k:1})` create duplicates of `b`?

✓ ANSWER

`MERGE` matches the **whole pattern**; if the relationship is missing it creates everything unbound in it — including a new `b`. Merge nodes separately first.

05

PART V

Advanced Cypher

Path semantics, shortest paths and quantified path patterns, subqueries, schema, execution plans, batching, algorithms and GraphRAG.

Variable-length paths

Path semantics

Shortest paths

QPP

Subqueries

CALL

Parameters

Indexes

Constraints

PROFILE

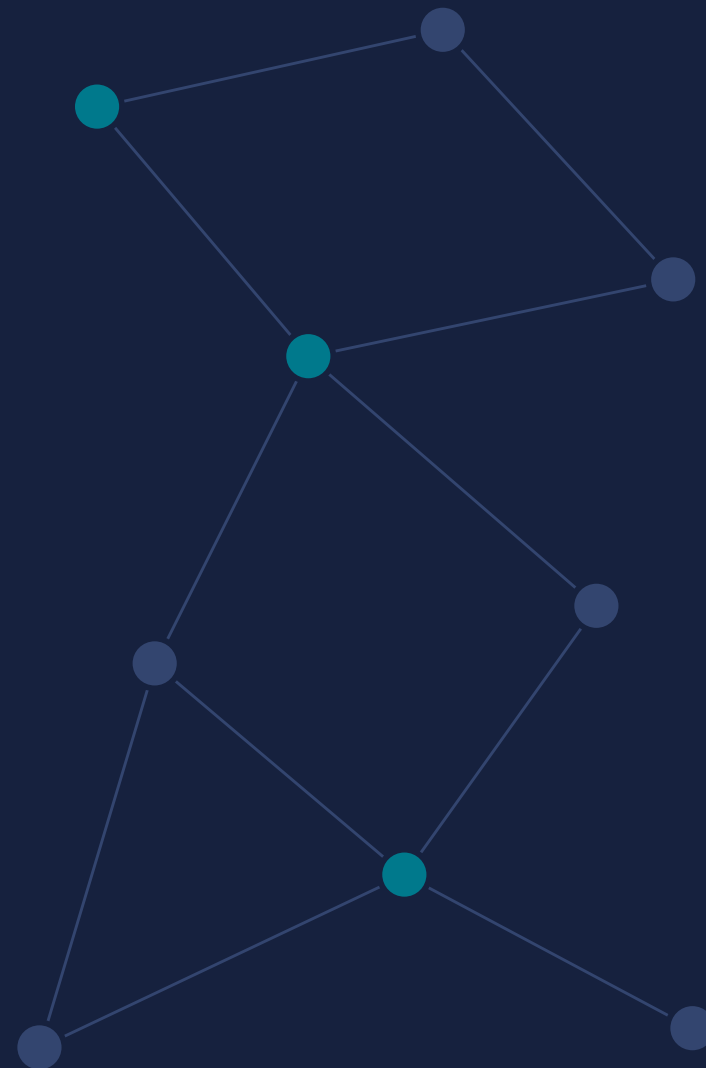
Batching

LOAD CSV

APOC

GDS

Vectors



Variable-length paths in depth

Bind the path, then inspect it: its length, its nodes, its relationships.

```
1 MATCH p = (a:Person {name: 'Keanu Reeves'})
2   -[:KNOWS*1..]->(b:Person)
3 RETURN b.name AS reach,
4   length(p) AS hops,
   [n IN nodes(p) | n.name] AS route
ORDER BY hops, reach
```

CYPHER

RESULT

reach	hops	route
Carrie-Anne Moss	1	[Keanu, Carrie-Anne]
Chad Stahelski	1	[Keanu, Chad]
Laurence Fishburne	2	[Keanu, Carrie-Anne, Laurence]
Hugo Weaving	3	[Keanu, Carrie-Anne, Laurence, Hugo]

1 `p` = names the path so we can analyse it.

2 `*1..` = at least one hop, no upper bound. Fine here; risky on big graphs.

3 `length(p)` = number of relationships.

4 Map every node on the path to its name.

INSIGHT

Unlike SPARQL property paths, Cypher returns **every path** as a separate row — including several routes to the same node on richer graphs. Add `DISTINCT b` or use shortest-path syntax if you only need reachability.

Path semantics: what counts as a valid match?



Relationship uniqueness

Within a single MATCH, the same relationship is never used twice (**trail** semantics, "different relationships"). Nodes may repeat.



Consequence

In `(a) -[:ACTED_IN]->(m) <-[:ACTED_IN]-(b)`, `a` and `b` are automatically different people — the two ACTED_IN edges must differ.



Across MATCH clauses

Uniqueness applies per MATCH. Splitting a pattern into two MATCH clauses allows the same relationship to be reused.



GQL path modes

GQL names the options: **WALK** (anything), **TRAIL** (no repeated edges), **ACYCLIC** (no repeated nodes), **SIMPLE** (acyclic except a closed loop).



Versus SPARQL

SPARQL BGPs use **homomorphism**: nothing is forced distinct. That is why slide 30 needed `FILTER(?co != ?keanu)`.



Termination

Trail semantics guarantees termination on cycles, but the **number** of trails can still be exponential. Bound the length.

shortestPath and allShortestPaths

Classic functions for "how are these two connected?" — breadth-first search under the hood.

```

1 MATCH (a:Person {name: 'Keanu Reeves'}),
   (b:Person {name: 'Hugo Weaving'})
2 MATCH p = shortestPath((a)-[:KNOWS|ACTED_IN*]-(b))
3 RETURN [n IN nodes(p) | coalesce(n.name, n.title)] AS route,
   length(p) AS hops

```

CYPHER

RESULT

route	hops
['Keanu Reeves', 'The Matrix', 'Hugo Weaving']	2

INSIGHT

Two shortest routes exist (via The Matrix or via Reloaded); `shortestPath` returns **one**, `allShortestPaths` returns both. Via KNOWS alone the distance would be 3.

PITFALL

A WHERE that filters the path after `shortestPath` may force the planner into an exhaustive search. Prefer the GQL-style syntax on the next slides.

1 Bind both endpoints first (two index seeks).

2 Any mix of `KNOWS` or `ACTED_IN`, either direction, any length.

3 `coalesce` because path nodes can be people (`name`) or films (`title`).

Quantified path patterns (QPP)

Neo4j 5 / GQL: repeat a whole sub-pattern, with predicates evaluated at every iteration.

```
MATCH (a:Person {name: 'Keanu Reeves'})
1 ( (x:Person) -[:KNOWS]-> (y:Person)
2 WHERE y.born > x.born ) {1,3}
3 (b:Person)
4 RETURN b.name AS reached
```

CYPHER

RESULT

reached

Carrie-Anne Moss

Chad Stahelski



TIP

Outside the group, `x` and `y` become **lists** (group variables): `RETURN [n IN x | n.name]` lists every hop's start node.

- 1 The parenthesised group is the unit of repetition. Its first node joins `a`, its last joins `b`.
- 2 Inline predicate checked on **each hop**: every next person must be younger. Impossible with the old `*1..3` syntax.
- 3 `{1,3}` quantifier; also `+`, `*`, `{n}` and `{n,}`.
- 4 Keanu→Carrie-Anne (1967 > 1964 ✓) → Laurence (1961 ✗ stops). Keanu→Chad (1968 ✓).

GQL-style shortest paths: SHORTEST k and ALL SHORTEST

Path selectors in the MATCH itself (Neo4j 5.21+). Planner-friendly and composable with QPP.

```
1 MATCH p = SHORTEST 1 (a:Person {name: 'Keanu Reeves'})
    ( () -[:KNOWS]- () ) {1,5}
    (b:Person {name: 'Hugo Weaving'})
2 RETURN length(p) AS hops;

MATCH p = ALL SHORTEST
    (a:Person {name: 'Keanu Reeves'})
3 -[:ACTED_IN|DIRECTED]-+
    (b:Person {name: 'Lilly Wachowski'})
4 RETURN [n IN nodes(p) | coalesce(n.name, n.title)] AS route;
```

CYPHER

1 **SHORTEST k** returns up to k shortest paths per endpoint pair.

2 Via KNOWS only: Keanu–Carrie–Anne–Laurence–Hugo → 3.

3 Quantified relationship shorthand: **- [...]-+** = one or more hops.

4 Two routes of length 2: via The Matrix and via Reloaded.



INSIGHT

Selectors: **ANY** (one path), **SHORTEST k**, **ALL SHORTEST**, **SHORTEST k GROUPS** (all paths of the k shortest lengths). These are the ISO GQL keywords.



PRO MOVE

Predicates inside the pattern are applied **during** the search, so the engine can prune early instead of filtering after.

Pattern subqueries: EXISTS, COUNT and COLLECT

Evaluate a pattern per row — no extra MATCH, no row multiplication, no grouping headaches.

```
MATCH (p:Person)
1 WHERE EXISTS { (p)-[:DIRECTED]->(:Movie) }
RETURN p.name AS director,
2 COUNT { (p)-[:DIRECTED]->( ) } AS films,
3 COLLECT { MATCH (p)-[:DIRECTED]->(m)
            RETURN m.title ORDER BY m.released } AS titles
ORDER BY director
```

CYPHER

1 **EXISTS** { } — a boolean test; the SPARQL analogue is **FILTER EXISTS**.

2 **COUNT** { } counts matches of the pattern for this row.

3 **COLLECT** { } runs a full subquery returning one column, as a list — here **ordered**.

RESULT

director	films	titles
Chad Stahelski	1	['John Wick']
Lana Wachowski	3	['The Matrix', 'The Matrix Reloaded', 'Cloud Atlas']
Lilly Wachowski	3	['The Matrix', 'The Matrix Reloaded', 'Cloud Atlas']



TIP

Pattern comprehensions are the older, still-valid form: `[(p)-[:DIRECTED]->(m) | m.title]` returns a list per row.

CALL subqueries: per-row top-k

A subquery runs once per incoming row and can sort, limit and aggregate independently.

```
1 MATCH (p:Person)
2 CALL (p) {
3   MATCH (p)-[:ACTED_IN]->(m:Movie)
4   RETURN m.title AS best
5   ORDER BY m.rating DESC
6   LIMIT 1
7 }
8 RETURN p.name AS actor, best
9 ORDER BY actor
```

CYPHER

RESULT

actor	best
Carrie-Anne Moss	The Matrix
Hugo Weaving	The Matrix
Keanu Reeves	The Matrix
Laurence Fishburne	The Matrix

1 Variable-scope clause `CALL (p)` imports `p` (Neo4j 5.23+).
Older form: `CALL { WITH p ... }`.

2 Sorting and LIMIT apply **per person** — impossible with a single ORDER BY.

3 People with no films produce no inner rows and disappear;
use `OPTIONAL CALL` to keep them.

INSIGHT

This is the Cypher answer to SPARQL's "argmax" subquery pattern (slide 46) — shorter, and ties are broken by `LIMIT 1` rather than duplicated.

UNION and UNION ALL

Combine the results of complete queries with identical column names.

```
1 MATCH (p:Person)-[:ACTED_IN]->(:Movie {title: 'Cloud Atlas'})
2 RETURN p.name AS name, 'actor' AS role
3 UNION
4 MATCH (p:Person)-[:DIRECTED]->(:Movie {title: 'Cloud Atlas'})
5 RETURN p.name AS name, 'director' AS role
```

CYPHER

RESULT

name	role
Hugo Weaving	actor
Lana Wachowski	director
Lilly Wachowski	director

 TIP

To post-process a union (sort, aggregate), wrap it: `CALL () { ... UNION ... } RETURN role, count(*) ORDER BY role.`

- 1 Each branch is a full query ending in RETURN.
- 2 UNION removes duplicate rows; UNION ALL keeps them (and is cheaper).
- 3 Column names and order must match exactly.

Parameters: safe, fast, cacheable queries

Never concatenate user input into Cypher. Pass values as \$parameters.

```
1 :param minRating => 7.3
  :param names => ['Keanu Reeves', 'Hugo Weaving']

MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
2 WHERE p.name IN $names
3 AND m.rating >= $minRating
RETURN p.name AS actor, collect(m.title) AS titles
ORDER BY actor
```

CYPHER

1 `:param` is a Neo4j Browser/cypher-shell command. In drivers you pass a dictionary.

2 `$names` — a list parameter works with `IN`.

3 The query text never changes, so its execution plan is **cached** and reused.

RESULT

actor	titles
Hugo Weaving	['The Matrix', 'Cloud Atlas']
Keanu Reeves	['The Matrix', 'John Wick']

PITFALL

Parameters cannot replace labels, types or property **names** in classic Cypher. Recent Neo4j 5 releases add dynamic labels/types (`:$($label)`); otherwise whitelist values in code.

Indexes in Neo4j 5

Indexes find anchor nodes (and relationships) fast. Without one, MATCH starts with a label scan.

Index	Create with	Serves
Range	<code>CREATE INDEX person_name FOR (p:Person) ON (p.name)</code>	Equality, ranges, <code>STARTS WITH</code> , <code>IS NOT NULL</code> , sorting
Composite	<code>CREATE INDEX movie_year_rating FOR (m:Movie) ON (m.released, m.rating)</code>	Predicates on several properties together
Text	<code>CREATE TEXT INDEX movie_title FOR (m:Movie) ON (m.title)</code>	<code>CONTAINS</code> , <code>ENDS WITH</code> on strings
Point	<code>CREATE POINT INDEX place_loc FOR (p:Place) ON (p.location)</code>	Spatial distance and bounding-box queries
Full-text	<code>CREATE FULLTEXT INDEX titles FOR (m:Movie) ON EACH [m.title]</code>	Lucene search via <code>db.index.fulltext.queryNodes</code>
Vector	<code>CREATE VECTOR INDEX plots FOR (m:Movie) ON m.embedding</code>	Approximate nearest-neighbour search on embeddings
Token lookup	(created by default)	Finding nodes by label / relationships by type



TIP

Indexes also exist for **relationships**: `CREATE INDEX FOR ()-[r:ACTED_IN]-() ON (r.roles).`



NOTE

Inspect with `SHOW INDEXES`. New indexes populate in the background; wait for state `ONLINE` before benchmarking.

Constraints: your schema, enforced

Constraints guarantee data quality and back MERGE with an index.

```
1 CREATE CONSTRAINT person_name_unique IF NOT EXISTS
   FOR (p:Person) REQUIRE p.name IS UNIQUE;

2 CREATE CONSTRAINT movie_key IF NOT EXISTS
   FOR (m:Movie) REQUIRE (m.title, m.released) IS NODE KEY;

3 CREATE CONSTRAINT movie_rating_type IF NOT EXISTS
   FOR (m:Movie) REQUIRE m.rating IS :: FLOAT;

4 CREATE CONSTRAINT acted_roles IF NOT EXISTS
   FOR ()-[r:ACTED_IN]-() REQUIRE r.roles IS NOT NULL;

SHOW CONSTRAINTS;
```

CYPHER

1 **Uniqueness** — also creates a range index. Real systems key on a stable id, not a name.

2 **Node key**: existence + uniqueness of a property combination.

3 **Property type** constraint (Neo4j 5.9+).

4 **Existence** constraint on a relationship property.

NOTE

In Neo4j, uniqueness constraints are available in all editions; existence, node-key and type constraints require Enterprise Edition or Aura. Other engines differ — check yours.

EXPLAIN and PROFILE: reading what the engine did

EXPLAIN shows the plan without running; PROFILE runs it and reports rows and database hits per operator.

```
1 PROFILE
2 MATCH (p:Person {name: 'Keanu Reeves'})
  -[:ACTED_IN]->(m:Movie)
RETURN m.title
```

CYPHER

1 Prefix any query. Use **EXPLAIN** on writes or slow queries you do not want to execute.

2 With an index on **:Person(name)** the plan starts with a **NodeIndexSeek**; without it, a **NodeByLabelScan**.

PLAN (READ BOTTOM-UP · FIGURES ILLUSTRATIVE)

Operator	Rows	Purpose
ProduceResults	3	Streams rows to the client
Projection	3	Computes m.title
Filter	3	Checks m:Movie label
Expand(All)	3	Follows ACTED_IN from each p
NodeIndexSeek	1	Finds Keanu via index on :Person(name)

RULE OF THUMB

Look for: label scans where you expected seeks, *CartesianProduct*, *Eager*, and any operator whose row count is far larger than the final result.

Plan operators worth recognising

Operator	Meaning	Good or bad?
NodeIndexSeek	Exact lookup via an index	Good — ideal anchor
NodeIndexScan	Scan all entries of an index	OK for range/existence
NodeByLabelScan	Every node with the label	Suspicious on big labels
AllNodesScan	Every node in the database	Bad — missing label
Expand(All)	Follow relationships from a node	Normal traversal
Expand(Into)	Check a relationship between two bound nodes	Efficient
CartesianProduct	Cross product of two inputs	Usually a bug
EagerAggregation	Aggregation needing all input	Expected with count/collect
Eager	Materialise all rows before continuing	Memory risk in writes
VarLengthExpand(All)	Variable-length traversal	Watch row counts

 INSIGHT

DB hits measure storage work. Optimise for fewer DB hits, not just wall-clock time on a warm cache.

 TIP

Newer runtimes (pipelined, parallel) may fuse operators; the logical meaning stays the same.

Performance tuning checklist for Cypher



Anchor with an index

Start patterns at a node found by an indexed property. Put the most selective node first in your mind; the planner will too.



Be specific

Always give labels, relationship types and directions when you know them. `(a) -- (b)` scans every relationship of `a`.



Filter early, collect late

Reduce rows with WHERE and `WITH ... LIMIT` before expensive expansions; aggregate with `collect()` only at the end.



Bound your paths

Use upper limits, QPP predicates and `SHORTEST` selectors; never unbounded `*` on dense data.



Parameters everywhere

Stable query text = plan cache hits. Literal values in the text force re-planning.



Mind supernodes

Nodes with millions of relationships (e.g. a `:Country`) make expansions explode. Expand `*into*` them, not out of them.

Large writes: CALL ... IN TRANSACTIONS

Split a huge update into many small transactions to bound memory and lock contention.

```
1 :auto
  MATCH (m:Movie)
  CALL (m) {
2   SET m.ratingBand = toInteger(m.rating)
3 } IN TRANSACTIONS OF 10000 ROWS
4 ON ERROR CONTINUE
```

CYPHER

INSIGHT

Why it matters: a single transaction touching 50M nodes must hold everything in memory until commit. The **Eager** operator can also force full materialisation of reads before writes.

PRO MOVE

`IN CONCURRENT TRANSACTIONS` (Neo4j 5.21+) parallelises batches. For older setups, `apoc.periodic.iterate` offers the same idea.

- 1 Browser prefix: run in an implicit (auto-commit) transaction, required for batching.
- 2 The body runs for each incoming row.
- 3 Commit every 10,000 rows.
- 4 Error behaviour per batch: `ON ERROR FAIL` (default), `CONTINUE`, `BREAK` or `RETRY`.

Importing data with LOAD CSV

Stream a CSV row by row and upsert it into the graph.

```
1 LOAD CSV WITH HEADERS FROM 'file:///movies.csv' AS row
2 WITH row WHERE row.title IS NOT NULL
  MERGE (m:Movie {title: row.title})
3 SET m.released = toInteger(row.released),
  m.rating = toFloat(row.rating)
  WITH m, row
4 UNWIND split(row.genres, '|') AS g
  MERGE (gen:Genre {name: trim(g)})
  MERGE (m) -[:IN_GENRE]->(gen)
```

CYPHER

1 `file:///` resolves to Neo4j's import directory; `http(s)` URLs also work.

2 Skip malformed rows early.

3 CSV values are **always strings**: convert explicitly.

4 Split a multi-valued column into rows and create genre nodes.



PRO MOVE

For big files wrap the body in `CALL (row) { ... } IN TRANSACTIONS`. For an initial load of billions of records use the offline `neo4j-admin database import` tool.



TIP

Create constraints **before** importing so every `MERGE` is an index lookup.

APOC: the standard library of procedures

Awesome Procedures On Cypher — hundreds of utilities. Call them with CALL ... YIELD.

Procedure / function	Purpose
<code>apoc.periodic.iterate</code>	Batch a write over a read query (pre-5.x classic)
<code>apoc.load.json</code> / <code>apoc.load.jdbc</code>	Import from JSON APIs or SQL databases
<code>apoc.path.expandConfig</code>	Configurable traversals with label/type filters
<code>apoc.refactor.mergeNodes</code>	Merge duplicate nodes, keeping relationships
<code>apoc.create.relationship</code>	Create relationships with a dynamic type
<code>apoc.meta.schema</code>	Infer the graph schema from data
<code>apoc.text.*</code>	String similarity, cleaning, formatting
<code>apoc.export.*</code>	Export to CSV, JSON, GraphML, Cypher scripts

NOTE

APOC Core ships with Neo4j; APOC Extended is a separate plugin. Many classic APOC use cases now have native Cypher equivalents (CALL IN TRANSACTIONS, dynamic labels, COLLECT subqueries).

RULE OF THUMB

Prefer native Cypher when it exists — it is optimised by the planner and portable to GQL-compatible engines.

Graph Data Science: algorithms from Cypher

Project a graph into memory, run algorithms, stream or write results back.

```
1 CALL gds.graph.project (
  'people',
  'Person',
2 {KNOWS: {orientation: 'UNDIRECTED'}}
);

3 CALL gds.pageRank.stream('people')
  YIELD nodeId, score
  RETURN gds.util.asNode(nodeId).name AS name,
         round(score, 3) AS pageRank
  ORDER BY pageRank DESC;

4 CALL gds.louvain.write('people', {writeProperty: 'community'});
```

CYPHER

- 1 An in-memory **projection**: which labels and relationship types the algorithm sees.
- 2 Treat KNOWS as undirected for this analysis.
- 3 Execution modes: `stream`, `stats`, `mutate` (in projection), `write` (to the database).
- 4 Community detection, persisted as a node property for later queries.



PRO MOVE

For ML: `gds.fastRP`, `gds.node2vec` and `gds.graphSage` produce **node embeddings**;
`gds.beta.pipeline.*` trains link-prediction and node-classification models inside GDS.

Vector search + traversal: the GraphRAG pattern

Retrieve semantically similar nodes, then expand through the graph to collect grounded context for an LLM.

```
1 CREATE VECTOR INDEX movie_plot IF NOT EXISTS
  FOR (m:Movie) ON m.plotEmbedding
  OPTIONS { indexConfig: {
2     `vector.dimensions`: 1536,
     `vector.similarity_function`: 'cosine'
  } };

3 CALL db.index.vector.queryNodes('movie_plot', 5, $questionEmbedding)
  YIELD node AS m, score
4 MATCH (m) <-[:ACTED_IN]-(a:Person)
  OPTIONAL MATCH (m) <-[:DIRECTED]-(d:Person)
  RETURN m.title, score,
         collect(DISTINCT a.name) AS cast,
         collect(DISTINCT d.name) AS directors
  ORDER BY score DESC
```

CYPHER

- 1 Approximate-nearest-neighbour index on an embedding property.
- 2 Dimensions must match your embedding model.
- 3 Top-5 most similar plots to the question embedding (passed as a parameter).
- 4 **Graph expansion:** facts the vector alone cannot provide — cast, directors, sequels.

INSIGHT

Vectors find **what is relevant**; the graph supplies **how things relate**. Feeding both to the LLM reduces hallucination and enables multi-hop answers.

Temporal and spatial values

Native types for dates, times, durations and points — with arithmetic and indexing support.

```
1 RETURN date('1999-03-31') AS release,  
2 date('1999-03-31').year AS y,  
3 duration.between(date('1999-03-31'),  
4 date('2003-05-15')).years AS yearsToSequel,  
5 date('1999-03-31') + duration('P1M') AS monthLater,  
datetime({timezone: 'Europe/London'}) AS nowLondon,  
point.distance(  
    point({latitude: -33.8688, longitude: 151.2093}),  
    point({latitude: 51.5074, longitude: -0.1278})  
)/ 1000 AS kmSydneyLondon
```

CYPHER



TIP

Store instants as `datetime` with a time zone; store calendar facts (release dates, birthdays) as `date`. Mixing them causes subtle comparison bugs.

- 1 Types: `date`, `localtime`, `time`, `localdatetime`, `datetime`, `duration`.
- 2 Component access: `.year`, `.month`, `.dayOfWeek`, `.week...`
- 3 Reloaded opened 15 May 2003 → 4 whole years later.
- 4 Calendar arithmetic: end-of-month is clamped → 1999-04-30.
- 5 Geodesic distance in metres for WGS-84 points — roughly 17,000 km here.

Cypher from Python: the official neo4j driver

execute_query for simple calls; managed transactions for writes with automatic retries.

```
from neo4j import GraphDatabase

URI, AUTH = "neo4j://localhost:7687", ("neo4j", "password")

query = """
MATCH (p:Person)-[:ACTED_IN]->(m:Movie)
1 WHERE m.rating >= $min
RETURN p.name AS actor, collect(m.title) AS titles
"""

2 def add_knows(tx, a, b):
    tx.run("MATCH (x:Person {name: $a}), (y:Person {name: $b}) "
          "MERGE (x)-[:KNOWS]->(y)", a=a, b=b)

3 with GraphDatabase.driver(URI, auth=AUTH) as driver:
    records, summary, keys = driver.execute_query(
        query, min=7.3, database_="neo4j")
    for r in records:
        print(r["actor"], r["titles"])

4 with driver.session(database="neo4j") as s:
    s.execute_write(add_knows, "Hugo Weaving", "Keanu Reeves")
```

PYTHON

1 `$min` parameter, supplied as a keyword argument below.

2 A **transaction function**: the driver may call it again on transient errors — keep it idempotent (MERGE).

3 `execute_query` handles sessions, retries and routing; `database_` (trailing underscore) is a driver option.

4 `execute_write` routes to the cluster leader and retries.



TIP

Result to pandas: `driver.execute_query(q, result_transformer=neo4j.Result.to_df)`.

Transactions, consistency and clustering



ACID

Every Cypher statement runs in a transaction. Neo4j transactions are fully ACID; isolation is **read-committed** with write locks.



Deadlocks & retries

Concurrent writers can deadlock; the database aborts one with a transient error. Driver transaction functions retry automatically.



Implicit vs. explicit

Auto-commit (implicit) for simple statements and batching; explicit transactions to group several statements atomically.



Clusters

Writes go to the leader of each database; reads can be served by secondaries. The driver routes via `neo4j://` URIs.



Causal consistency

Bookmarks ensure a later read sees your earlier write, even on another server. `execute_query` manages them for you.



Lost updates

Read-then-write in application code can race. Do the read-modify-write in **one** Cypher statement (`SET n.c = n.c + 1`) or lock explicitly.

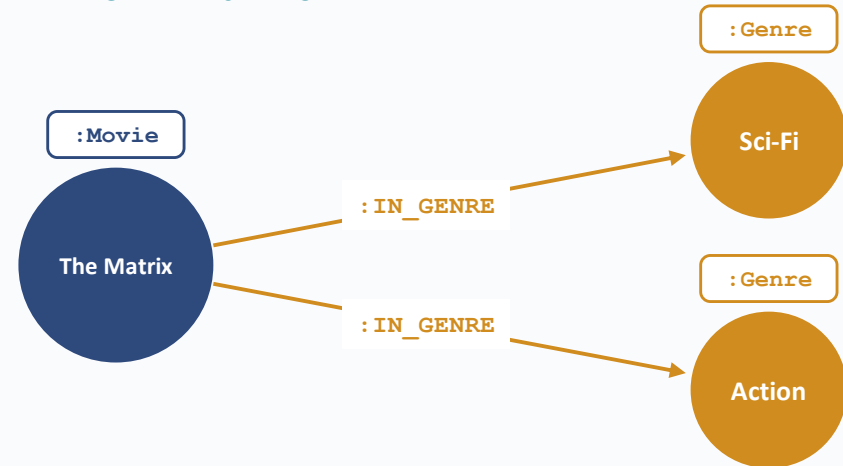
Data modelling: property, node or relationship?

The same fact — "The Matrix is a Sci-Fi film" — modelled two ways. The right choice depends on the questions you ask.

OPTION A — GENRE AS A PROPERTY



OPTION B — GENRE AS A NODE



CHOOSE A WHEN

- You only display or filter the value
- Values are simple and rarely shared
- You never traverse **through** the value



CHOOSE B WHEN

- You ask "which other films share a genre?" (traversal)
- The value has its own properties (description, parent genre)
- You need to count or aggregate by it efficiently

Property-graph modelling anti-patterns



Supernodes

A node like `(:Gender {value:'F'})` connected to millions of people. Traversals through it explode. Keep low-cardinality values as properties or labels.



Generic relationships

`[ :RELATED_TO {type:'acted'} ]` hides semantics from the planner. Use specific types: `ACTED_IN`, `DIRECTED`.



Internal ids as keys

`elementId()` values may be reused after deletes. Store your own stable identifiers.



Foreign keys in lists

A `movieIds: [12, 57]` property is a relational habit. If you traverse it, make it relationships.



Time only as properties

If questions follow time ("events in the same week"), consider time-tree nodes or event nodes instead of timestamps alone.



Label explosion

Hundreds of labels such as `:Movie2019` encode data in schema. Use properties for data; labels for classification.

Cypher and ISO GQL: what is shared, what differs

Feature	Cypher (Neo4j 5)	ISO GQL (39075:2024)
Node / edge patterns	<code>(a:Person) -[:KNOWS]->(b)</code>	<code>(a:Person) -[:KNOWS]->(b)</code> – same
Label expressions	<code>:Person Movie</code> <code>:!Movie</code> <code>:%</code>	<code>:Person Movie</code> <code>:!Movie</code> <code>:%</code>
Quantified paths	<code>((a)-[:R]->(b)){1,3}</code>	same syntax
Path selectors	<code>SHORTEST 1</code> · <code>ALL SHORTEST</code> · <code>ANY</code>	same keywords
Path modes	default "different relationships" (trail)	<code>WALK</code> · <code>TRAIL</code> · <code>ACYCLIC</code> · <code>SIMPLE</code>
Create data	<code>CREATE (n:Person {...})</code>	<code>INSERT (n:Person {...})</code>
Unwind a list	<code>UNWIND list AS x</code>	<code>FOR x IN list</code>
Pipeline	<code>WITH ...</code>	<code>NEXT (linear composition)</code> , <code>LET</code> , <code>FILTER</code>
Schema	indexes + constraints	graph types (closed schemas)



Implementations are converging: Neo4j adds GQL keywords and conformance, and new engines target GQL directly. SQL:2023 Part 16 (SQL/PGQ) brings the same pattern syntax into relational databases via [GRAPH_TABLE](#).

Checkpoint V — advanced Cypher

Q1

Best-rated film **per director**, one row each.

```
MATCH (d:Person) WHERE EXISTS { (d)-[:DIRECTED]->() }
CALL (d) {
  MATCH (d)-[:DIRECTED]->(m:Movie)
  RETURN m.title AS best ORDER BY m.rating DESC LIMIT 1 }
RETURN d.name, best
```

Q2

Why is `MATCH (a:Person), (b:Movie)`
`RETURN a, b` dangerous?

✓ ANSWER

The two patterns are disconnected → a **CartesianProduct**: every person paired with every movie ($7 \times 4 = 28$ rows here, billions in production).

Q3

KNOWS chains from Keanu where **every** person was born before 1966.

```
MATCH (a:Person {name:'Keanu Reeves'})
      ((x)-[:KNOWS]->(y) WHERE y.born < 1966)+ (b)
RETURN b.name
// → no rows: Carrie-Anne (1967) and Chad (1968)
// both fail the very first hop
```

Q4

Which index type serves `WHERE m.title`
`CONTAINS 'Atlas'`?

✓ ANSWER

A **TEXT** index. Range indexes serve equality, ranges and `STARTS WITH`; full-text indexes are queried through a procedure, not `CONTAINS`.

06

PART VI

SPARQL and Cypher, Side by Side

Translate fluently between the two languages and understand the semantic differences that make naïve translations wrong.

Concept map

Matching

Optional

Filtering

Aggregation

Paths

Negation

Updates

Subqueries

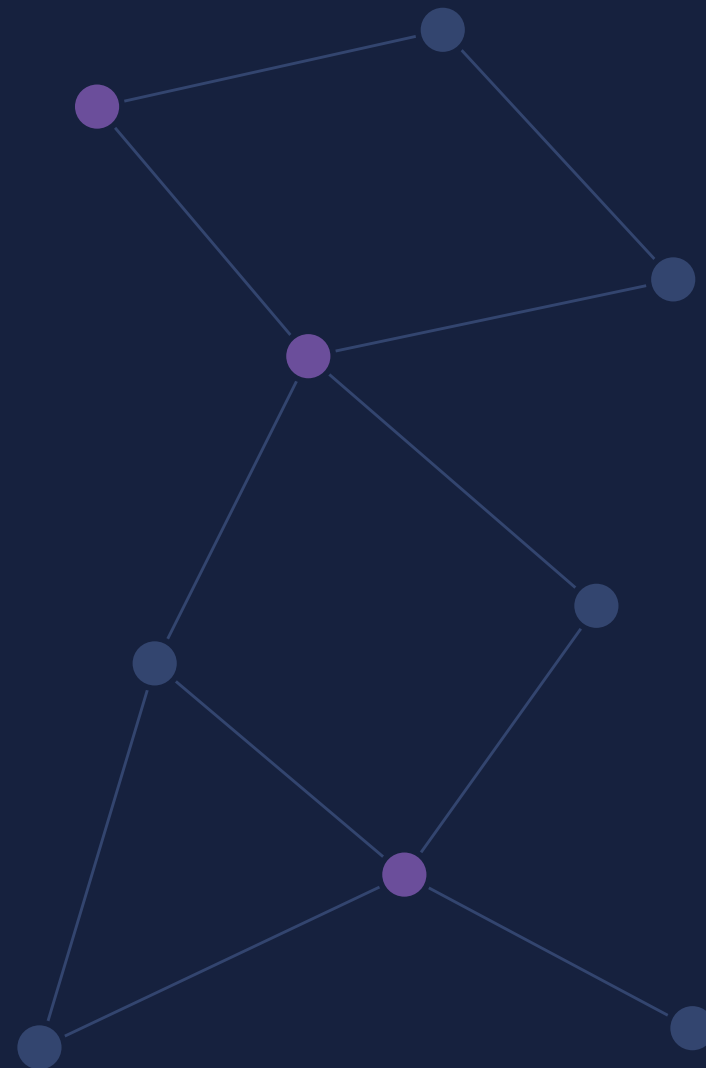
Semantics

Edge properties

Interop

Choosing

Graph ML



The concept map: SPARQL ↔ Cypher

Concept	SPARQL	Cypher
Pattern unit	<code>?s ex:p ?o .</code>	<code>(s)-[:P]->(o)</code>
Variable	<code>?x</code>	<code>x</code>
Node type	<code>?x a ex:Person</code>	<code>(x:Person)</code>
Identity	IRI <code><http://example.org/keanu></code>	<code>elementId(x) / key {name: '...'}</code>
Edge property	<code><< ?s ex:p ?o >> ex:k ?v (RDF 1.2)</code>	<code>-[r:P {k: v}]-> r.k</code>
Filter	<code>FILTER (...)</code>	<code>WHERE ...</code>
Optional part	<code>OPTIONAL { ... }</code>	<code>OPTIONAL MATCH ...</code>
Alternatives	<code>{ ... } UNION { ... }</code>	<code>query UNION query · :A B</code>
Negation	<code>FILTER NOT EXISTS { ... } · MINUS</code>	<code>WHERE NOT EXISTS { ... }</code>
Projection	<code>SELECT ?a (expr AS ?b)</code>	<code>RETURN a, expr AS b</code>
Grouping	<code>explicit GROUP BY + HAVING</code>	<code>implicit keys + WITH ... WHERE</code>
Chaining	<code>nested sub-SELECTs</code>	<code>WITH pipeline · CALL { }</code>
Paths	<code>ex:p+ ex:p* ^ex:p ex:a/ex:b</code>	<code>-[:P*1..]-> QPP {1,3} SHORTEST</code>
Missing value	<code>unbound (error in FILTER)</code>	<code>null (three-valued logic)</code>
Write	<code>INSERT DATA · DELETE/INSERT WHERE</code>	<code>CREATE · MERGE · SET · DELETE</code>
Remote data	<code>SERVICE <endpoint> { }</code>	<code>composite databases / USE</code>

Head to head ①: basic matching

Keanu's films. Shared variables vs. drawn relationships.

SPARQL

```
SELECT ?title WHERE {  
  ?p ex:name "Keanu Reeves" ;  
    ex:actedIn ?m .  
  ?m ex:title ?title .  
}
```

SPARQL

Cypher

```
MATCH (:Person {name: 'Keanu Reeves'})  
      -[:ACTED_IN]->(m:Movie)  
RETURN m.title AS title
```

CYPHER

RESULT (BOTH LANGUAGES)

title

The Matrix

The Matrix Reloaded

John Wick

INSIGHT

SPARQL needs three triple patterns because name and title are separate triples. In Cypher they are properties inside the nodes, so one pattern suffices.

TIP

Result in both: The Matrix · The Matrix Reloaded · John Wick. Neither guarantees order without ORDER BY.

Head to head ②: optional data

Every film with its prequel, if any.

SPARQL

```
SELECT ?title ?preTitle WHERE {
  ?m a ex:Movie ; ex:title ?title .
  OPTIONAL {
    ?m ex:sequelOf ?pre .
    ?pre ex:title ?preTitle .
  }
}
```

SPARQL

Cypher

```
MATCH (m:Movie)
OPTIONAL MATCH (m)-[:SEQUEL_OF]->(pre)
RETURN m.title AS title,
       pre.title AS preTitle
```

CYPHER

RESULT (— = UNBOUND IN SPARQL, null IN CYPHER)

title	preTitle
The Matrix	—
The Matrix Reloaded	The Matrix
John Wick	—
Cloud Atlas	—

INSIGHT

Missing values: SPARQL leaves `?preTitle` **unbound** (absent from the JSON binding); Cypher returns **null**. Client code must handle each differently.

PITFALL

Filter placement matters in both languages: inside `OPTIONAL { }` / directly after `OPTIONAL MATCH` it only restricts the optional part.

Head to head ③: filtering and strings

Films released 2000 or later whose title contains "Matrix".

SPARQL

```
SELECT ?title WHERE {  
  ?m ex:title ?title ;  
    ex:released ?y .  
  FILTER (?y >= 2000 &&  
    CONTAINS(?title, "Matrix"))  
}
```

SPARQL

Cypher

```
MATCH (m:Movie)  
WHERE m.released >= 2000  
  AND m.title CONTAINS 'Matrix'  
RETURN m.title AS title
```

CYPHER

RESULT (BOTH LANGUAGES)

title
The Matrix Reloaded



TIP

Result: The Matrix Reloaded. Note the string delimiters: SPARQL prefers double quotes (single also allowed); Cypher uses either.



INSIGHT

SPARQL uses functions (`CONTAINS(a, b)`, `STRSTARTS`, `REGEX`); Cypher uses infix operators (`CONTAINS`, `STARTS WITH`, `=~`).

Head to head ④: aggregation

Actors with at least three films.

SPARQL

```
SELECT ?name (COUNT(?m) AS ?films)
WHERE {
  ?p ex:actedIn ?m ;
      ex:name ?name .
}
GROUP BY ?name
HAVING (COUNT(?m) >= 3)
```

SPARQL

Cypher

```
MATCH (p:Person)-[:ACTED_IN]->(m)
WITH p.name AS name,
      count(m) AS films
WHERE films >= 3
RETURN name, films
```

CYPHER

RESULT (BOTH LANGUAGES)

name	films
Hugo Weaving	3
Keanu Reeves	3

INSIGHT

SPARQL groups **explicitly**; Cypher infers grouping keys from the non-aggregated columns. HAVING becomes `WITH ... WHERE`.

TIP

Both return Hugo Weaving 3 and Keanu Reeves 3. `GROUP_CONCAT(string) ↔ collect()` (list).

Head to head ⑤: paths

Everyone reachable from Keanu via one or more KNOWS hops.

SPARQL

```
SELECT DISTINCT ?name WHERE {  
  ex:keanu ex:knows+ ?p .  
  ?p ex:name ?name .  
}
```

SPARQL

Cypher

```
MATCH (:Person {name:'Keanu Reeves'})  
      -[:KNOWS]->+(p:Person)  
RETURN DISTINCT p.name AS name
```

CYPHER

RESULT (BOTH LANGUAGES)

name

Carrie-Anne Moss

Chad Stahelski

Laurence Fishburne

Hugo Weaving

INSIGHT

SPARQL+ returns **reachable nodes** (set semantics, no path). Cypher enumerates **paths** (one row per trail) — hence DISTINCT — but can return the path, its length and its nodes.

TIP

Result: Carrie-Anne, Chad, Laurence, Hugo. `-[:KNOWS]->+` is the QPP shorthand; the classic form is `-[:KNOWS*1..]->`.

Head to head ⑥: negation

People who never directed a film.

SPARQL

```
SELECT ?name WHERE {  
  ?p a ex:Person ; ex:name ?name .  
  FILTER NOT EXISTS {  
    ?p ex:directed ?m }  
}
```

SPARQL

Cypher

```
MATCH (p:Person)  
WHERE NOT EXISTS {  
  (p)-[:DIRECTED]->(:Movie)  
}  
RETURN p.name AS name
```

CYPHER

RESULT (BOTH LANGUAGES)

name
Keanu Reeves
Carrie-Anne Moss
Laurence Fishburne
Hugo Weaving

INSIGHT

The constructs are almost identical: an existential subquery negated. Both return Keanu, Carrie-Anne, Laurence and Hugo.

PITFALL

Open vs. closed world: in RDF, "no triple" formally means **unknown**. SPARQL still answers over the data you have (closed-world query semantics), but ontology reasoning would not conclude "not a director".

Head to head ⑦: updates

Add John Wick: Chapter 2 and link Keanu to it — idempotently.

SPARQL

```
INSERT DATA {  
  ex:johnwick2 a ex:Movie ;  
  ex:title "John Wick: Chapter 2" ;  
  ex:released 2017 .  
  ex:keanu ex:actedIn ex:johnwick2 .  
}
```

SPARQL

Cypher

```
MERGE (m:Movie {title: 'John Wick: Chapter 2'})  
  ON CREATE SET m.released = 2017  
WITH m  
MATCH (k:Person {name: 'Keanu Reeves'})  
MERGE (k) -[:ACTED_IN]->(m)
```

CYPHER

INSIGHT

RDF insertion is **naturally idempotent**: a graph is a set, and the IRI is the identity. In Cypher, idempotency comes from MERGE + a uniqueness constraint.

TIP

Blank nodes break RDF idempotency (each run creates a fresh node) — just as CREATE does in Cypher.

Head to head ⑧: subqueries and pipelines

The best-rated film of each director.

SPARQL

```
SELECT ?dir ?title WHERE {  
  { SELECT ?d (MAX(?r) AS ?best)  
    WHERE { ?d ex:directed/ex:rating ?r }  
    GROUP BY ?d }  
  ?d ex:name ?dir ; ex:directed ?m .  
  ?m ex:rating ?best ; ex:title ?title .  
}
```

SPARQL

Cypher

```
MATCH (d:Person)-[:DIRECTED]->(m:Movie)  
WITH d, m ORDER BY m.rating DESC  
WITH d, collect(m)[0] AS best  
RETURN d.name AS dir, best.title AS title
```

CYPHER

RESULT

dir	title
Chad Stahelski	John Wick
Lana Wachowski	The Matrix
Lilly Wachowski	The Matrix

INSIGHT

SPARQL nests a grouped subquery and joins back (ties → several rows). Cypher sorts, then collects per group and takes the head (ties → one arbitrary row).

TIP

Result: Chad → John Wick; Lana → The Matrix; Lilly → The Matrix. Note the property path `ex:directed/ex:rating` shortening the inner pattern.

Semantic differences that break naïve translations



Homomorphism vs. trails

SPARQL lets two variables bind the same node **and** reuse the same triple. Cypher never reuses a relationship within one MATCH. Co-star queries differ by a FILTER.



Bag semantics — both

Both return duplicates unless told otherwise (**DISTINCT**). But SPARQL ***/+** paths are set-valued while Cypher var-length patterns are path-valued.



Open vs. closed world

RDF/OWL reasoning treats missing facts as unknown and has **no unique-name assumption** (**owl:sameAs**). Property graphs behave like databases: absent = false, different node = different thing.



Unbound vs. null

SPARQL: unbound variables make FILTER expressions **errors**. Cypher: null propagates through three-valued logic. Both drop the row in a filter, but **COALESCE**, **bound()** and **IS NULL** differ.



Typing

RDF literals carry datatypes per value ("**7**"^{^^xsd:string} vs **7**). Cypher values are typed too, but a property can hold different types on different nodes unless constrained.



Identity

IRIs are global and meaningful across datasets; Cypher ids are local. Merging two RDF graphs is a union; merging two property graphs needs key matching.

Three ways to say "Keanu played Neo in The Matrix"

Relationship properties are the sharpest modelling difference between the two worlds.

LPG — NATIVE



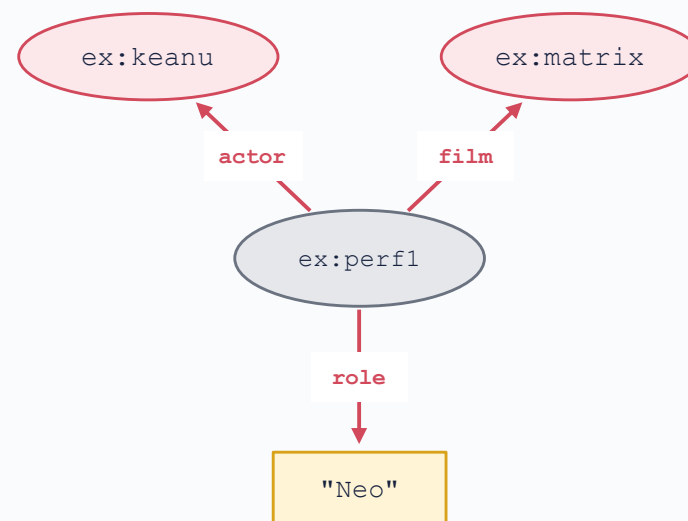
Query: `-[r:ACTED_IN]->then r.roles`



TIP

Simplest to write and query. But the relationship cannot link to *other* things (e.g. a costume designer).

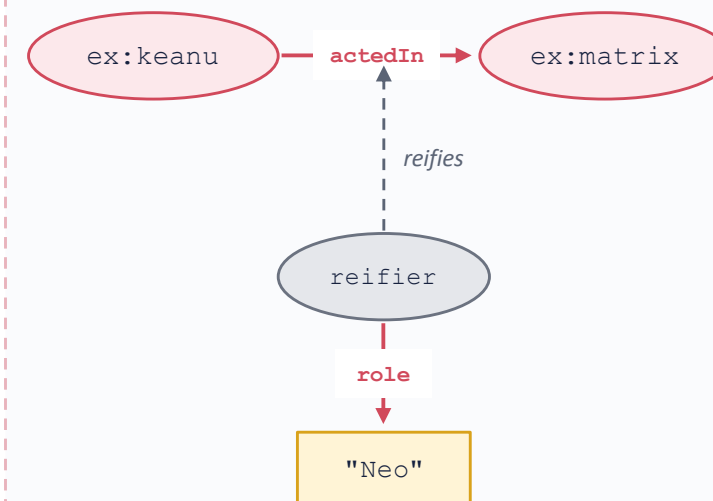
RDF — N-ARY NODE



RULE OF THUMB

Works in every RDF store and scales to rich events. One extra hop in every query.

RDF 1.2 — REIFIED TRIPLE



INSIGHT

Keeps the simple triple **and** annotates it. Needs RDF 1.2 / RDF-star support.

Interoperability: RDF inside Neo4j with neosemantics (n10s)

The *n10s* plugin imports RDF into a property graph and can serve the graph back as RDF.

```
1 CREATE CONSTRAINT n10s_unique_uri IF NOT EXISTS
  FOR (r:Resource) REQUIRE r.uri IS UNIQUE;

2 CALL n10s.graphconfig.init({handleVocabUris: 'SHORTEN'});
  CALL n10s.nsprefixes.add('ex', 'http://example.org/');

3 CALL n10s.rdf.import.fetch(
  'https://example.org/movies.ttl', 'Turtle');

4 MATCH (m:ex__Movie)
  RETURN m.ex__title AS title LIMIT 5;
```

CYPHER

1 Every imported resource becomes a `:Resource` node keyed by its IRI.

2 `SHORTEN` turns IRIs into `prefix__localName` labels, types and property keys.

3 Literal triples → properties; IRI objects → relationships;
`rdf:type` → labels.

4 `ex:Movie` became the label `ex__Movie`; `ex:title` the property `ex__title`.

INSIGHT

The mapping is lossy in places: language tags, datatypes, blank nodes and multi-valued literals need configuration choices. Round-trip tests are essential.

NOTE

Other bridges: Amazon Neptune (both models, one service), RDF4J/Jena LPG adapters, and R2RML/RML for mapping relational data to RDF.

Choosing: a decision framework

If your priority is...	Lean towards SPARQL / RDF	Lean towards Cypher / LPG
Integrating data from many organisations	✓ global IRIs, shared vocabularies	needs a key-reconciliation layer
Publishing open / linked data	✓ W3C standards, Wikidata, DBpedia	not designed for it
Formal semantics, ontologies, inference	✓ RDFS/OWL reasoning, SHACL	constraints only; logic in code
Deep traversal and path analytics	paths without path objects	✓ paths, QPP, shortest-path selectors
Rich relationship attributes	RDF 1.2 or n-ary nodes	✓ native properties
Developer ergonomics and app backends	steeper learning curve	✓ ASCII art, drivers, tooling
Graph algorithms / graph ML	export or store extensions	✓ GDS library, embeddings
GraphRAG for LLM applications	✓ ontology-grounded context	✓ vector + traversal in one query

RULE OF THUMB

Common production architecture: **RDF for the integration/semantic layer** (the enterprise knowledge graph) and **property graphs for application-specific analytic views** derived from it.

Knowledge graphs for machine learning

Both query languages are feature-engineering tools. Here is where each fits in an ML workflow.



Tabular features

Degree, co-occurrence counts, path counts, neighbour aggregates — computed with GROUP BY / COUNT { } and exported to pandas.



Node embeddings

FastRP, node2vec, GraphSAGE in Neo4j GDS; knowledge-graph embeddings (TransE, RotatE, ComplEx) on RDF triples with PyKEEN or DGL-KE.



GNN training data

Extract edge lists and node features with SPARQL CONSTRUCT or Cypher, then load into PyTorch Geometric or DGL.



Link prediction

Predict missing actedIn / KNOWS edges; write scores back as relationships with a probability property for review.



Entity resolution

Blocking with graph queries + similarity models; record decisions as owl:sameAs (RDF) or merged nodes (LPG).

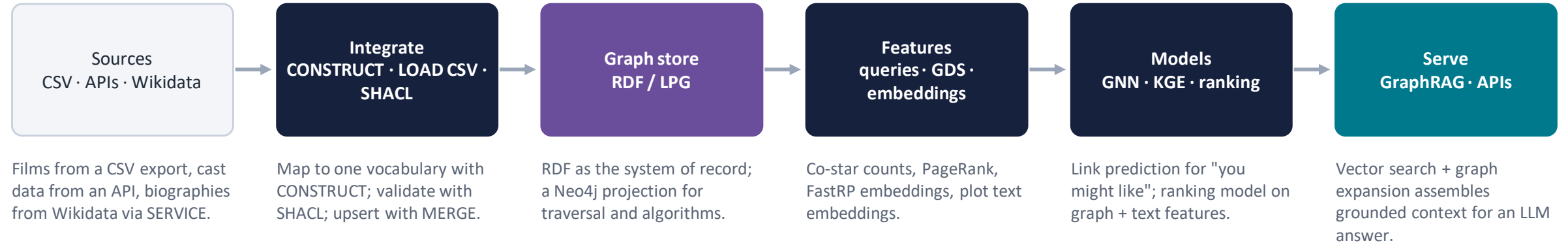


Data quality gates

SHACL shapes and Cypher constraint checks as CI tests before any training run — cheap insurance against silent label noise.

Case study: an end-to-end knowledge-graph ML pipeline

A movie-recommendation assistant built on both technologies. Each stage lists the query technique it relies on.



PRO MOVE

Version every stage: named graphs per load (RDF) and projection names with dates (GDS). Reproducible features are worth more than clever ones.

PITFALL

Leakage: if the test edges are still in the graph when you compute embeddings or path features, your offline metrics will be inflated. Remove held-out edges **before** feature computation.

07

PART VII

Mastery: Capstones & Reference

Two end-to-end problems with worked solutions, then the reference material to keep beside you.

[SPARQL capstone](#)

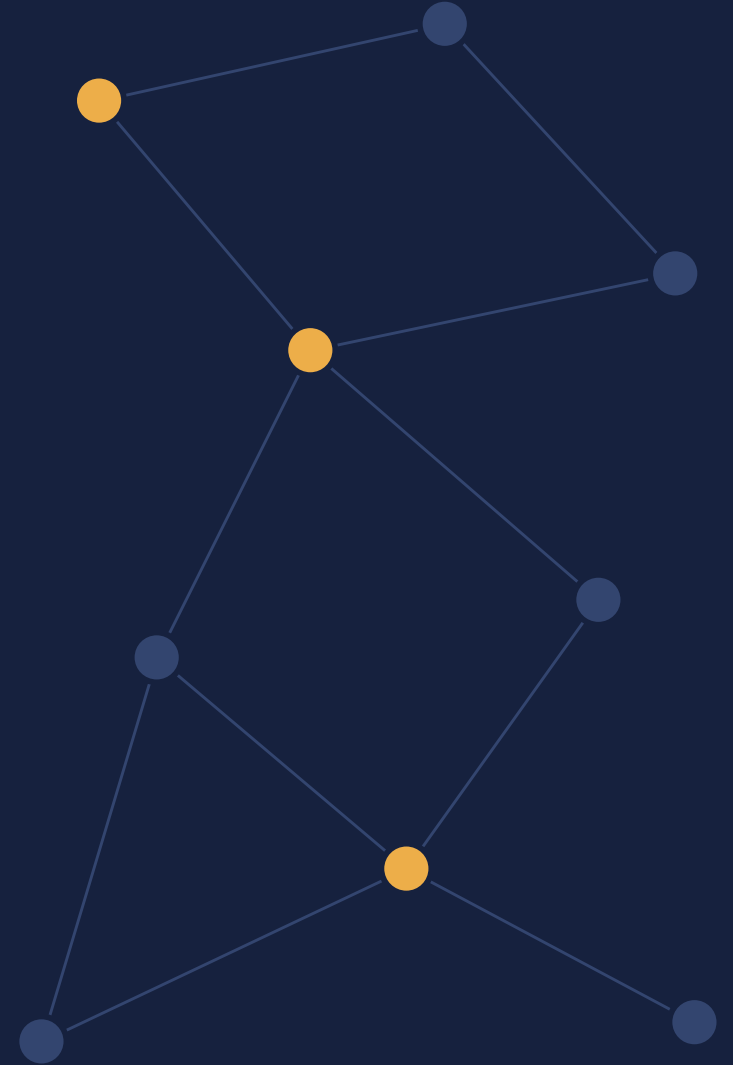
[Cypher capstone](#)

[Cheat sheets](#)

[Glossary](#)

[Resources](#)

[Next steps](#)



Capstone 1 (SPARQL): a film analytics report

Try it before turning the page. Everything you need was covered in Parts II and III.

Write one SELECT query over the running dataset that returns, for each film:

- its **title**
- its **directors**, as one comma-separated string
- the **cast size** (number of distinct actors)
- the **average age** of the cast in the release year

Keep only films with at least two cast members, ordered by average age (youngest cast first).

Stretch goal: add the title of the prequel, if there is one, without changing any other column.



HINTS

- Age = `?released - ?born`, computed with `BIND`
- Directors are optional per film → `OPTIONAL`
- Joining actors × directors duplicates rows: which aggregates need `DISTINCT?`



EXPECTED SHAPE

2 rows: The Matrix and The Matrix Reloaded — both directed by Lana & Lilly Wachowski, cast of 4.



PITFALL

Films with a single actor (John Wick, Cloud Atlas) must disappear — filter **groups**, not rows.

Capstone 1 — worked solution

```
SELECT ?title
1 (GROUP_CONCAT(DISTINCT ?dName; SEPARATOR=", ") AS ?directors)
2 (COUNT(DISTINCT ?a) AS ?cast)
3 (AVG(?age) AS ?avgAge)
4 (SAMPLE(?preTitle) AS ?prequel)
WHERE {
  ?m a ex:Movie ; ex:title ?title ; ex:released ?y .
  ?a ex:actedIn ?m ; ex:born ?b .
5 BIND (?y - ?b AS ?age)
  OPTIONAL { ?d ex:directed ?m ; ex:name ?dName . }
  OPTIONAL { ?m ex:sequelOf/ex:title ?preTitle . }
}
GROUP BY ?title
6 HAVING (COUNT(DISTINCT ?a) >= 2)
ORDER BY ?avgAge
```

RESULT

?title	?directors	?cast	?avgAge	?prequel
The Matrix	Lana Wachowski, Lilly Wachowski	4	36	—
The Matrix Reloaded	Lana Wachowski, Lilly Wachowski	4	40	The Matrix

- 1 **DISTINCT** — each director appears once per actor row otherwise.
- 2 Count actors, not rows: 4 actors × 2 directors = 8 rows per film.
- 3 AVG survives the duplication here because **every** actor is duplicated equally (×2). Not true in general — be careful!
- 4 Stretch goal: **SAMPLE** because the prequel is constant within the group.
- 5 Matrix ages: 35, 32, 38, 39 → 36. Reloaded: 39, 36, 42, 43 → 40.
- 6 Group filter drops John Wick and Cloud Atlas.

Capstone 2 (Cypher): collaborative recommendations

Build the classic "people who worked with you also appeared in..." recommender.

Recommend films to Carrie-Anne Moss:

- find her co-actors (people who acted in a film with her)
- collect the **other** films those co-actors appeared in
- **exclude** films she already acted in
- **score** each candidate by the number of distinct co-actors leading to it
- return title, score, the co-actors as evidence, and the rating

Order by score, then rating, both descending.

Stretch goal: also boost films directed by someone who directed one of her films.



HINTS

- One long pattern: `me → film ← co-actor → candidate`
- Negative pattern predicate: `NOT EXISTS { ... }`
- Evidence as a list: `collect(DISTINCT ...)`



THINK ABOUT

Do you need `WHERE co <> me`? (Recall the path semantics of slide 112.)

Capstone 2 — worked solution

```
1 MATCH (me:Person {name: 'Carrie-Anne Moss'})
   -[:ACTED_IN]->(m:Movie)<-[:ACTED_IN]-(co:Person)
   -[:ACTED_IN]->(rec:Movie)
2 WHERE NOT EXISTS { (me)-[:ACTED_IN]->(rec) }
3 WITH rec, collect(DISTINCT co.name) AS via
RETURN rec.title AS recommendation,
       size(via) AS score,
       via,
       rec.rating AS rating
ORDER BY score DESC, rating DESC
```

CYPHER

1 `co` can never be `me`: that would reuse the same `ACTED_IN` relationship, which trail semantics forbids. No extra `WHERE` needed.

2 Existential subquery removes *The Matrix and Reloaded*.

3 `DISTINCT` — a co-actor reachable through both *Matrix* films counts once.

4 Score = number of distinct co-actors pointing at the film.

RESULT

recommendation	score	via	rating
John Wick	1	['Keanu Reeves']	7.4
Cloud Atlas	1	['Hugo Weaving']	7.4



PRO MOVE

Stretch: `OPTIONAL MATCH (me)-[:ACTED_IN]->()-[:DIRECTED]-(d)-[:DIRECTED]->(rec)` then add `count(DISTINCT d) * 0.5` to the score. *Cloud Atlas* would win (the Wachowskis directed it).

SPARQL cheat sheet

SKELETON

```
PREFIX ex: <http://example.org/>
SELECT DISTINCT ?a (expr AS ?b)
FROM <g> FROM NAMED <g2>
WHERE { ... }
GROUP BY ?a HAVING (...)
ORDER BY DESC(?b)
LIMIT 10 OFFSET 20
```

TRIPLE PATTERNS

```
?s ex:p ?o .
?s a ex:Class ;
  ex:p1 ?x , ?y .
?s ex:p "text"@en .
?s ex:p "1"^^xsd:integer .
```

OTHER FORMS

```
ASK { ... }
CONSTRUCT { tmpl } WHERE { ... }
CONSTRUCT WHERE { bgp }
DESCRIBE ?x WHERE { ... }
```

GRAPH PATTERN OPERATORS

```
OPTIONAL { ... }
{ ... } UNION { ... }
MINUS { ... }
FILTER (expr)
FILTER [NOT] EXISTS { ... }
BIND (expr AS ?v)
VALUES ?v { 1 2 }
GRAPH ?g { ... }
SERVICE <url> { ... }
{ SELECT ... } # subquery
```

PROPERTY PATHS

```
p1/p2   p1|p2   ^p
p*      p+      p?   !(p1|p2)
?x a/rdfs:subClassOf* ?c
```

FUNCTIONS

```
STR LANG DATATYPE BOUND
IRI BNODE STRDT STRLANG
CONTAINS STRSTARTS STRENDS
REGEX(?s, "^a", "i") REPLACE
CONCAT SUBSTR STRLEN LCASE
YEAR MONTH DAY NOW
IF COALESCE IN NOT IN
COUNT SUM AVG MIN MAX
SAMPLE GROUP_CONCAT
```

UPDATE

```
INSERT DATA { ... }
DELETE DATA { ... }
DELETE { ... } INSERT { ... }
  WHERE { ... }
DELETE WHERE { ... }
WITH <g> ...
LOAD / CLEAR / DROP / COPY
MOVE / ADD / CREATE
```

Cypher cheat sheet

READ

```
MATCH (a:L {k: v})-[:T]->(b)
OPTIONAL MATCH (b)<-[:T2]-(c)
WHERE a.x > 1 AND b:L2
WITH a, count(b) AS n
WHERE n > 2
RETURN DISTINCT a.k AS k, n
ORDER BY n DESC
SKIP 10 LIMIT 5
```

PATTERNS

```
(a)-->(b)      (a)--(b)
(a)-[:T|U]->(b)
(a)-[*1..3]->(b)
(a) ((x)-[:T]->(y)) {1,3} (b)
p = SHORTEST 1 (a)-[]-+(b)
(n:A|B)  (n:A&B)  (n:!A)
```

WRITE

```
CREATE (n:L {k: v})
MERGE (n:L {key: $k})
  ON CREATE SET n.c = 1
  ON MATCH SET n.c = n.c + 1
SET n += {a: 1}, n:Extra
REMOVE n.a, n:Extra
DETACH DELETE n
UNWIND $rows AS row ...
```

SUBQUERIES

```
EXISTS { (a)-[:T]->() }
COUNT { (a)-[:T]->() }
COLLECT { MATCH ... RETURN x }
CALL (a) { ... }
CALL (r) { ... }
  IN TRANSACTIONS OF 1000 ROWS
```

EXPRESSIONS

```
[x IN l WHERE x > 1 | x * 2]
reduce(s = 0, x IN l | s + x)
n { .name, k: expr }
CASE WHEN c THEN a ELSE b END
coalesce(a, b)  x IS NULL
s STARTS WITH 'a'  s =~ '(?i).*'
size() length() nodes() keys()
collect() count() avg() max()
```

SCHEMA & TOOLS

```
CREATE INDEX FOR (n:L) ON (n.k)
CREATE TEXT INDEX ...
CREATE CONSTRAINT FOR (n:L)
  REQUIRE n.k IS UNIQUE
SHOW INDEXES / CONSTRAINTS
EXPLAIN ...  PROFILE ...
:param x => 1
```

SPARQL / RDF term	Meaning
BGP	Basic graph pattern: a set of triple patterns matched together.
IRI	Global identifier for an RDF resource.
Literal	An RDF value: lexical form + datatype (+ language tag).
Blank node	Anonymous RDF node, local to a document.
Named graph	A set of triples identified by an IRI; with it, a triple becomes a quad.
Entailment	Triples implied by the data plus RDFS/OWL semantics.
SHACL	W3C language for validating RDF graphs against shapes.
Property path	Regular expression over predicates in SPARQL.
Unbound	A SPARQL variable with no value in a solution.

Cypher / LPG term	Meaning
Label	Tag classifying a property-graph node (:Person).
Relationship type	The single, mandatory type of an LPG relationship.
Trail	A path with no repeated relationships — Cypher's default.
QPP	Quantified path pattern: a repeated sub-pattern $((...))\{m,n\}$.
WITH	Cypher clause that projects and pipes rows between query parts.
Eager	Plan operator that materialises all rows before continuing.
DB hit	Unit of storage work reported by PROFILE.
Supernode	A node with a very large number of relationships.
GQL	ISO/IEC 39075:2024 standard property-graph query language.

Where to go next: authoritative resources

Resource	What it is	Why read it
SPARQL 1.1 Query Language (W3C)	The normative specification (w3.org/TR/sparql11-query)	Precise semantics for every operator used in Parts II–III
RDF 1.1 Primer · SPARQL 1.1 Update · SHACL (W3C)	Official primers and specs	Data model, updates and validation from the source
Wikidata Query Service + examples	query.wikidata.org	Hundreds of real queries over a huge live graph
Learning SPARQL — Bob DuCharme	Book (O'Reilly, 2nd ed.)	Readable, example-driven deep dive
Neo4j Cypher Manual	neo4j.com/docs/cypher-manual	Current syntax: QPP, SHORTEST, subqueries, indexes
Neo4j GraphAcademy	Free interactive courses	Hands-on Cypher, data modelling, GDS and GenAI
openCypher · ISO/IEC 39075 (GQL)	opencypher.org · ISO standard	Where the language is heading
Graph Databases — Robinson, Webber, Eifrem	Book (O'Reilly)	Property-graph modelling principles
Knowledge Graphs — Hogan et al.	Survey / book (ACM CSUR 2021, Springer)	The broad scientific picture across both models

 **TIP**
Best practice loop: read a spec section → reproduce its examples on the running dataset → break them on purpose → explain the result.

KEY TAKEAWAYS

Think in patterns. Query the shape.



RDF + SPARQL

Triples with global IRIs; patterns joined on shared variables; paths return endpoints; OPTIONAL, FILTER NOT EXISTS, GROUP BY; named graphs for provenance; SHACL for quality.



LPG + Cypher

Rich nodes and relationships; ASCII-art patterns; WITH pipelines and implicit grouping; MERGE on keys; QPP and SHORTEST; indexes, PROFILE and batching.



Both

Watch the semantic gaps: homomorphism vs. trails, unbound vs. null, sets vs. paths, global vs. local identity. Use each where it is strongest.

Next step: re-implement one real dataset from your own work in both models, and translate ten queries between them.